



DEPARTMENT OF ELECTRICAL
AND COMPUTER ENGINEERING

RAÚL JOSÉ MESTRE DINIS

Bsc in Electrical and Computer Engineering

TECHNOLOGY - AGNOSTIC MULTI-AGENT FRAMEWORK FOR DISTRIBUTED CONTROL OF CYBER-PHYSICAL PRODUCTION SYSTEMS

MASTER IN ELECTRICAL AND COMPUTER ENGINEERING
SPECIALIZATION IN SPECIALIZATION NAME

NOVA University Lisbon
September, 2026

TECHNOLOGY - AGNOSTIC MULTI-AGENT FRAMEWORK FOR DISTRIBUTED CONTROL OF CYBER - PHYSICAL PRODUCTION SYSTEMS

RAÚL JOSÉ MESTRE DINIS

Bsc in Electrical and Computer Engineering

Supervisor: Prof. Dr. André Dionísio Bettencourt da Silva Parreira Rocha
Assistant Professor, NOVA School of Science and Technology

Co-supervisor: António Luís Monte Pegado
PHD Student, NOVA School of Science and Technology

MASTER IN ELECTRICAL AND COMPUTER ENGINEERING
SPECIALIZATION IN SPECIALIZATION NAME

NOVA University Lisbon
September, 2026

Technology-Agnostic Multi-Agent Framework for Distributed Control of Cyber-Physical Production Systems

Copyright © Raúl José Mestre Dinis, NOVA School of Science and Technology, NOVA University Lisbon.

The NOVA School of Science and Technology and the NOVA University Lisbon have the right, perpetual and without geographical limitations, to archive and publish this dissertation in print or digital form, or by any other existing or future means, and to disseminate it through scientific repositories and to allow its copying and distribution for non-commercial educational or research purposes, provided that proper credit is given to the author and the publisher.

To my Avó Neta, who "wouldn't see me graduate".

ACKNOWLEDGEMENTS

The last five years have changed my life completely. It's not only about the incredible academic and professional development, but also about the social and personal growth. It started with pure responsibility, and ended with passion and a desire to keep learning and to do better. I now have the need to be bigger, and I came to realize the impact everyone who crossed my path had on this idea.

The NOVA FCT Professors were fundamental. Even though some made it easier than others, I cannot go without mentioning my supervisor, colleague, first boss, and friend, Professor André Dinísio Rocha. I'll start by thanking you for pushing me beyond what I believed I could do. All the opportunities, projects, and conversations you fostered have deeply inspired me to do and be more.

I also have to thank the Uninova RICS team: Miguel Arvana for leaving the classroom door open until the very end for us; if it wasn't for that day we shook hands on the internship, I wouldn't be here at all. My senior, Nelson Freitas, taught me to ping and answer forms. Mafalda Parreira and I talked for hours on end. Tiago Gouveia, also and forever known as *Dani*, was more than a colleague; he was the friend who went through everything else with me; António Pegado *pegou* in my ideas and fostered them as his, while opening the doors for me to work with him at Trash4Goods. Thank you for co-advising my thesis, your presence and support were crucial. It was a great pleasure working with every single one of you (Nelson, you still owe me and Tiago lunch).

It's hard to think how NOVA FCT became my second home, as I don't believe I would be as happy in any other university. I will never forget the solidarity from my colleagues, all the talks and initiatives that were always happening, and the sandwiches from *A Tia*. It was a truly remarkable time, and I hope everyone who comes after takes it in as I did.

Two figures in particular made everything possible for me: my father and first friend, José Armando, and my mother and partner in crime, Helena. Whatever I accomplish in my life will never be enough to repay you for all the hugs, conversations, meals, and love that you both invested in me. Everything will always be owed to you, as I am because of you. I was also blessed with three amazing sisters: Cláudia (*A Agressiva*), Janekas (*A Calma*), and Catarina (*A Maluca*). I hold close to my heart all the laughter and support. I also want to thank my Ti Isabel, Tia Nor, Patrícia, João Pedro and Tomé. I told my grandmother *Avó*

Neta she would still live to see me get my Master's, even though getting her to remember is another matter on its own. My *Madrinha* is also screaming *Divinal!* once she sees this.

António José Lopes Dias Freixo, also known as Ti Freixo, is one of my best friends, who I have with me every day. He made a big impact on me, placing me in this engineering path. I bet he is saying "*Brutel Manel*" right now. Just know I am doing my best.

I cannot go without mentioning the person who stays by my side no matter what happens: *A minha senhora*, Beatriz Nestor, my other half. I love you and appreciate the infinite support I can always count on. This marks the first step in our lives, and I promise to always do everything in my power to see you smile, starting with a *Milka Oreo*.

The first thing I learned when I arrived at the university was that "*O curso não se faz sozinho*". And it's true, just ask any of the *Felices los 4* group. Thank you Fernando, Gui, and Dani; all the all-nighters we pulled and the lying on the library lawn paid off. In this regard, I also need to thank Lourenço and Gabriel for crossing my path (*A Casa do Gui* is owed).

Credit is also due to Santos, Hugo, Túbal and Costa, my best friends. I can only imagine what we will do when we get older, ok (*pouca trapaça*). I was lucky to have so many brilliant people by my side. Thank you Raquel, Marta, Carlos, Maria João and Duarte for making these years way more enjoyable.

As my mother always says, "*Nem o Pai morre, nem a gente almoça!*" This means that we cannot stop, and it pairs perfectly with what my father says: "*Foi a Lena!*" My journey has only started, and with the way things are going, more than forty years of work follow. And oh boy, am I going in!

” *“Comme il faut!”*
— **António Freixo**
(Little Friend)

ABSTRACT

The Fourth Industrial Revolution (Industry 4.0) has introduced a paradigm shift in manufacturing, characterized by the integration of Cyber-Physical Systems (CPS) and the Internet of Things (IoT). To manage the increasing complexity and requirements for mass customization and personalized production, control architectures are evolving from centralized models to distributed systems that offer higher resilience, fault tolerance, and scalability. Multi-Agent Systems (MAS) have emerged as a robust solution for implementing such distributed control, enabling autonomy and cooperation among system components. However, the industrial implementation of MAS faces significant challenges regarding interoperability, integration with low-level automation devices, fault tolerance, and dependence on specific platform architectures. The author proposes a set of specifications for the creation of a framework that enables the development of platform agnostic, distributed multi-agent systems on CPSs in various hierarchy levels as a contribution towards solving these challenges. The Technology Agnostic Multi-Agent Framework (TAMAF), with its platform agnostic and distributed architecture, ensures that libraries developed for different programming languages and hardware configurations can operate seamlessly together, promoting interoperability and integration. In order to validate the specifications TAMAF encapsules a Python package for mid to high level Fog Computing a Java library for high-level Cloud Computing and a C++ library for low-level Edge Computing. Very promising results resulted from the conducted validation and stress tests, with three platforms containing different software and hardware settings collaborating seamlessly. It was also noted how the TAMAF Java implementation showed competitive results when compared with the widely adopted MAS development platform JADE for messaging speeds. This shows the potencial the TAMAF specifications have to implement interoperable, higly resilient and integrated MASs.

Keywords: Industry 4.0 · Multi-Agent Systems · Distributed Control · Platform Agnostic · Interoperability · Cyber-Physical Systems

RESUMO

A Quarta Revolução Industrial (Indústria 4.0) introduziu uma mudança de paradigma na manufatura, pautada pela integração de Sistemas Ciberfísicos (CPS) e da Internet das Coisas (IoT). Para gerir a crescente complexidade e os requisitos de personalização em massa e produção personalizada, as arquiteturas de controlo estão a evoluir de modelos centralizados para sistemas distribuídos que oferecem maior resiliência, tolerância a falhas e escalabilidade. Os Sistemas Multi-Agente (MAS) emergiram como uma solução robusta para implementar esse controlo distribuído, permitindo a autonomia e a cooperação entre componentes do sistema. Contudo, a implementação industrial de MAS enfrenta desafios significativos no que diz respeito à interoperabilidade, à integração com dispositivos de baixo nível, à tolerância a falhas e à dependência de arquiteturas de plataformas específicas. O autor propõe um conjunto de especificações para a criação de um framework que permite o desenvolvimento de sistemas multi-agente distribuídos e agnósticos em relação à plataforma, em vários níveis hierárquicos, como um contributo para a resolução destes desafios. O Technology Agnostic Multi-Agent Framework (TAMAF), com a sua arquitetura distribuída e agnóstica à plataforma, garante que bibliotecas desenvolvidas para diferentes linguagens de programação e configurações de hardware possam operar em conjunto de forma perfeitamente integrada, promovendo a interoperabilidade. De modo a validar as especificações, o TAMAF encapsula um pacote Python para Fog Computing de médio a alto nível, uma biblioteca Java para Cloud Computing de alto nível e uma biblioteca C++ para Edge Computing de baixo nível. Obtiveram-se resultados muito promissores a partir dos testes de validação e de stress conduzidos, com três plataformas de diferentes configurações de software e hardware a colaborar de forma contínua e integrada. Notou-se também que a implementação em Java do TAMAF apresentou resultados competitivos quando comparada com a plataforma de desenvolvimento de MAS amplamente adotada, a Jade, em termos de velocidade de mensagens. Isto mostra o potencial que as especificações TAMAF têm para implementar MAS interoperáveis, altamente resilientes e integrados.

Palavras-chave: Indústria 4.0 · Sistemas Multi-Agente · Controlo Distribuído · Agnóstica a Plataforma · Interoperabilidade · Sistemas Ciber-Físicos

DISCLOSURE OF USAGE OF GENERATIVE ARTIFICIAL INTELLIGENCE

The author, Raúl José Mestre Dinis, acknowledges the use of the following generative artificial intelligence tools in the development of this thesis. These technologies were utilized under strict supervision, and all AI-generated outputs were critically reviewed and verified for accuracy. The author accepts full responsibility for the validity and originality of the content, confirming that the use of these tools aligns with all institutional policies and standards of academic integrity.¹

Generative AI tools used: Gemini 2.5 Flash, Gemini 3.8 Flash, and Gemini 3.1 Pro.

Activities supported by Generative Artificial Intelligence Tools:²

Conceptualization _____		
☐ Idea generation	☐ Formulating research questions and hypotheses	☐ Hypothesis viability assessment
☐ Defining the research objective	☐ Feasibility assessment and risk evaluation	☐ Simulated debate and argument testing
Literature Review _____		
☐ Search and Discovery	☐ Concept Mapping and Systematization	☐ Gap Identification and Novelty Evaluation
☒ Literature summarization and synthesis	☒ Market/patent landscape analysis	☐ Cross-lingual literature comprehension
Methodology _____		
☐ Experimental Design Optimization	☐ Development of experimental or research protocols	☐ Methodological Instrument Selection

¹This declaration was generated using pdfL^AT_EX [1] and the L^AT_EX package aidisclose [2].
²This list extends the GAIDeT taxonomy [3].

Software Development and Automation

- | | | |
|--|--|--|
| ☒ Code Generation | ☒ Debugging and Repair | ☐ Algorithm design |
| ☒ Refactoring and Optimization | ☒ Process automation | ☐ Code documentation and comment generation |

Data Management

- | | | |
|---|--|--|
| ☐ Data collection | ☒ Data analysis | ☐ Synthetic data generation |
| ☐ Validation | ☐ Quantitative Plotting and Charting | ☐ De-identification and anonymization support |
| ☐ Data cleaning | ☐ Reproducibility and rerun checks | ☐ Audio-to-text transcription and diarization |
| ☐ Data curation and organization | ☐ Data labeling and annotation assistance | |

Visuals and Multimedia

- | | | |
|---|--|--|
| ☐ Synthetic Asset Generation | ☐ Image Enhancement and Editing | ☐ Diagrammatic and Schematic Design |
|---|--|--|

Writing and Editing

- | | | |
|--|---|--|
| ☐ Drafting Text | ☐ Formulation of conclusions | ☐ Citation formatting and bibliography management |
| ☒ Polishing and Editing | ☒ Tone Adjustment | ☐ Press releases and outreach materials |
| ☐ Abstract and Executive Summary Generation | ☐ Translation | ☐ Title Generation |

Ethics Review

- | | | |
|--|---|--|
| ☐ Bias analysis and discrimination assessment | ☐ Ethical risk analysis | ☐ Data confidentiality monitoring |
| | ☐ Monitoring compliance with ethical standards | |

Quality Assurance

- | | | |
|--|---|--|
| ☒ Simulated Peer Review | ☐ Identification of limitations | ☐ Publication support |
| ☐ Consistency checking against field trends | ☐ Publication strategy and journal selection | |

Additional comment:

The the core logic and the initial implementation built in Python were primarily the author's own work. Generative AI was used, in great part, to replicate and translate the code into both Java and C++. It was also very important in optimizing the code into the later versions of the three developed libraries that are reviewed in this document.

Additional comment: The data analysis process also used generative AI to help the author understand the mathematical equations that resulted and their meaning.

CONTENTS

Abstract	xi
Resumo	xiii
Disclosure of Usage of Generative Artificial Intelligence	xv
Contents	xvii
List of Figures	xix
List of Tables	xxi
Acronyms	xxv
1 Introduction	1
1.1 Scope and Motivation	1
1.2 Research Purpose and Goals	2
1.3 Dissertation Structure	3
2 State of the Art	5
2.1 From Industry 4.0 to Distributed Control	5
2.1.1 Emerging Paradigms	6
2.1.2 Challenges	8
2.1.3 From CPSs to CPPSs and ICPSs	9
2.2 Distributed Control Systems	11
2.2.1 Analysis	12
2.3 Multi-Agent Systems	13
2.3.1 FIPA Standards	15
2.3.2 MAS Development Platforms	18
2.3.3 Implementation Challenges	19
2.3.4 Platform-Independent MAS	20
2.4 Gap Analysis	21
3 Proposed Framework	23
3.1 Distributed Ecosystem Topology	24

3.1.1	Management Services	25
3.2	Agent Internal Composition	28
4	Architecture Specifications	31
4.1	Agent and Service Descriptions	33
4.2	MTS	34
4.2.1	MTS Internal Logic	35
4.3	EMA, ASDS, and Environment Management Services	37
4.3.1	ASDS	37
4.3.2	Management Ontologies	38
4.3.3	Local Management Services	39
4.3.4	External Management Services	43
4.4	Behaviors	45
4.4.1	Core Behavior Lifecycle	45
4.4.2	Complex Behaviors	46
4.5	AMS	48
4.5.1	Lifecycle States	48
4.5.2	Execution Kernel	49
5	Implementation	53
5.1	Common Implementation Specifications	54
5.2	Python Implementation	55
5.3	EMA Python Implementation	65
5.3.1	ASDS	65
5.3.2	Behaviors	66
5.4	Java Implementation	67
5.5	C++ Implementation	77
6	Validation and Testing	87
6.1	Stress Testing	87
6.1.1	Registration Time	88
6.1.2	Round-Trip Time	90
6.2	Functionality Use Case	92
6.3	Combined Results Analysis	99
7	Conclusions and Future Work	101
7.1	Future Work	101
	Bibliography	105

LIST OF FIGURES

2.1	CPS-IIoT interaction (adapted from Ghobakhloo et al. [21]).	7
2.2	The 3Cs (Computation, Communication, Control) in Cyber-Physical Systems (adapted from Sinha and Roy [5]).	9
2.3	Network models (adapted from Alves et al. [15]).	11
2.4	Distributed control architectures (adapted from Alves et al. [15]).	13
2.5	Classification of CPS control architectures (adapted from Alves et al. [15]). .	13
2.6	FIPA standard agent platform reference architecture (adapted from [10]). . .	16
2.7	FIPA Request Interaction Protocol (adapted from [40]).	17
2.8	FIPA Contract Net Interaction Protocol (adapted from [41]).	18
2.9	UML class diagram of API route models in OPACA (adapted from Acar et al. [16]).	21
3.1	TAMAF deployment structure for a specific technology.	24
3.2	TAMAF distributed environment ecosystem topology.	26
3.3	Agent internal composition Component Diagram.	29
4.1	Class diagram of the TAMAF specification data model.	32
4.2	Class diagram of AgentDescription, ServiceDescription, AgentID, and Address.	33
4.3	MTS and messaging class diagram.	35
4.4	MTS Synchronous Message Receiving Process.	36
4.5	MTS Asynchronous Message Receiving Process.	37
4.6	Agent Registration Process Sequence Diagram.	40
4.7	EMA Local Agents Persistence Service Activity Diagram.	41
4.8	Activity diagram of the heartbeat handling process in the Local Agents Persis- tence Service.	42
4.9	Activity diagram of the EMA multicast heartbeat handling process within the Environment Discovery Service.	44
4.10	State machine diagram of the agent lifecycle.	49
4.11	Kernel Loop Logic Activity Diagram.	51

4.12	Kernel Behavior Execution Logic Activity Diagram.	52
5.1	TAMAF Python, Java, and C++ deployments.	53
5.2	Python Agent Class Diagram.	57
5.3	Python MTS Class Diagram.	58
5.4	Python Behavior Class Diagram.	58
5.5	Python Behavior Receive Activity Diagram.	59
5.6	Python Behavior Execute Activity Diagram.	60
5.7	Python AMS Class Diagram.	61
5.8	Python AMS Kernel Behaviors Unblocking, Additions, and Removals Activity Diagram.	62
5.9	Python AMS Kernel Behavior Execution Logic Activity Diagram.	63
5.10	Python InitializationBehavior Logic Activity Diagram.	64
5.11	Java Agent Class Diagram.	68
5.12	Java MTS Class Diagram.	69
5.13	Java Behavior Class Diagram.	69
5.14	Java Behavior Receive Activity Diagram.	70
5.15	Java Behavior Execute Activity Diagram.	71
5.16	Java AMS Class Diagram.	72
5.17	Java AMS Kernel Behaviors Unblocking, Additions, and Removals Activity Diagram.	73
5.18	Java AMS Kernel Behavior Execution Logic Activity Diagram.	74
5.19	Java AMS Kernel End Execution Logic Activity Diagram.	75
5.20	Java InitializationBehavior Logic Activity Diagram.	76
5.21	C++ Agent Class Diagram.	78
5.22	C++ MTS Class Diagram.	79
5.23	C++ Behavior Class Diagram.	80
5.24	C++ Behavior Receive Activity Diagram.	80
5.25	C++ Behavior Execute Activity Diagram.	81
5.26	C++ AMS Class Diagram.	82
5.27	C++ Kernel Wakeup and Behavior Transfer Activity Diagram.	83
5.28	C++ Kernel Behavior Execution Logic Activity Diagram.	85
5.29	C++ InitializationBehavior Logic Activity Diagram.	86
6.1	Agent Registration Time vs. Network Scale.	89
6.2	Average Message Round-Trip Time vs. Network Scale.	90
6.3	Physical Fischertechnik Automation Kit and Agent Modules.	93
6.4	Functionality Use Case Sequence Diagram.	94
6.5	Functionality Use Case Components Diagram.	95
6.6	EMA Persistence Use Case Components Diagram.	97

LIST OF TABLES

2.1 Comparative evaluation of centralized, decentralized, and distributed control architectures (adapted from Alves et al. [15]). 14

6.1 Regression Models and Evaluation Metrics for JADE. 88

6.2 Regression Models and Evaluation Metrics for Python TAMAF. 88

6.3 Regression Models and Evaluation Metrics for Java TAMAF. 89

6.4 Regression Models and Evaluation Metrics for JADE. 91

6.5 Regression Models and Evaluation Metrics for Python TAMAF. 91

6.6 Regression Models and Evaluation Metrics for Java TAMAF. 91

LIST OF LISTINGS

6.1	Functionality Use Case Execution Log.	95
6.2	Functionality Use Case EMA Persistence Log.	98

ACRONYMS

AAS	Asset Administration Shell (pp. 9, 20, 102)
ACL	Agent Communication Language (pp. 16, 19, 20, 55)
AMS	Agent Management System (pp. xx, 16, 19, 21, 29, 31, 45, 48, 56, 57, 61–63, 67, 71–75, 77, 79, 82, 84)
AP	Agent Platform (pp. 15, 16, 19, 20)
ASDS	Agent and Service Directory Service (pp. 26–29, 31, 37–41, 43, 45, 65–67)
BDI	Belief-Desire-Intention (pp. 15, 18)
cfp	Call For Proposal (pp. 17, 18, 93)
CPPS	Cyber-Physical Production System (pp. 1, 3, 5, 8, 10, 15, 92, 99, 103)
CPS	Cyber-Physical System (pp. 5, 6, 8–12)
DF	Directory Facilitator (pp. 16, 19, 21)
DT	Digital Twin (pp. 6, 9, 20)
EMA	Environment Management Agent (pp. 25–29, 37–43, 45, 48, 49, 54–57, 61, 64–67, 75, 77, 84, 87, 88, 90, 93, 94, 97–100, 102)
FIPA	Foundation for Intelligent Physical Agents (pp. 1, 15–17, 19, 20, 26, 31, 34, 48, 54, 55, 93, 99)
FIPA-ACL	FIPA Agent Communication Language (pp. 1, 16, 19, 20, 31, 34, 38)
FMS	Flexible Manufacturing System (p. 6)
FoF	Factory of the Future (pp. 6, 10)
FreeRTOS	Free Real-Time Operating System (pp. 77–79, 82, 99)
FSM	Finite State Machine (p. 48)
GIL	Global Interpreter Lock (p. 91)

HMS	Holonic Manufacturing System (pp. 1, 7, 13, 14)
I4.0	Industry 4.0 (pp. 1, 5–9, 101)
ICPS	Industrial Cyber-Physical System (pp. 1, 5, 9–11)
IEC	International Electrotechnical Commission (p. 9)
IEEE	Institute of Electrical and Electronics Engineers (pp. 15, 20)
IIoT	Industrial Internet of Things (p. 6)
IoT	Internet of Things (pp. 6–8, 20)
JADE	Java Agent DEvelopment Framework (pp. xxi, 1, 3, 19, 21, 31, 87–91, 99)
JSON	JavaScript Object Notation (pp. 54, 68, 77, 79, 91, 101)
JVM	Java Virtual Machine (pp. 67, 89, 92)
KQML	Knowledge Query and Manipulation Language (p. 16)
LC	Laptop Computer (pp. 87, 93, 94, 97, 98)
MAS	Multi-Agent System (pp. 1–3, 5, 13–16, 18–21, 23, 24, 26, 87, 101, 102)
MQTT	Message Queuing Telemetry Transport (pp. 20, 101)
MTS	Message Transport System (pp. xx, 16, 29, 31, 34–36, 39, 43, 46, 50, 54, 56–59, 61, 67–71, 75, 77–79, 81, 82, 84, 86, 87, 101, 102)
NIO	Non-blocking Input/Output (p. 68)
OPACA	Open Language and Platform-Independent API for Containerized Agents (pp. 2, 20)
OPC-UA	Open Platform Communications Unified Architecture (pp. 2, 20, 102)
PADE	Python Agent DEvelopment platform (pp. 1, 19, 21)
PLC	Programmable Logic Controller (pp. 1, 20, 21, 25, 92, 102)
RAMI 4.0	Reference Architectural Model Industrie 4.0 (pp. 9, 102)
REST	Representational State Transfer (pp. 2, 20, 21)
RMS	Reconfigurable Manufacturing System (p. 6)
RMSE	Root Mean Square Error (pp. 88, 89, 91)
RTT	Round-Trip Time (pp. 88, 90, 91)
SCADA	Supervisory Control and Data Acquisition (p. 1)
SPADE	Smart Python Agent Development Environment (pp. 19, 21)

TAMAF	Technology-Agnostic Multi-Agent Framework (pp. <i>xx, xxi, 3, 23–25, 28, 29, 31, 33, 34, 37, 46, 49, 50, 53, 55, 56, 77, 87–93, 99–103</i>)
TCP	Transmission Control Protocol (pp. <i>35, 43, 54, 56, 78, 88, 89, 92, 99, 101</i>)
TCP/IP	Transmission Control Protocol / Internet Protocol (pp. <i>34, 90, 92</i>)
UDP	User Datagram Protocol (pp. <i>43, 66</i>)
UML	Unified Modeling Language (pp. <i>20, 25, 29, 36, 41, 42, 50, 61, 72, 93</i>)
XMPP	Extensible Messaging and Presence Protocol (p. <i>19</i>)

INTRODUCTION

This chapter establishes the foundational context for this dissertation by addressing its scope, motivation, and primary objectives. Furthermore, it outlines the structural organization of the dissertation, providing a roadmap for the subsequent chapters.

1.1 Scope and Motivation

The rapid evolution of Industry 4.0 (I4.0) and Cyber-Physical Production Systems (CPPSs) has fundamentally reshaped modern manufacturing. Traditional centralized automation hierarchies, characterized by rigid Programmable Logic Controller (PLC) structures and enterprise-level Supervisory Control and Data Acquisition (SCADA) systems, are increasingly unable to satisfy the demands of high-mix, low-volume production, mass customization, and dynamic reconfiguration [4, 5]. To address these limitations, decentralized and modular control paradigms have emerged, pushing decision-making to the network edge and transforming shop-floor equipment into autonomous, cooperative entities [6, 7].

Multi-Agent Systems (MASs) have long been recognized as a prime architectural framework for realizing decentralized control in Industrial Cyber-Physical Systems (ICPSs) based on Holonic Manufacturing Systems (HMSs) [8, 9]. Applied across diverse domains such as manufacturing, logistics, and supply chain management, MASs offer pronounced advantages in flexibility, robustness, and complex task execution through the collaboration of autonomous agents. By encapsulating computational logic, physical hardware abstractions, and social behaviors into software agents, MAS frameworks enable flexible negotiation, real-time adaptability, and fault-tolerant operations. Furthermore, standard specifications established by the Foundation for Intelligent Physical Agents (FIPA), most notably the FIPA Agent Communication Language (FIPA-ACL), provide formal semantics for inter-agent interaction [10, 11].

Despite decades of academic research and industrial trials, the deployment of MASs across heterogeneous industrial environments remains severely constrained by technology dependencies and architectural centralization bottlenecks [12]. Established MAS frameworks, such as the Java Agent DEvelopment Framework (JADE) [13] and the Python

Agent DEvelopment platform (PADE) [14], are strictly coupled to specific programming languages and runtime execution environments.

Furthermore, conventional MAS platforms rely on centralized container architectures, with central message transport brokers and management systems. This centralized topology introduces single points of failure, restricts horizontal scalability across multi-subnet industrial facilities, and creates communication bottlenecks when scaling to hundreds of concurrent edge nodes [15, 16]. However, decentralizing MAS architectures introduces substantial complexity, rendering fully distributed paradigms challenging to deploy, synchronize, and maintain [17].

In response to these limitations, significant research has focused on overcoming the lack of native interoperability across heterogeneous frameworks. One notable approach utilizes Open Platform Communications Unified Architecture (OPC-UA) as a platform-independent, service-oriented architecture to decouple high-level software logic from physical hardware specifications [18]. Another major advancement is the Open Language and Platform-Independent API for Containerized Agents (OPACA) framework, which deploys modular agents inside Docker or Kubernetes containers, leveraging Representational State Transfer (REST)-based web services to deliver a language-agnostic, microservice-based multi-agent architecture [16].

1.2 Research Purpose and Goals

Although these initiatives make significant strides toward interoperability, they often rely on heavyweight containerization technologies that remain unsuitable for bare-metal edge devices and resource-constrained microcontrollers. Furthermore, they frequently retain centralized control topologies by depending on persistent central brokers and cluster orchestration engines. Given the decentralized paradigm of MASs, enabling autonomous agents to execute directly on the shop floor within a distributed architecture capable of supporting highly customized production systems, represents a pressing research challenge.

Existing MAS platforms, regardless of their architectural maturity, offer proven utility within specific operational domains. Consequently, the core research challenge lies in eliminating runtime and platform dependencies by establishing an architectural framework that unifies heterogeneous agents, irrespective of their underlying implementation language or target hardware platform.

These considerations culminate in the fundamental research question:

What architectural design enables the creation of interoperable agents across heterogeneous technologies and platforms?

Given this research question, the author formulates the following hypothesis: A framework grounded in abstract specifications, standardized communication semantics, and asynchronous execution logic, where centralized services are distributed across

multiple agent groups, will minimize computational complexity and eliminate single points of failure. By preserving direct peer-to-peer messaging and avoiding centralized brokers or heavyweight runtimes, this distributed architecture allows autonomous agents to execute natively across the industrial automation hierarchy, bridging resource-constrained edge microcontrollers with high-level fog infrastructures.

1.3 Dissertation Structure

This dissertation is organized into seven chapters, progressing systematically from foundational literature review and architectural specification to empirical validation and concluding reflections:

Chapter 2 presents a comprehensive review of the state of the art, positioning this research within the historical evolution from traditional multi-agent systems to modern decentralized paradigms. Key concepts including centralized, decentralized, and distributed architectures, Industrial Cyber-Physical Systems, and MASs are examined to identify existing limitations and motivate the research.

Chapter 3 introduces the conceptual Technology-Agnostic Multi-Agent Framework (TAMAF), formulated directly to address the formulated research question.

Chapter 4 provides an exhaustive specification of the TAMAF architecture, elaborating on its design choices, core components, service interactions, data models, and provided tools.

Chapter 5 translates these theoretical specifications into practice by detailing three reference implementations across Python, Java, and C++. It examines runtime-specific characteristics, cross-platform interoperability mechanisms, and deployment strategies across distinct levels of the industrial automation hierarchy.

Chapter 6 validates the framework through two empirical evaluations: a physical laboratory CPPS testbed demonstrating functional cross-platform interoperability, and a high-concurrency benchmark stress-testing TAMAF against an established JADE deployment to assess communication latency and registration scalability.

Chapter 7 concludes the dissertation by synthesizing the research findings, reviewing the fulfillment of the defined objectives, and outlining directions for future investigation.

STATE OF THE ART

This chapter presents a review of the state of the art across domains relevant to this dissertation. The discussion is structured into three primary thematic areas followed by a gap analysis:

It begins with an examination of the impact of I4.0, focusing on emerging production and computing paradigms as well as their underlying challenges. Next, the application of Cyber-Physical Systems (CPSs) in industrial scenarios is explored through ICPs and CPPs. The concepts of centralized, decentralized, and distributed control systems are then presented, providing a comparative analysis of their capabilities, use cases, and limitations. MASs are subsequently reviewed, including supporting standards, development platforms, and implementation challenges. Finally, the latest research efforts regarding agent interoperability solutions and their practical applications are presented.

These insights establish a solid foundation for understanding the current state of manufacturing control systems and the pivotal role of MASs in distributed decision-making and control, thereby motivating the architectural design and implementation proposed in this dissertation.

2.1 From Industry 4.0 to Distributed Control

The fourth industrial revolution (I4.0) was first introduced in 2013 when the German federal government launched a strategic initiative to modernize the nation's manufacturing sector. It marks a fundamental paradigm shift in industrial history, following three preceding revolutions: the first characterized by the mechanization of production through steam power; the second driven by industrial electrification and mass production; and the third enabled by shop-floor computerization and automated control systems [4].

I4.0 fosters innovation through the seamless convergence of physical, digital, and virtual systems over industrial networks. This integration has fostered the emergence of novel technologies and manufacturing paradigms, including [19]:

- **Cyber-Physical Systems.** CPSs integrate physical processes with computational algorithms, enabling autonomous communication, mutual control, and environmental

interaction.

- **Internet of Things.** The Internet of Things (IoT) enables ubiquitous networking that digitalizes and interconnects industrial processes, creating an integrated smart environment.
- **Big Data Analytics.** The systematic aggregation, processing, and analysis of large-scale industrial data to optimize manufacturing performance and decision-making.
- **Simulation.** The deployment of computational models for predictive analysis and the generation of virtual representations of industrial processes, equipment, products, and human operators.
- **Additive Manufacturing.** The implementation of high-performance, decentralized production techniques prioritizing geometrical flexibility and rapid customization.

Another key concept introduced within I4.0 is the Factory of the Future (Factory of the Future (FoF)), or Smart Factory. Anumbe et al. [20] define it as “*a highly connected, very intelligent, and broadly digitized production facility*”. The FoF encompasses advanced capabilities such as automated production planning and scheduling, dynamic inventory management, smart logistics, and collaborative product design, superseding traditional manufacturing facilities reliant on rigid, legacy workflows.

Recent literature has increasingly identified CPSs, IoT, and Digital Twins (DTs) as the primary technological pillars of I4.0 [21]. The rapid adoption of IoT within industrial domains established the Industrial Internet of Things (IIoT). IIoT enhances manufacturing traceability through real-time monitoring supported by continuous data streams generated across all shop-floor assets [22]. In parallel, CPSs integrate physical and computational operations in real time, enabling rapid adaptation to environmental changes and serving as the foundational infrastructure for digitalization. IIoT and CPSs are deeply complementary building blocks of modern industrial architectures. Figure 2.1 illustrates how CPSs leverage IIoT protocols to communicate across heterogeneous sensors, actuators, edge controllers, and cloud services.

2.1.1 Emerging Paradigms

With these technologies, manufacturing has undergone progressive paradigm shifts: from **Mass Production**, which prioritized high volumes of standardized goods, to **Mass Customization**, which introduced product variety tailored to market segments. The latest evolution is **Personalized Production**, driven by demand for highly customized, low-volume products. To accommodate such extreme variability and shorten time-to-market, conventional Flexible Manufacturing Systems (FMSs) are increasingly superseded by Reconfigurable Manufacturing Systems (RMSs). RMS architectures facilitate the rapid reconfiguration of hardware and software components to scale production capacity and adapt functionality in real time [23].

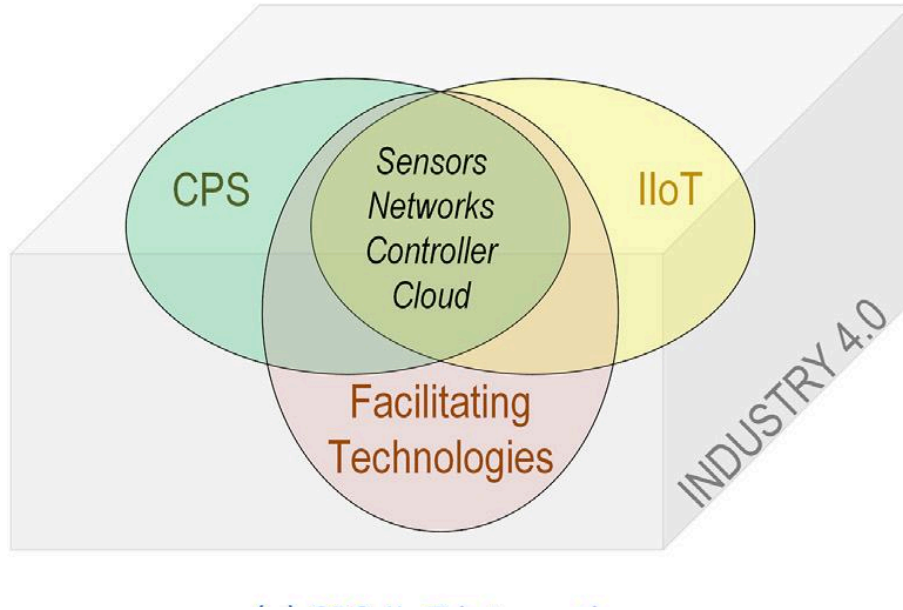


Figure 2.1: CPS–IIoT interaction (adapted from Ghobakhloo et al. [21]).

Concepts such as HMSs emerged as solutions to this new wave of manufacturing.

The foundational concept of holonic systems was proposed in 1967 by Arthur Koestler, who defined a holon as an autonomous, cooperative entity that is simultaneously a whole and a part of a larger hierarchy. Translated into the manufacturing domain in the 1990s, a holon denotes an autonomous and cooperative building block comprising an information-processing module and an optional physical processing resource capable of transforming, transporting, storing, and validating artifacts. HMSs emerged directly in response to the inability of traditional Computer Integrated Manufacturing to accommodate dynamic reconfiguration and high product variability [6].

2.1.1.1 Computing Paradigms

In [24] and [25], the authors evaluate the core computing paradigms supporting I4.0 deployments: Cloud Computing, Fog Computing, and Edge Computing.

Cloud Computing is a centralized architecture that delivers computing services including compute, storage, databases, and software over wide-area networks. It utilizes remote data centers to provide virtually unlimited storage and processing power, making it ideal for long-term data analysis and warehousing. However, because data must travel to remote servers, it suffers from high latency and bandwidth consumption, making it less suitable for real-time applications.

Fog Computing acts as a decentralized intermediary layer between the cloud and IoT devices, effectively extending cloud capabilities to the edge of the network. It handles data processing, storage, and networking closer to the data source, which reduces the volume

of data sent to the cloud and improves response times. This paradigm is characterized by high heterogeneity and geographical distribution, supporting location awareness and mobility.

Edge Computing relocates processing power, storage, and control logic directly to field assets and CPPS nodes. By executing computation at the network edge, it minimizes latency and conserves bandwidth, making it ideal for deterministic, time-critical control loops. Unlike the cloud, edge nodes operate with bounded processing and storage resources, frequently offloading non-time-critical historical aggregation to fog or cloud servers.

2.1.2 Challenges

Extensive digitalization within I4.0 has rendered manufacturing systems substantially more efficient and autonomous, while simultaneously increasing their operational and architectural complexity. Nevertheless, technologies such as CPSs and IoT are instrumental in mitigating these challenges. Herrera-Vidal et al. [17] analyze the underlying technological and organizational drivers of complexity in I4.0 environments. Industrial architectures must actively address complexity along several critical dimensions [20]:

- **Mass Customization.** The operational flexibility required to tailor manufactured goods to unique customer specifications without sacrificing throughput.
- **Data Management and Transparency.** The systematic acquisition, semantic integration, and real-time processing of operational data across the production lifecycle.
- **Integration and Interoperability.** The ability of heterogeneous machines, sensors, and enterprise systems to seamlessly interconnect and exchange structured data across disparate communication protocols.
- **Decentralized Decision-Making.** The capability of distributed autonomous nodes to maintain local context awareness and execute operational decisions independently.
- **Human-Machine Interaction.** The establishment of intuitive, collaborative workflows that safely synchronize human operators and automated assets.

The primary obstacles encountered when addressing these challenges include [17]:

- **Integration and Interoperability Bottlenecks.** The deployment of emerging I4.0 technologies in the absence of universally adopted semantic standards exacerbates system integration complexity.
- **Data Ingestion and Analytics Overhead.** Ingesting, reconciling, and analyzing high-velocity, multi-modal industrial data streams introduces substantial computational and architectural overhead.
- **Cybersecurity Vulnerabilities.** Pervasive connectivity expands the attack surface, exposing interconnected operational technology (OT) to sophisticated cyber threats.

- **Organizational and Change Management Pressures.** Technological adoption alters workforce requirements and organizational dynamics, directly influencing the agility of the manufacturing enterprise.

Another central challenge in I4.0 is governing the digital transformation of manufacturing workflows [26]. A cornerstone of this transformation is the DT. According to Onaji et al. [27], a DT is *“a virtual replica of its physical asset built mainly of structural and behavioural models for basic control, monitoring and evaluation of its performance”*, serving as an indispensable foundation for asset digitalization. The development of DTs has historically suffered from fragmented modeling approaches; however, this is being systematically resolved by the Asset Administration Shell (AAS). Standardized under International Electrotechnical Commission (IEC) 63278, the AAS serves as the core implementation construct of the Reference Architectural Model Industrie 4.0 (RAMI 4.0). It provides a vendor-neutral digital representation for physical and software assets by exposing standardized APIs, submodels, and semantic definitions that mirror real-time operational status, capabilities, and parameters across industrial ecosystems [28].

2.1.3 From CPSs to CPPSs and ICPSs

The Cyber-Physical System paradigm predated the formal declaration of Industry 4.0, having been introduced in 2006 by Helen Gill at the US National Science Foundation. Sinha et al. [5] synthesize Gill’s definition of a CPS as *“large scale, interconnected, complex, heterogeneous, and networked systems whose operations are monitored, coordinated, controlled, and integrated by computing, communication, and components interacting with the physical environment”*. The authors conceptualize the three foundational pillars of a CPS, Computation, Communication, and Control (the 3Cs), as illustrated in Figure 2.2.

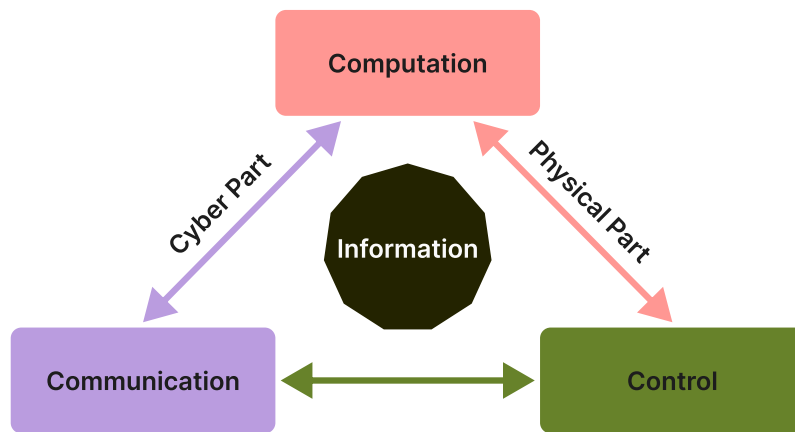


Figure 2.2: The 3Cs (Computation, Communication, Control) in Cyber-Physical Systems (adapted from Sinha and Roy [5]).

While CPS research has expanded across diverse sectors, their specialization in industrial environments gave rise to the ICPS. Zhang et al. [9] define an ICPS as a CPS augmented with industrial-grade cyber representation and communication infrastructures.

Unlike conventional CPSs, which generally govern localized physical equipment, ICPSs facilitate enterprise-scale collaboration by interconnecting distributed production facilities, supply-chain partners, and remote services. Another conceptual class is the CPPS, which denotes a manufacturing system composed of autonomous, collaborating CPSs. Müller et al. [29] emphasize embedded intelligence as the principal differentiator between standard CPSs and CPPSs. Consequently, CPPSs operate as adaptive, self-configuring, and self-organizing manufacturing networks prioritizing energy efficiency, resource optimization, and throughput. Within the FoF vision [20], Sinha et al. [5] argue that the primary objective of smart factories is to establish *"large, flexible, robust, reliable, and secure networks of CPSs to deal with the inherent challenges so that the on-demand products can be manufactured flexibly, resource and energy efficiently, and profitably"*. To achieve this, industrial CPS nodes must exhibit high reliability, modularity, and maintainability. Specifically, Sinha et al. identify four essential capabilities:

- **Interoperability.** The ability of all CPS nodes to exchange information and execute coordinated actions across heterogeneous networks.
- **Virtualization.** The continuous synthesis of physical shop-floor states into high-fidelity digital models and simulation environments.
- **Modularity.** The architectural flexibility to group, reconfigure, or decouple CPS nodes into autonomous functional subsystems.
- **Distributed Control.** The capability of individual nodes to process sensory feedback and execute localized control actions independently, driving system-wide optimization from the bottom up.

2.1.3.1 Implementation Challenges

In a comprehensive review, Zhang et al. [9] delineate key research challenges and future directions for ICPS deployments across three primary domains: information acquisition, communication networking, and advanced control.

Regarding **information acquisition and data analytics**, a fundamental challenge lies in formulating sensing topologies that balance energy conservation, data integrity, and network traffic, particularly across resource-constrained wireless sensor nodes. Furthermore, the resulting industrial datasets exhibit extreme heterogeneity across modalities and sampling rates, challenging conventional data fusion pipelines and raising security concerns regarding sensitive proprietary data traversing multi-tiered networks.

In **communication networks**, the hierarchical and heterogeneous topology of ICPSs struggles to guarantee bounded latency for closed-loop deterministic control. Wireless and fieldbus reliability is further compromised by electromagnetic interference and physical attenuation in harsh industrial environments, inducing packet loss and jitter.

Advanced control is hindered by the intractability of constructing precise first-principles models in highly non-linear, large-scale systems [17], as well as the theoretical complexity of bridging continuous physical dynamics with discrete cyber events.

To resolve these bottlenecks, Zhang et al. outline three strategic research trajectories. First, integrated co-design frameworks must simultaneously synthesize monitoring, communication, and control policies, departing from isolated local optimizations toward plant-wide optimization. Second, systematic migration strategies must accommodate legacy equipment, enabling legacy brownfield assets to integrate with service-oriented ICPS architectures without sacrificing determinism. Third, the co-existence of physical safety risks and cyber vulnerabilities demands unified AI-assisted frameworks capable of conducting concurrent functional safety evaluations and cyber threat mitigations.

2.2 Distributed Control Systems

The evolution of CPSs and their specialized industrial variants has catalyzed advanced manufacturing control architectures. Decentralized computational capacity enables CPS nodes to transcend passive command execution, empowering them to formulate control decisions autonomously. By tightly coupling physical processes with cyberspace, modern control systems can adopt diverse structural paradigms. Alves et al. [15] classify the primary topological models underpinning industrial control, emphasizing distributed strategies.

The underlying network topology determines how messages and state variables are exchanged across interconnected nodes. The two predominant models, illustrated in Figure 2.3, are the peer-to-peer model and the client-server model.

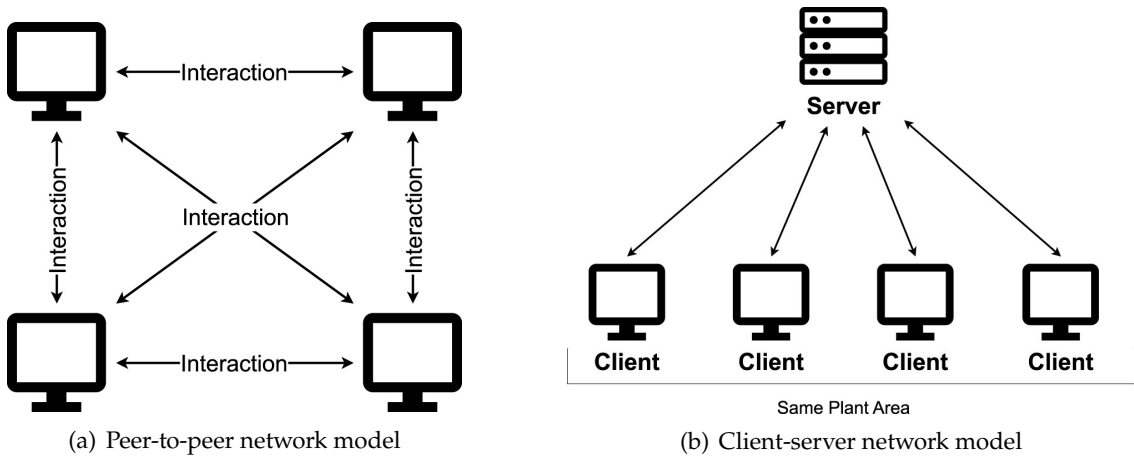


Figure 2.3: Network models (adapted from Alves et al. [15]).

The **peer-to-peer network model** treats all participating nodes as hierarchically equal, enabling direct communication between any pair of nodes. Operating without centralized arbitration, it serves as the foundational communication pattern for distributed control systems. Conversely, the **client-server network model** establishes a strict structural

hierarchy where a designated central server arbitrates communication and data storage; clients interact exclusively through the server. Because client nodes rely entirely on the central server for operational coordination, this topology introduces an acute single point of failure.

The architectural topology of a CPS dictates how computational authority and control logic are organized. These structures are profoundly influenced by the underlying network models. The **Centralized architecture**, illustrated in Figure 2.5, directly mirrors the client-server pattern: a single central controller executes control algorithms, stores system state, and dispatches setpoints to subordinate field devices. While centralized architectures simplify management and data consistency, they enforce rigid operational dependencies and suffer from catastrophic failure modes. If the central controller fails, the entire system collapses.

The **Decentralized architecture** represents a hybrid compromise, frequently structured as an aggregation of local client-server clusters interconnected via peer-to-peer links, as illustrated in Figure 2.4. While offering superior scalability and flexibility compared to centralized systems, decentralized topologies retain localized vulnerabilities. Cluster heads act as intermediary gateways; consequently, the compromise or hardware failure of these coordinating nodes can isolate entire production cells or trigger cascading system disruptions [7].

The **Distributed architecture** is established upon a pure peer-to-peer topology wherein the manufacturing facility is decomposed into autonomous, collaborating subsystems that coordinate dynamically to achieve global operational goals, as shown in Figure 2.5. Nodes establish localized communication neighborhoods to prevent network congestion, ensuring high scalability. Distributed control exhibits exceptional fault tolerance and resilience: the overall system remains operational unless a vast majority of nodes fail concurrently. Partial hardware outages or localized network partitions do not halt production, allowing self-organizing algorithms to dynamically reroute workflows around failed components without manual intervention.

2.2.1 Analysis

Selecting an appropriate control architecture requires balancing functional and non-functional requirements against operational constraints. The emergence of personalized production and dynamic reconfigurability has elevated the importance of scalability, fault tolerance, software modularity, and communication agility. Alves et al. [15] provide a comparative evaluation of centralized, decentralized, and distributed architectures across these core criteria, as summarized in Table 2.1.

Despite these advantages, distributed architectures introduce substantial algorithmic and operational challenges, most notably myopic decision-making. Myopic decision-making arises when an autonomous node executes localized optimizations based exclusively on immediate sensor data, inadvertently degrading global system performance.

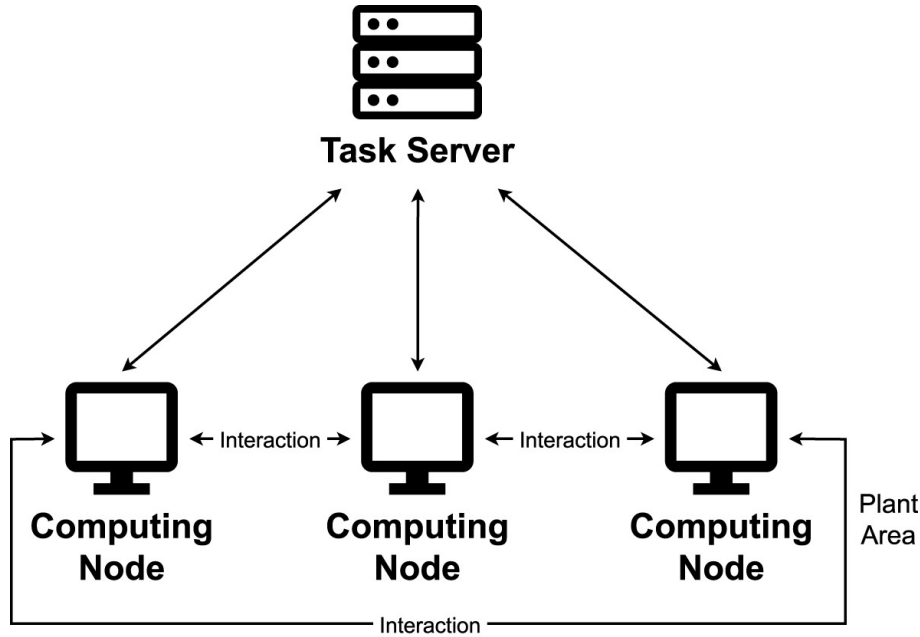


Figure 2.4: Distributed control architectures (adapted from Alves et al. [15]).

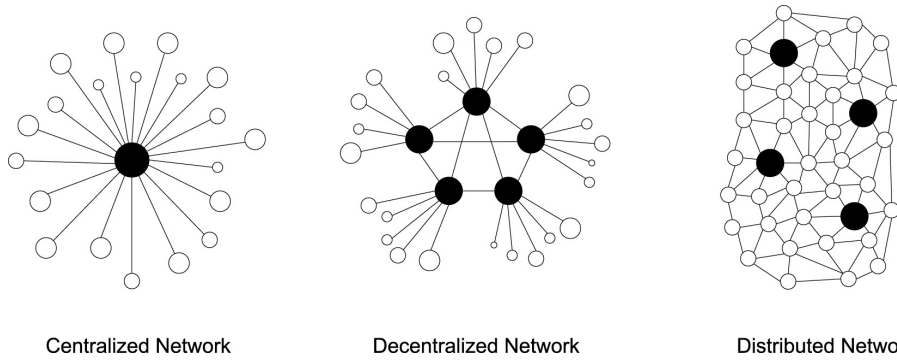


Figure 2.5: Classification of CPS control architectures (adapted from Alves et al. [15]).

Mitigating myopic behavior without re-introducing centralized bottlenecks is a major focus of recent control research. For example, Ma et al. [30] propose the anarchic manufacturing paradigm, a fully distributed framework wherein nodes self-organize without hierarchical supervision. Individual manufacturing assets pursue localized optimization objectives designed to collectively satisfy emergent plant-level schedules. This design enhances flexibility and operational resilience in disturbance-filled environments. By deferring commitment to specific machine assignments until execution time, nodes adapt dynamically, mitigating localized myopia and improving global scheduling performance.

2.3 Multi-Agent Systems

Within distributed control paradigms, MASs have emerged as the leading technology for modeling, executing, monitoring, and evaluating distributed manufacturing logic [7], widely recognized as the predominant implementation methodology for HMS architectures

Table 2.1: Comparative evaluation of centralized, decentralized, and distributed control architectures (adapted from Alves et al. [15]).

Feature	Centralized	Decentralized	Distributed
Scalability	Low	Moderate	High
Fault Tolerance	Susceptible to single point of failure	Localized single points of failure at cluster heads	Resilient; requires concurrent failure of majority of nodes
Reliability	Low; dependent on central node	Moderate; bounded by cluster head reliability	High; distributed redundantly across multiple nodes
Flexibility	Low	Moderate	High
Collaboration	Dictated strictly by central authority	Coordinated via regional cluster coordinators	Emergent; autonomous peer-to-peer negotiation across all nodes
Communication Flow	Rigid vertical flow through central hub	Hybrid flow channeled through cluster gateways	Dynamic peer-to-peer mesh; optimized local data sharing

[31].

The industrial viability of an HMS hinges upon two complementary principles: autonomy and cooperation. Autonomy empowers an individual holon to synthesize and execute local operational schedules without continual supervisory intervention, whereas cooperation ensures that multiple holons negotiate mutually feasible execution commitments to fulfill plant-wide goals. Consequently, the research community predominantly utilizes MASs as the implementation substrate for holonic control. Crucially, while agents are exclusively software entities, holons typically encapsulate both the software control algorithm and the underlying physical manufacturing hardware [6].

An agent is an autonomous, computational entity situated within an environment, capable of interacting socially with peer agents to achieve defined objectives [32]. A distributed architecture constitutes a MAS when multiple autonomous agents, each possessing bounded perception and computational capacity, collaborate to solve complex, non-deterministic tasks. Pulikottil et al. [8] summarize the four core properties of an intelligent agent:

- **Autonomy.** The capability to control internal states and overt behavior without direct external intervention.
- **Reactivity.** The capability to perceive environmental disturbances and respond predictably in real time to satisfy operational goals.

- **Proactiveness.** The capability to exhibit goal-directed behavior by taking initiative rather than merely reacting to environmental changes.
- **Social Ability.** The capability to interact with other agents using structured communication protocols to negotiate, delegate, and execute shared workflows.

In manufacturing automation, agents are mapped directly to physical entities such as workpieces, processing stations, automated guided vehicles, and storage buffers, reflecting the intrinsic structural symmetry between agents and CPPS assets [33]. Consequently, manufacturing MASs are typically deployed across fog and edge tiers based on latency and processing requirements. Deploying agents closer to physical machinery yields lower communication latency and higher fieldbus bandwidth, but requires accommodating constrained compute and memory footprints [34].

To model sophisticated deliberative behavior, MAS architectures frequently adopt the Belief-Desire-Intention (BDI) paradigm. A BDI agent is conceptualized as a rational entity characterized by three primary mental attitudes: Beliefs, representing informational models of the environment and internal states; Desires, representing target states of affairs the agent seeks to attain; and Intentions, representing committed execution plans [35]. The BDI execution model decouples goal selection from procedural plan execution. Guided by external sensor percepts, the agent continuously reconciles its belief base, evaluates applicable pre-compiled plan libraries, and schedules intention stacks to trigger low-level actuation in real time.

2.3.1 FIPA Standards

Having evolved over several decades, MASs have undergone extensive international standardization. Founded in the late 1990s and subsequently integrated into the Institute of Electrical and Electronics Engineers (IEEE) Computer Society, the FIPA established formal standards to accelerate the commercialization of interoperable agent technologies. FIPA specifications standardize agent lifecycle management, abstract reference architectures, communication ontologies, and interaction protocols. This section reviews the core FIPA specifications governing industrial MAS implementations [10].

2.3.1.1 Agent Platform

Because underlying computing architectures vary widely, FIPA defined the Agent Management Reference Model to standardize the topological organization of agent systems. The runtime infrastructure is formalized as an Agent Platform (AP), comprising four foundational components [36] [37], as illustrated in Figure 2.6:

- **Agent:** The computational entities executing application logic within the environment.

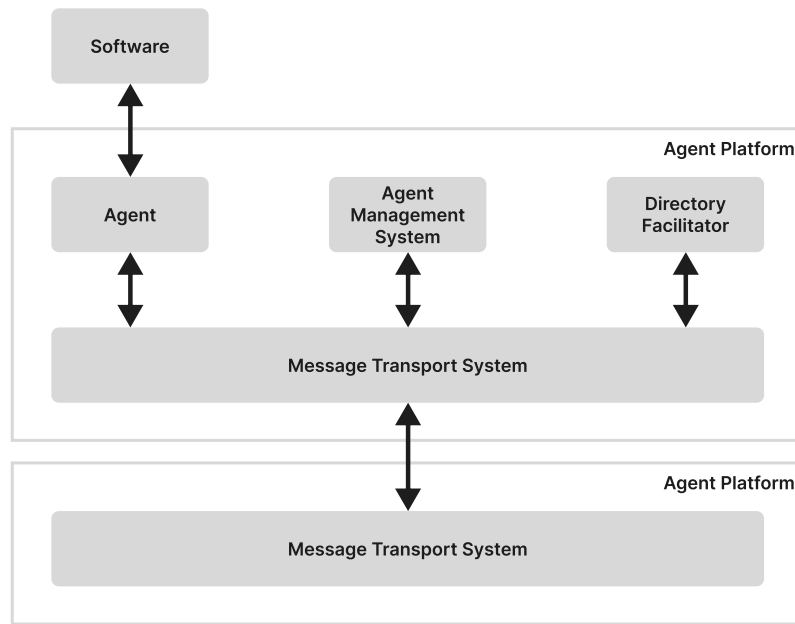


Figure 2.6: FIPA standard agent platform reference architecture (adapted from [10]).

- **Agent Management System (AMS):** The mandatory supervisory authority governing platform lifecycle operations, including agent instantiation, registration, suspension, migration, and termination. Every agent must register with the AMS to obtain a globally unique Agent Identifier.
- **Directory Facilitator (DF):** The optional directory component providing yellow-pages service discovery, enabling agents to publish their capabilities and dynamically discover external services.
- **Message Transport System (MTS):** The communications backbone managing the routing and physical transport of structured messages between agents across local and distributed platforms.

Importantly, FIPA is not specific on how to implement each component, leaving room for the application of different and new technologies.

2.3.1.2 Agent Communication Language

Robust communication mechanisms are fundamental to decentralized agent coordination. Establishing a standardized Agent Communication Language (ACL) has consequently represented a primary objective of the MAS community. An early foundational protocol was the Knowledge Query and Manipulation Language (KQML), which pioneered the decoupling of communicative intent (performatives) from application-specific payload content [38]. This speech-act framework was subsequently formalized in the FIPA-ACL standard, which ensures that heterogeneous APs interpret messages consistently through formal communicative acts, content ontologies, and interaction protocols [11].

for distributed task allocation and market-oriented negotiation, as illustrated in Figure 2.8. An Initiator seeking to delegate a computational or physical task broadcasts a call for proposals (cfp) encoding task specifications, evaluation criteria, and bidding deadlines [41]. Candidate Participant agents evaluate the cfp against localized capacity and respond with either a refuse act or a binding propose act specifying cost, duration, and preconditions. Following deadline expiration, the Initiator evaluates competing bids against its objective function, transmitting an accept-proposal act to the selected contractor(s) and a reject-proposal act to the remaining bidders. The selected contractor executes the contracted workflow and concludes the session with an inform-done, inform-result, or failure message.

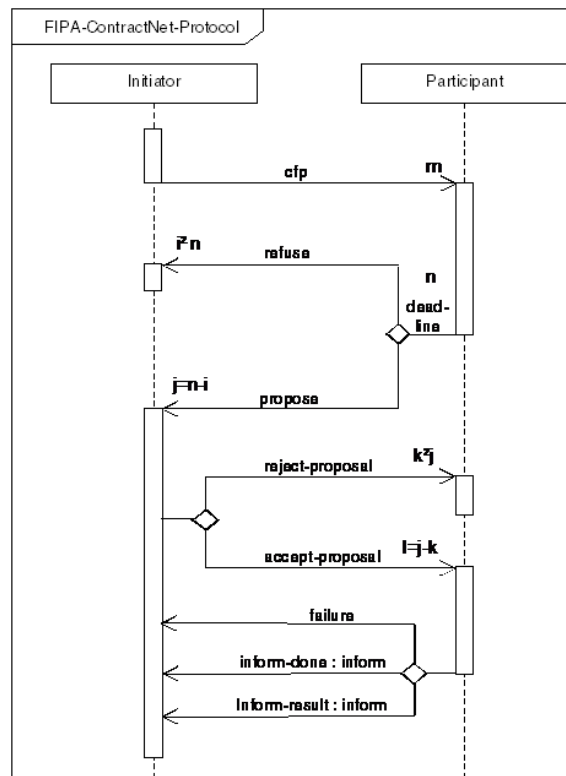


Figure 2.8: FIPA Contract Net Interaction Protocol (adapted from [41]).

2.3.2 MAS Development Platforms

Multiple platforms for modeling, testing, and implementing MASs have been developed over the years. Wrona and Dunin-Kępicz [12] classify MAS development frameworks into general-purpose platforms, which provide comprehensive environments for modeling, instantiation, and simulation, and specialized platforms dedicated to niche applications such as cognitive architectures or machine learning exploration. To remain focused on industrial control, this review concentrates on general-purpose MAS frameworks that provide native execution containers, protocol stacks, and inspection tooling without mandating specific internal cognitive reasoning engines (e.g., pure BDI). The most prominent

frameworks fulfilling these criteria are:

JADE is a mature, open-source framework facilitating the development of FIPA-compliant MASs [13]. Originally designed to validate early FIPA standards, JADE provides a peer-to-peer agent environment supporting asynchronous FIPA-ACL messaging. The runtime environment consists of distributed containers that collectively constitute an AP, abstracting underlying operating systems and hardware from enterprise servers down to mobile devices. To optimize resource utilization, JADE employs a cooperative thread-per-agent execution model: tasks, modeled as Behaviour classes, are scheduled cooperatively within a single Java thread per agent, avoiding the overhead of heavy thread-per-task concurrency.

PADE is an open-source Python framework designed to streamline distributed MAS development on embedded hardware [14, 42]. Built upon the Twisted asynchronous networking engine, PADE relies on an event-driven, single-threaded reactor model well suited for resource-constrained platforms such as single-board computers (e.g., Raspberry Pi). PADE provides FIPA-ACL compliance, centralizes lifecycle tracking through an AMS container, and supplies a web-based graphical management interface.

Smart Python Agent Development Environment (SPADE) is an asynchronous Python MAS framework that utilizes the Extensible Messaging and Presence Protocol (XMPP) for inter-agent communication [43]. By leveraging XMPP, SPADE provides presence notifications, push messaging, and native human-to-agent interaction capabilities while maintaining a compact memory footprint suitable for edge gateways [44]. SPADE agents execute asynchronous behaviors (e.g., cyclic, periodic, and one-shot routines) driven by Python’s native `asyncio` event loop. Agents authenticate against an external XMPP broker and utilize integrated DF services for capability advertising.

Despite its maturity, JADE remains widely utilized in industrial and academic automation due to its established robustness, scalability, and protocol determinism [45]. JADE features inspection and administration tools, including the graphical *Sniffer* agent for monitoring inter-agent ACL traffic and the *Introspector* for inspecting internal behavior queues. Furthermore, JADE supports third-party extensions for web-service integration, rule-based inference, and alternative message transports. Recent research continues to expand its testing capabilities for ultra-large-scale agent deployments; for instance, Briola et al. [46] developed *SimJADE*, combining JADE runtime semantics with the *PeerSim* overlay simulator to evaluate thousands of concurrent agents.

2.3.3 Implementation Challenges

While MASs represent the predominant paradigm for architecting distributed industrial control, their widespread industrial adoption remains constrained by several persistent engineering hurdles [8]:

Standardization remains an enduring challenge. Although FIPA established early architectural and communicative baselines, practical industrial deployment requires

standardization across lower automation tiers. To address this, IEEE Standard 2660.1-2020 [47] defined recommended engineering practices for integrating software agents with low-level industrial controllers and physical processes. In parallel, recent literature investigates marrying MAS architectures with AASs. Jirsa et al. [48] formalized AAS-compliant agent descriptions to achieve standardized semantic interoperability with DTs. Similarly, Komesker et al. [49] introduced a semi-heterarchical design utilizing AAS models to orchestrate resilient material flows across modular production lines. Furthermore, Palanca et al. [43] proposed IoT Artifacts as structured abstractions to seamlessly integrate smart sensors into agent environments.

Ontology is indispensable for ensuring semantic comprehension across disparate agent implementations, particularly in heterogeneous, platform-independent systems. Gan et al. [11] extended the standard FIPA-ACL ontology to enhance semantic validation and message reasoning across collaborative engineering workflows.

Integration with Low-Level Industrial Hardware presents a severe practical bottleneck due to mismatched execution paradigms: high-level asynchronous agent platforms cannot natively execute on deterministic, scan-based PLCs or low-power microcontrollers. To bridge this divide, Oliveira et al. [50] formulated a modular integration framework inserting an asynchronous middleware layer between physical equipment and the MAS. This middleware abstracts industrial protocols, including OPC-UA, WebSockets, and Message Queuing Telemetry Transport (MQTT), translating raw fieldbus variables into structured ACL messages while preserving a FIPA-compliant, high-level agent architecture.

2.3.4 Platform-Independent MAS

The architectural isolation between proprietary MAS runtime environments and external web services has encouraged research into platform-agnostic architectures.

Seitz et al. [18] introduced an OPC-UA-based platform-independent MAS framework. Rather than restricting agents to a shared language runtime, each agent exposes its communication endpoints and semantic capabilities via standard OPC-UA information models, functioning as a service-oriented architecture. This strategy allows agents written in C++, Python, or Java to interoperate across heterogeneous platforms. Using standard OPC-UA address-space interfaces to bridge high-level agent logic with physical shop-floor controllers effectively decouples distributed software control from underlying proprietary hardware.

Another major milestone is the OPACA proposed by Acar et al. [16]. In OPACA, each agent is encapsulated within a lightweight container (e.g., Docker or Kubernetes), establishing modular, independently deployable software services. Agent coordination is achieved via REST microservice APIs over HTTP, ensuring complete programming language and platform neutrality. Figure 2.9 illustrates the Unified Modeling Language (UML) class model of the OPACA architecture: an AP hosts containerized instances provisioned from standardized images, with container endpoints managing internal agent

lifecycles and REST route registrations.

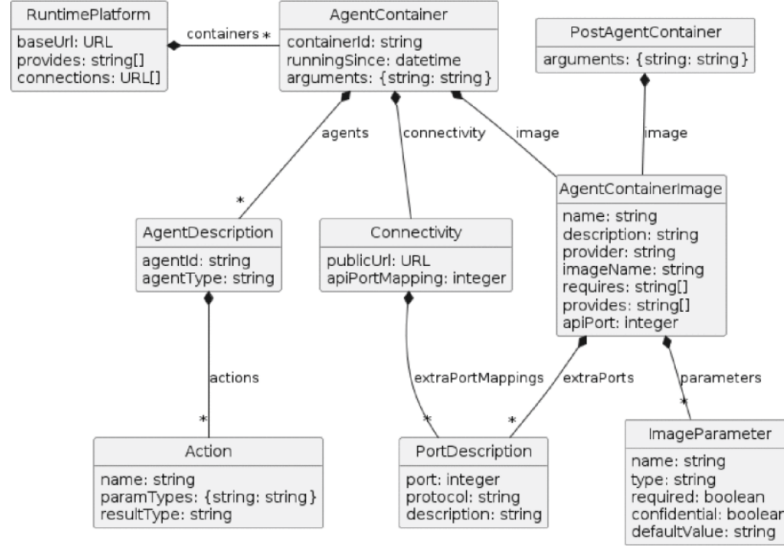


Figure 2.9: UML class diagram of API route models in OPACA (adapted from Acar et al. [16]).

2.4 Gap Analysis

The reviewed literature highlights three fundamental architectural gaps hindering the widespread industrial deployment of multi-agent control systems:

1. **Low Interoperability Across Heterogeneous MAS Implementations.** Existing frameworks (such as JADE, PADE, and SPADE) are tightly bound to specific programming languages and proprietary runtime message brokers, preventing cross-framework agent communication without complex custom bridging.
2. **Impeded Integration with Low-Hierarchy Field Devices.** Conventional MAS platforms exhibit substantial runtime footprints that cannot execute natively on deterministic, resource-constrained microcontrollers and PLCs, leaving shop-floor hardware disconnected from agent-level coordination.
3. **Architectural Vulnerability from Centralized Platform Topologies.** Despite promoting decentralized control in theory, conventional MAS platforms depend on centralized message transport brokers and registry services (AMS/DF), introducing catastrophic single points of failure and limiting horizontal scaling across multi-subnet industrial facilities.

Synthesizing these hurdles exposes the fundamental research problem addressed in this dissertation: the need for a unified, technology-agnostic multi-agent architecture that is sufficiently lightweight to execute deterministically on resource-constrained edge microcontrollers, sufficiently robust to deploy across fog and cloud infrastructures, standardized

to bridge software and physical hardware across all automation tiers, and completely free of centralized single-point bottlenecks.

PROPOSED FRAMEWORK

This chapter provides an overview of the proposed architectural solution to the research question formulated in Chapter 1. It establishes a high-level conceptual framework by delineating the system’s composition and its fundamental enabling services, while an exhaustive technical specification is presented in Chapter 4.

To address the limitations identified in Chapter 2, this work introduces the design specifications of the TAMAF. Through this framework, the proposed architecture provides a robust, adaptable solution capable of satisfying the evolving demands of modern distributed industrial automation.

The TAMAF specifications establish a technology-agnostic framework for the development and deployment of MASs. This architectural independence allows the framework to bridge heterogeneous computational environments, enabling systems with disparate hardware and software configurations to seamlessly coexist, cooperate, and achieve common operational goals.

Rather than enforcing a specific runtime container or rigid programming language bindings, TAMAF defines agents through abstract interfaces, management services, and standardized message structures as the sole mandatory components required to construct MASs. Consequently, an agent can be implemented in any target language, such as Java on fog servers, Python in the cloud, and C++ on resource-constrained edge devices, while preserving strict semantic and operational interoperability.

Minimizing computational complexity was a primary design driver, as low-level embedded devices must be capable of operating with the framework. Agents are therefore designed to be as lightweight as possible, with native services restricted strictly to those essential for robust and reliable operation.

As a foundational framework, TAMAF only provides the core infrastructure required to construct and maintain MASs. To achieve a functional deployment, the developer must utilize the abstract tools and objects provided by a specific framework implementation to construct the agents and instantiate the application-specific logic for their target use case. Consequently, each agent represents a hybrid entity comprising core TAMAF infrastructure and user-defined domain logic. Figure 3.1 illustrates this division for a deployment across specific hardware and software configurations. Multiple such deployments, realized across

disparate technologies, collectively form an ecosystem of communicative, interoperable, cooperative, and resilient agents.

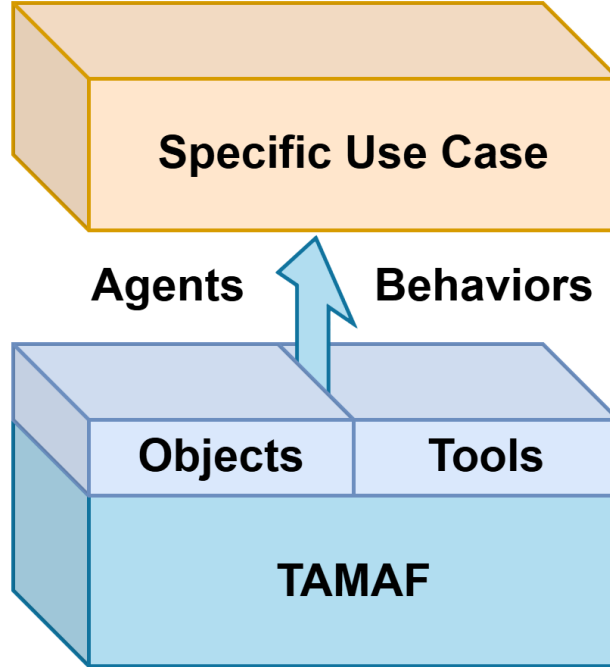


Figure 3.1: TAMAF deployment structure for a specific technology.

The remainder of this chapter details the architectural topology of the proposed TAMAF ecosystem. This architecture is organized into two distinct analytical perspectives: the macro-level distributed environment and the micro-level individual agent. The first perspective examines the distributed environment, defining the role, purpose, and operational capabilities of each constituent element, as well as the design of their core management services. The second perspective examines the internal architecture of the individual agent, detailing its execution model, communication subsystems, and persistence mechanisms. Furthermore, it delineates the internal composition of the agent, formalizing the responsibilities of each constituent module.

3.1 Distributed Ecosystem Topology

Topologically, the TAMAF ecosystem is organized into an architectural hierarchy comprising environments, platforms, and agents. Agents represent autonomous entities that execute discrete tasks to accomplish shared or individual objectives. Peer-to-peer collaboration constitutes a core pillar of MASs, as agents continuously request services, negotiate parameters, share state, and coordinate actions during task execution.

Because the framework is technology-agnostic, agents can operate across heterogeneous hardware and software environments, adapted to the specific physical constraints and application requirements of their host nodes. The coupled physical execution hardware

and underlying software stack hosting one or more agents is formally designated as a platform. A platform exists within the ecosystem only while hosting at least one active agent; because a platform possesses no standalone TAMAF management infrastructure, maintaining an empty platform serves no architectural purpose. In practice, platforms can range from resource-constrained industrial PLCs to industrial computers, virtual machines, or enterprise cloud instances.

Environments encompass one or more platforms, representing the subsequent tier of architectural abstraction. To enable autonomous operation without central coordination, TAMAF eliminates centralized message brokers and distributes directory and lifecycle management responsibilities across local environments throughout the ecosystem. An environment can span a single platform hosting an isolated agent or an entire industrial manufacturing facility comprising numerous heterogeneous platforms and hundreds of agents. Each environment is autonomously responsible for hosting local directory and management services for its resident agents, federating these records with neighboring environments, and maintaining the global ecosystem alive and synchronized. A TAMAF ecosystem is formally defined as the collection of these interconnected environments.

Within each environment, governance responsibilities are encapsulated into a specialized agent designated as the Environment Management Agent (EMA). Standard agents actively participate in distributed governance as well, hosting dedicated persistence services that assist the EMA in monitoring environment integrity. The following subsection provides a more detailed description of these services and responsibilities. Architecturally, an EMA shares the identical internal composition and execution semantics of any standard agent, distinguished solely by the additional management services it hosts. Consequently, any agent with sufficient computational capability can assume the role of an EMA, if needed. It is critical to emphasize that the EMA does not act as a centralized message broker; agents interact directly through decentralized point-to-point communications. Nevertheless, for lifecycle management and directory resolution, all agents within an environment must initially register with their local EMA. Peer EMAs interconnect across network boundaries via a multicast communication model, enabling dynamic discovery and mutual state synchronization without centralized intervention.

Figure 3.2 presents a UML component diagram illustrating an ecosystem composed of two environments, each hosting an EMA, distributed across three platforms and four agents. In this topology, dashed arrows denote unicast point-to-point communications between peer agents, whereas solid arrows denote multicast coordination channels among EMAs.

3.1.1 Management Services

When designing a fully decentralized architecture, one must first analyze the fundamental services traditionally provided by centralized coordinators, seeking either to distribute them across the network or eliminate dependency on them entirely. Classical

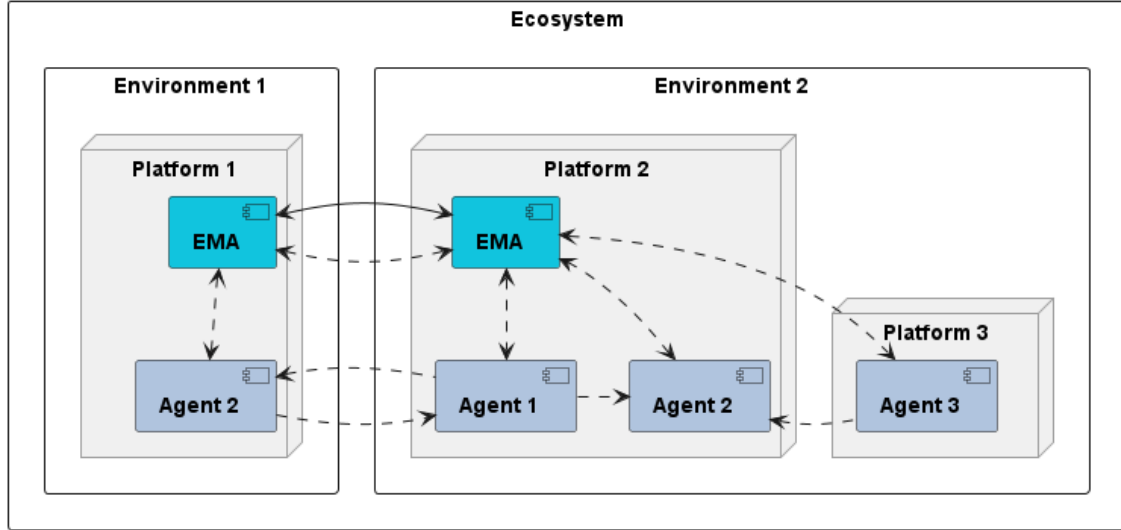


Figure 3.2: TAMAF distributed environment ecosystem topology.

MAS architectures, specifically FIPA-compliant ones, as analyzed in Chapter 2, rely on:

- **A centralized Directory Service:** A registry of active agents and their capabilities that maintains the real-time operational state of the system, updated upon every topology modification.
- **An Ecosystem Lifecycle Management Service:** Conventionally tasked with tracking agent availability, updating the Directory Service, and coordinating agent registration, deregistration, and mobility. In certain architectures, this coordinator also supervises task execution and behavior scheduling.
- **A centralized Message Broker Service:** Used to route inter-agent messages. Whenever an agent initiates a transmission, it must route the request through the broker, which subsequently forwards the payload to the intended recipient agents.

TAMAF distributes these services across the multiple environments that constitute the ecosystem. These services are provided solely by agents, including EMAs. In this context, each environment provides three foundational categories of services: the Agent and Service Directory Service (ASDS), local management services, and external management services.

The ASDS maintains an active registry not only for the agents resident within its local environment, but also for all known external environments and their registered agents. This mechanism establishes a federated, distributed knowledge base wherein each environment maintains a local synchronized replica of the global ecosystem state. When an agent requires a specific capability or service, it transmits a directory lookup query directly to the ASDS hosted by its local EMA.

3.1.1.1 Local Management Services

Three local management services govern the lifecycle and state consistency of agents within an environment, ensuring structured onboarding, active health monitoring, and real-time synchronization of ASDS records:

- **Agent Registration Service:** Hosted by the EMA, this service manages network address allocations within the environment and processes agent registration, modification, and deregistration requests in the ASDS without disrupting concurrent agent operations.
- **Local Agents Persistence Service:** An EMA-hosted service tasked with monitoring the liveness of all registered local agents and maintaining real-time directory synchronization regarding their operational states and published capabilities.
- **Local EMA Persistence Service:** Embedded within every single agent, including the EMA, this service verifies the continuous availability of the local EMA and periodically transmits heartbeat updates encapsulating the agent's current state. The EMA, in this case, directly updates its state inside the ASDS.

The Local EMA Persistence Service is vital because passive monitoring by the EMA is insufficient to detect silent node failures; the EMA requires periodic state assertions dispatched by resident agents via *KeepAlive* heartbeat messages.

To preserve fault tolerance in a decentralized architecture, the unexpected failure of an EMA must not compromise environment stability. Consequently, eligible agents previously registered with the failed EMA possess the capability to instantiate a replacement EMA or occupy its place by providing the necessary management services. The cooperative design of the EMA's Local Agents Persistence Service and the agents' Local EMA Persistence Service enables the newly instantiated EMA to rapidly reconstruct environment state and resume coordination seamlessly.

The type of agent deployed is frequently determined by host platform capabilities. Consequently, an agent executing on a low-level microcontroller might lack the memory or networking stack required to host an EMA, precluding local failover capabilities. To mitigate this constraint, the Local EMA Persistence Service can be selectively disabled on resource-constrained devices, allowing them to participate as lightweight leaf agents while offloading governance tasks to more capable nodes.

3.1.1.2 External Management Services

To discover peer environments and propagate distributed directory updates without central discovery registries, each EMA executes two complementary external management services:

- **Environment Discovery Service:** Provides dynamic multicast discovery mechanisms enabling environments to identify peer nodes and announce local state updates across the network.
- **Environments Persistence Service:** Maintains a real-time synchronized registry within the ASDS containing the metadata and agent descriptors of all federated environments across the ecosystem.

In the absence of centralized registries, environments dynamically discover peer nodes by dispatching periodic *KeepAlive* multicast announcements. All active environments join a designated multicast group to listen for incoming announcements. However, because connectionless multicast protocols provide no delivery guarantees, transmitting large payloads over multicast significantly increases packet fragmentation and loss. Full environment descriptors, encompassing numerous agent specifications and service metadata, can become substantial, undermining network efficiency and robustness if continuously broadcast. To mitigate this, multicast *KeepAlive* messages transmit only a lightweight update timestamp reflecting the most recent modification to the local ASDS. Neighboring environments detect state changes by comparing timestamps, subsequently issuing reliable point-to-point (unicast) requests to retrieve complete environment descriptors only when discrepancies arise.

3.2 Agent Internal Composition

Without centralized message brokers and external execution schedulers, each agent is autonomously responsible for managing its own business logic and network communications. This decentralized paradigm requires agents to process synchronous communication events while concurrently executing internal domain tasks. Furthermore, the Local EMA Persistence Service must execute independently of application-specific logic, safeguarding the operational persistence and liveness of the agent within the environment.

To satisfy these constraints with minimal computational overhead, the TAMAF specifications establish a decoupled agent architecture that cleanly isolates internal business logic from networking services. Under this model, domain behaviors execute asynchronously while network transport operations are handled synchronously, yielding at least two concurrent execution threads per agent.

To realize this design, an agent is structured around four core architectural modules:

- **Agent Description:** An encapsulating data structure containing the agent's identity metadata, including its unique name, network address locator, and the declarative descriptors of its published services.
- **Behaviors:** Autonomous execution units encapsulating domain-specific logic and tasks, scheduled and evaluated asynchronously by the agent's internal engine.

- **Message Transport System (MTS):** An independent communication subsystem managing point-to-point network messaging to and from the agent, buffering incoming and outgoing payloads to decouple synchronous network operations from asynchronous behavior execution.
- **Agent Management System (AMS):** The supervisory core of the agent, hosting the behavioral execution kernel that coordinates asynchronous behavior scheduling and handles the agent lifecycle.

Agents, described by an Agent Description, each integrate both an MTS and an AMS, hosting a set of behaviors executed by the AMS kernel and communicating via the MTS. The architectural relationships among these internal modules are illustrated in the UML component diagram in Figure 3.3.

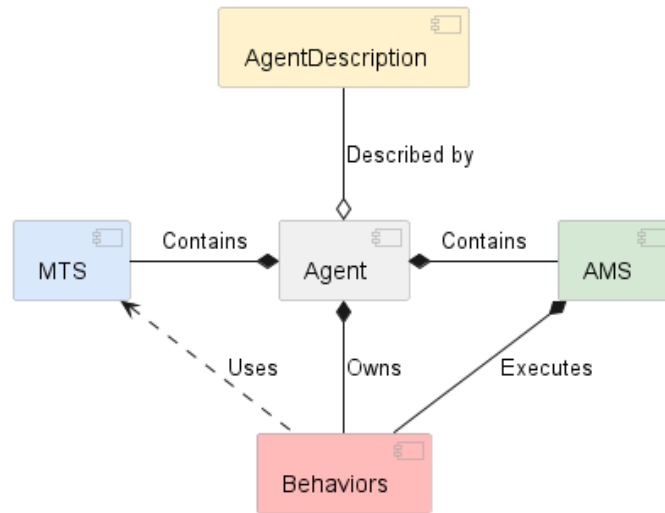


Figure 3.3: Agent internal composition Component Diagram.

As established previously, an EMA operates as a standard agent, built upon the identical architectural foundation. To deliver directory and discovery services, it hosts an instance of the ASDS that remains completely decoupled from the agent's core execution pipeline, thereby preserving internal architectural symmetry.

Chapter 4 provides a comprehensive technical specification of each module, interaction protocol, and development abstraction comprising the TAMAF architecture.

ARCHITECTURE SPECIFICATIONS

This chapter provides a highly detailed overview of the architectural concepts introduced in Chapter 3. The underlying purpose of the architectural design choices is discussed throughout, alongside descriptions of the core tools provided by the framework, delivering a comprehensive description of the TAMAF architecture. The chapter begins with an overview of the core data model and components, followed by the formal specification of TAMAF concepts, detailing how agents and services are represented, structured, and queried. The subsequent section extensively details the MTS architecture and messaging workflows. Next, the ASDS and the sustaining environment management services are examined across two operational scopes: Internal and External services. The structural design, capabilities, and execution semantics of behaviors are then analyzed. Finally, the chapter details how the AMS governs the agent lifecycle and coordinates behavior execution asynchronously. These specifications enable the practical deployment of TAMAF agents across heterogeneous technologies, with concrete implementations detailed in Chapter 5.

Thirteen classes constitute the TAMAF specification data model. A class diagram depicting these classes and their relationships is illustrated in Figure 4.1. This model extends the agent composition diagram introduced in Figure 3.3, incorporating classes for messaging, agent and service descriptions, and lifecycle management. Each is detailed in its respective section throughout the remainder of the chapter.

All classes were designed based on the FIPA standards [36], while drawing operational insights from established agent development platforms such as JADE. Agent communication complies with the FIPA-ACL standard, with targeted extensions introduced to support framework-specific requirements. Grounding the TAMAF specification in established standards is a core methodological choice: these proven foundations provide a validated baseline for the data model, while accelerating adoption through data structures already familiar to multi-agent systems developers.

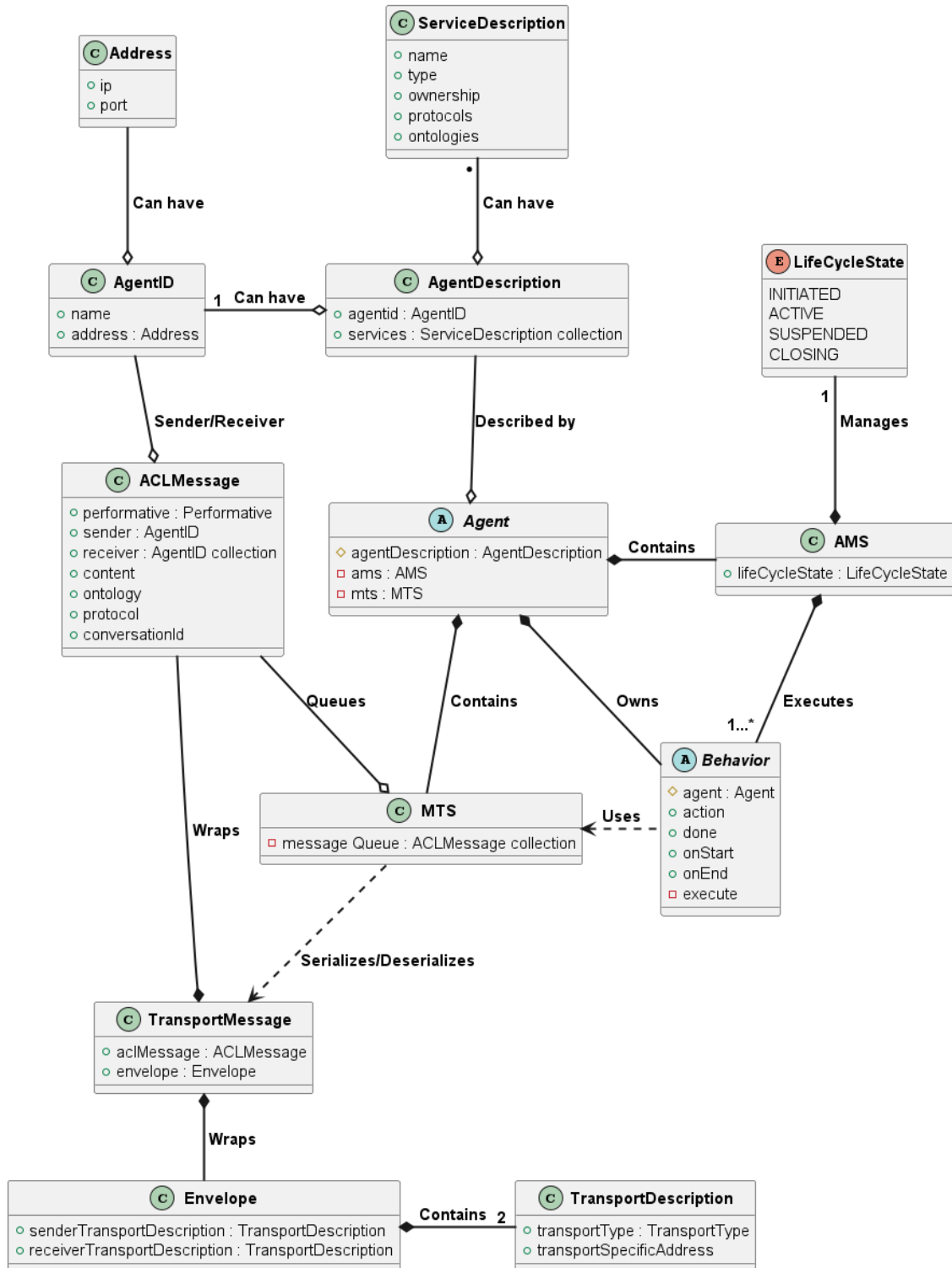


Figure 4.1: Class diagram of the TAMAF specification data model.

4.1 Agent and Service Descriptions

As introduced in Chapter 3, the foundational structure of an agent within this framework is comprehensively captured by the `AgentDescription` class. This core entity acts as a central repository for all essential metadata, formally defining how the agent is identified, how it can be accessed over the network, what capabilities it provides, and how those capabilities can be invoked.

To achieve this, `AgentDescription` is partitioned into two functional segments: identification and capability description. Identification is encapsulated in an `AgentID`, which manages the agent's unique identity via a designated name and specifies its network endpoint through an `Address` object that pairs an IP address with a specific port. The capabilities provided by an agent are defined as services and represented by a collection of `ServiceDescription` objects. Each `ServiceDescription` specifies a unique service offered by the agent, detailing its service type, ownership credentials, supported communication protocols, and the domain ontologies required to interpret exchanged data. The relationships among these classes are depicted in Figure 4.2.

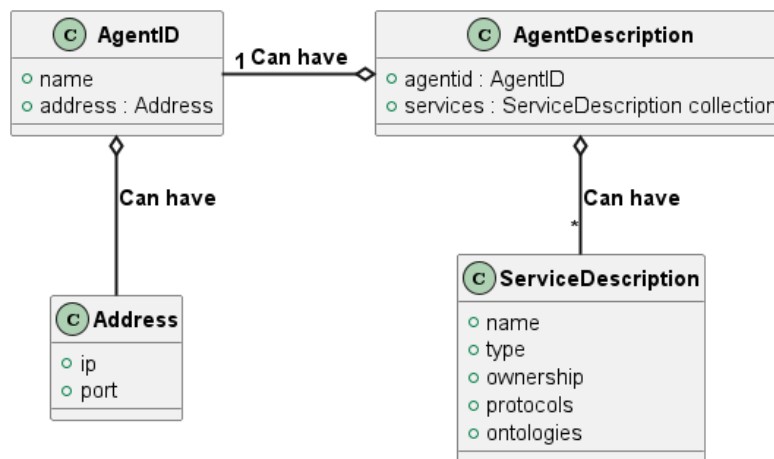


Figure 4.2: Class diagram of `AgentDescription`, `ServiceDescription`, `AgentID`, and `Address`.

The TAMAF specification exposes these data structures as part of the public interface, enabling agents across diverse scenarios to reference target agents or services using `AgentID`, `AgentDescription`, and `ServiceDescription` objects. The `AgentDescription` class was intentionally designed with minimal mandatory constraints, ensuring that the representation of agents and services can be tailored to the distinct contextual requirements of disparate architectural environments. A practical scenario underscoring this flexibility arises during service discovery: depending on the operational context, a discovery request may range from an exact query targeting a single, explicitly identified agent to a broad filter matching any agent offering a specific service category. Supporting this spectrum requires query parameters with varying levels of granularity and, consequently, an adaptable `AgentDescription`.

These descriptive structures establish the foundational addressing and service model required by the MTS, detailed in the following section.

4.2 MTS

As established in Chapter 3, task execution and communication are designed as fully decoupled subsystems. This architectural separation enables an agent to process synchronous inbound messaging from external agents while executing internal task logic asynchronously. In this context, the MTS is the core subsystem responsible for sending, receiving, and routing messages, both synchronously with external agents and asynchronously with internal behavior routines. This section is divided into two parts: the first presents the structural class model, and the second details the operational logic of the MTS.

To support FIPA-ACL-compliant messaging, all classes within this subsystem, with the exception of the MTS manager itself, are derived from the FIPA standard specifications. These specifications provide essential communicative semantics, including communicative acts (performatives), content ontologies, and conversation identifiers. Every message mandates a unique sender and one or more receivers, each identified by an `AgentID`. The TAMAF specification also supports complex interaction patterns through standardized interaction protocols, such as FIPA Contract Net and FIPA Request.

The TAMAF MTS and messages class structure consists of five connected components, as illustrated by the class diagram in Figure 4.3.

If the MTS is the manager, the `ACLMessage` is the fundamental message object. It represents the FIPA-ACL-compliant message object, encapsulating fields detailing the previously defined communication semantics: the communicative intent (`performative`), the sender's `AgentID`, a list of receiver `AgentIDs`, the payload (`content`), language, encoding, ontology, protocol, `conversationId`, and additional conversational metadata such as `replyBy`, `replyWith`, and `inReplyTo`. It is important to note that a new performative was added to the FIPA-ACL specified intentions called `KEEPALIVE`; it is highly relevant to these specifications, as the next subsection will explain. This is the element that is sent and received by the MTS to communicate.

The MTS, however, does not send the raw `ACLMessage`. This object is wrapped inside a `TransportMessage`, which is serialized and deserialized across the transport channel. Alongside the `ACLMessage`, an `Envelope` containing relevant information about the transport system used by the sender and the receiver is also part of the `TransportMessage`. This transport system information is structured using a `TransportDescription`, representing a wire-level routing description, with a `TransportType` referencing a communication model (such as Transmission Control Protocol / Internet Protocol (TCP/IP)), and a `TransportSpecificAddress` representing the address when contextualized with the `TransportType` (for TCP/IP: "x.x.x.x:y").

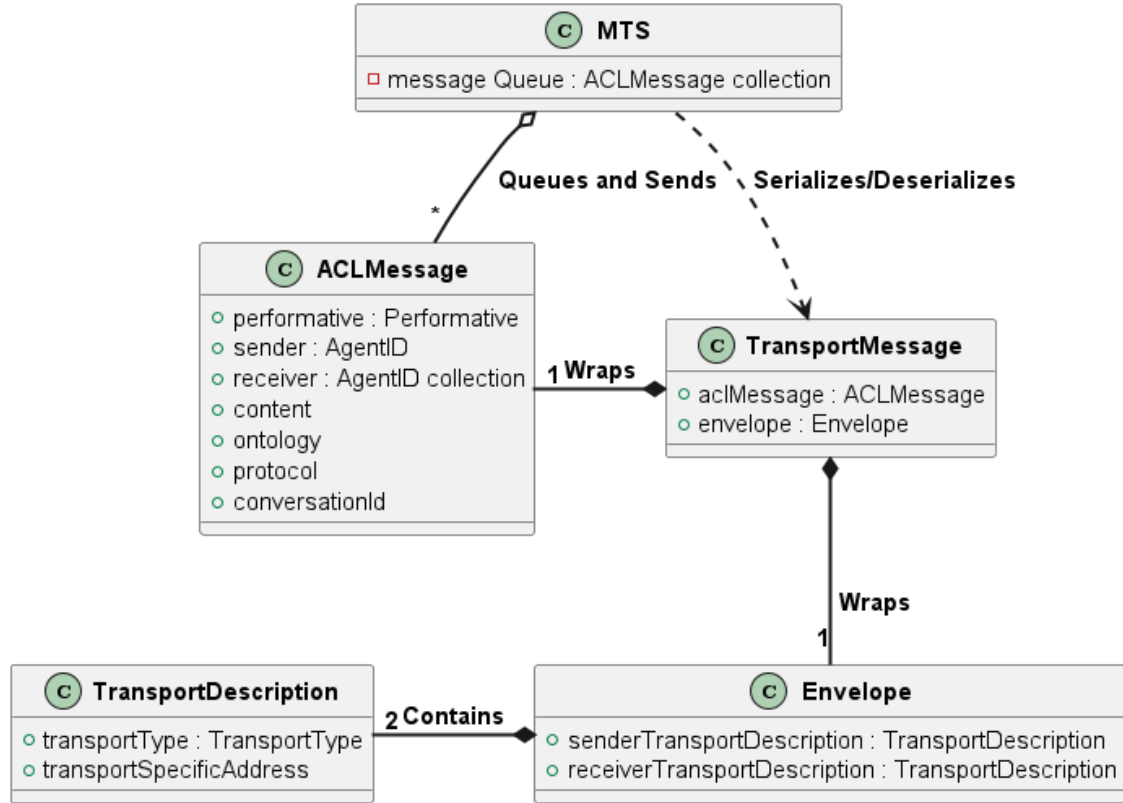


Figure 4.3: MTS and messaging class diagram.

4.2.1 MTS Internal Logic

The MTS message sending and receiving logic is split into three main processes: one for sending and two for receiving. The simplest process is message sending, where the MTS, once prompted, transmits a message directly to a target agent. Two receiving processes were designed to separate synchronous network message reception from asynchronous message retrieval by behaviors. Both of these message receiving processes are connected via an asynchronous message queue and a new-message flag. Whenever a message is synchronously received over the socket, it is placed inside the queue and the new-message flag is raised. When a behavior requests a message, if the new-message flag is raised, the message is extracted from the queue. An in-depth description of these processes and their interactions is provided below:

The **Message Sending Process** starts with a behavior requesting the MTS to transmit an `ACLMessage`. For each receiver, the MTS extracts the target's address from its `AgentID`, constructs a new `TransportMessage`, opens a new Transmission Control Protocol (TCP) connection, sends the message, and waits for confirmation. If a response other than a success message is received, an error is returned.

The **Synchronous Message Receiving Process** continuously listens for incoming connections using a concurrent message server. Upon receiving a new connection, it

is accepted, the incoming `TransportMessage` is deserialized, the `ACLMessage` payload is appended to a synchronized message queue, and the internal `newMessage` flag is asserted. A UML sequence diagram illustrating this process is presented in Figure 4.4.

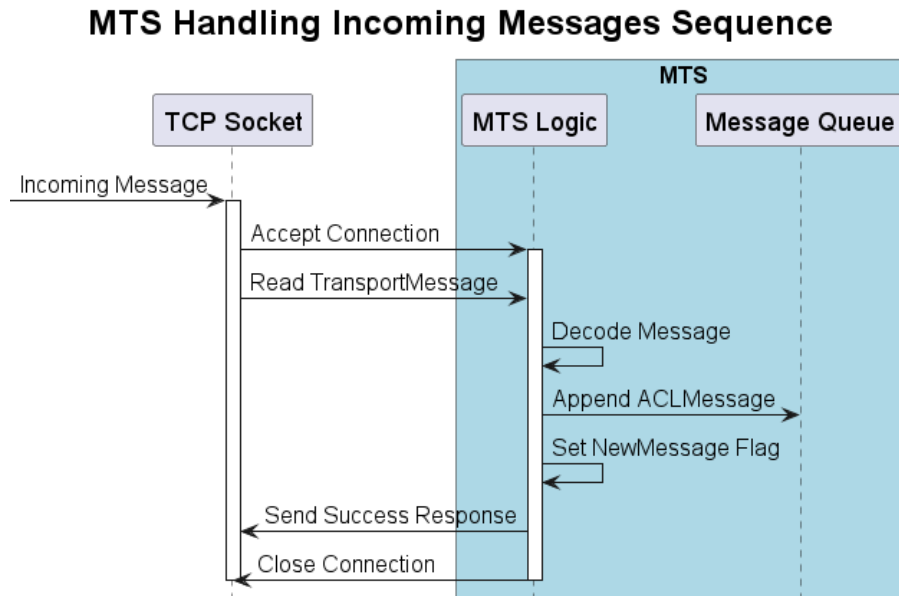


Figure 4.4: MTS Synchronous Message Receiving Process.

The **Asynchronous Message Receiving Process** enables behaviors to retrieve messages asynchronously from the internal message queue through the submission of an `ACLMessageTemplate`. The MTS iterates through the queue, matching messages against the template filter. If a match is found, the message is popped from the queue and returned to the behavior. A UML sequence diagram illustrating this process is presented in Figure 4.5. If a behavior retrieves a message but is unable to process it immediately, it places it back at the front of the queue for later use.

Three MTS operations are exposed to the agent logic, providing all the functionality necessary to communicate and collaborate with other agents:

- **Synchronously transmitting** `ACLMessage` payloads to target agents;
- **Asynchronously retrieving** matching `ACLMessage` payloads from the queue;
- **Requeueing** deferred `ACLMessage` payloads back into the asynchronous queue.

Without this decoupled MTS architecture, an agent would be restricted to a single monolithic behavior, preventing the concurrent execution of continuously running environment management logic and thus precluding a distributed multi-agent system. The design of these management services is detailed in the next section.

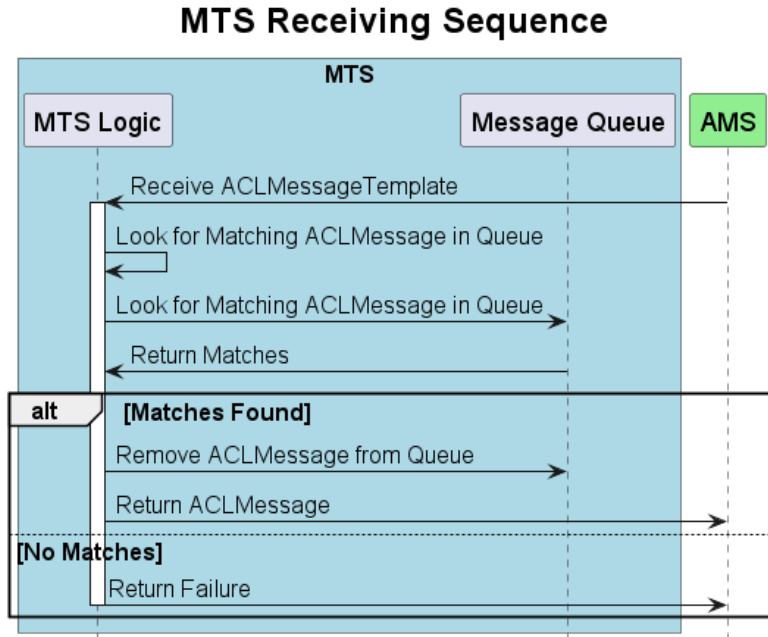


Figure 4.5: MTS Asynchronous Message Receiving Process.

4.3 EMA, ASDS, and Environment Management Services

Due to the complexity of a fully distributed system, the TAMAF architecture requires a set of internal services that maintain the ecosystem alive and up-to-date. These services were designed taking into consideration the different hardware topologies and limitations that a platform may have, accommodating bare-metal firmware devices, such as microcontrollers, and robust industrial computers concurrently. A conceptual description of each individual service was provided in Chapter 3, with a formal technical explanation provided below.

The environment management services are modeled as two distinct groups: Local Management Services and External Management Services. Local refers to operations internal to an environment. Both groups are essential for the proper functioning of a decentralized deployment.

The Local Management Services keep the EMA informed of the operational state of resident agents, maintain local agent liveness, and sustain the EMA itself. They also govern agent registration and deregistration. The External Management Services ensure that the local EMA stays synchronized with peer EMAs across the network, while simultaneously broadcasting its own state so that external environments can maintain up-to-date directory views.

4.3.1 ASDS

The ASDS plays a key role in the environment management services, acting as the distributed repository that stores metadata concerning agents and EMAs across the ecosystem alongside state variables relevant to their interactions. It also exposes discovery

interfaces that agents use to query for specific capabilities. In these interactions, the ASDS maintains:

- A collection of available local operational addresses;
- The local agent repository, containing `AgentDescription` objects for all registered resident agents;
- A heartbeat counter for each registered local agent;
- A timestamp recording the most recent state modification of the local agent repository;
- The external agent repository, containing `AgentDescription` objects for agents registered across peer environments;
- The external EMA repository, storing network addresses and synchronization metadata for peer EMAs;
- A heartbeat counter for each external EMA.

The motivation and mechanics underlying this information are detailed across the following service descriptions.

4.3.2 Management Ontologies

Leveraging the ontology attribute of FIPA-ACL messages, two distinct ontologies were defined to standardize communication: the `AgentManagementOntology` (for Agent-to-EMA exchanges) and the `EnvironmentManagementOntology` (for EMA-to-EMA exchanges). Both ontologies are structured symmetrically using actions and concepts. Actions define the communicative intent of the message, specializing the generic performative to ensure unambiguous semantic processing. Concepts define the structure and data types of the message payload, ensuring that senders and receivers parse parameters consistently.

The `AgentManagementOntology` governs lifecycle operations and service discovery within an environment. It defines actions for registering an agent (`register`), deregistering an agent (`deregister`), and modifying an existing descriptor (`modify`), alongside corresponding outcome notifications (`register-result`, `deregister-result`, and `modify-result`). Furthermore, it defines discovery actions scoped to the local environment, external environments, or the global ecosystem (`local-search`, `external-search`, and `global-search`), with corresponding query responses (`search-result`). The ontology also formalizes associated payload concepts: `agent-description` collections, `agent-address` locators, and `inform` status strings.

The `EnvironmentManagementOntology` governs heartbeat synchronization across peer EMAs. Its primary action is `keep-alive`, used by EMAs to multicast their operational status. The core concept within this ontology is `EnvironmentUpdate`, which encapsulates the state timestamp of the local ASDS.

4.3.3 Local Management Services

The Local Management Services are composed of two functional tiers: the *Agent Registration Service* and the *Local Persistence Services*. As established, the ASDS tracks the timestamp corresponding to the last time the local agent repository was modified. This timestamp is updated whenever an agent registers, modifies its capabilities, or deregisters from the ASDS, serving as a vital synchronization primitive for EMA-to-EMA coordination.

4.3.3.1 Agent Registration Service

The Agent Registration Service, hosted by the EMA, coordinates four interconnected interactions: *Registration*, *Modification*, *Deregistration*, and *Search*. All interactions use the `AgentManagementOntology` for structured data exchange. While initial registration and final deregistration are automated within the agent lifecycle, developers can also programmatically invoke registration updates at runtime.

The initial step for an agent joining the ecosystem is to **Register** with its local EMA. The network address of the EMA within the environment is standardized and already known, enabling agents to discover it without prior topology discovery. The agent temporarily binds to a designated registration address to receive initialization replies from the EMA, avoiding port collisions with active agents. It then transmits a `REQUEST` message containing the `register` action and an `agent-description` concept carrying its initial `AgentDescription`. The EMA verifies address availability in the ASDS, allocates a unique operational address, records the agent's presence in the local ASDS repository, and marks the assigned address as locked. The EMA replies with an `INFORM` message containing the `register-result` action and an `agent-address` concept carrying the allocated address. Upon reception, the agent migrates its MTS server socket from the registration address to the operational address, where it remains for the duration of its lifecycle. If the EMA has exhausted its pool of available local addresses, it returns a `register-result` message with the `inform` concept containing `no-ports-available`, prompting the agent to terminate gracefully. This sequence is embedded within the core agent lifecycle and executes automatically upon startup, as illustrated in Figure 4.6.

For resource-constrained platforms unable to host an internal EMA, resident agents can register as outsiders with an EMA located on a neighboring platform. Because the remote EMA cannot manage port allocations on foreign hardware, it treats outsider addresses as pre-assigned and fixed. The outsider agent registers directly with its static address, which the EMA registers into the ASDS without reallocating ports. In this scenario, the confirmation message carries the `register-result` action and `inform` concept with the value `outside-agent`.

The EMA binds directly to a predefined address. If that predefined address is already in use during the bind attempt, an EMA is already active in the environment, causing the new EMA to terminate.

The **Modification** interaction occurs when an agent alters its published services. The

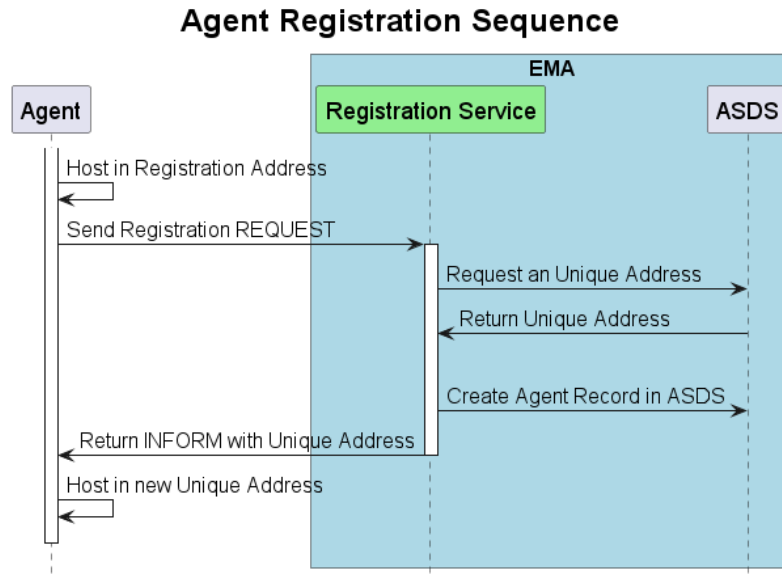


Figure 4.6: Agent Registration Process Sequence Diagram.

agent updates its local descriptor and dispatches an **INFORM** message containing the **modify** action and updated agent-description concept to the EMA. The EMA modifies the corresponding ASDS record and returns an acknowledgement with the **modify-result** action.

When an agent prepares to shut down, it executes the **Deregistration** interaction, dispatching a **REQUEST** message with the **deregister** action and agent-description concept. The EMA removes the agent's record from the ASDS, unlocks its assigned operational address, and acknowledges completion via a **deregister-result** message.

The EMA also exposes a directory discovery mechanism via the **Search** interaction. An agent issues a **REQUEST** message containing a search action (**local-search**, **external-search**, or **global-search**, depending on scope) and an agent-description concept specifying target capability criteria. The EMA queries the ASDS and returns an **INFORM** message containing the **search-result** action and a collection of matching **AgentDescription** objects.

The EMA, whenever using these four interactions, communicates directly with the ASDS without sending messages to itself. This way, there also isn't a counter for the EMA.

4.3.3.2 Local Persistence Services

Liveness monitoring across agents and the local EMA is maintained mutually through the **KeepAlive** interaction, supported jointly by the *Local EMA Persistence Service* (hosted on each agent) and the *Local Agents Persistence Service* (hosted on the EMA). Each agent periodically transmits an updated heartbeat descriptor to the EMA, while the EMA tracks elapsed time to identify unresponsive nodes.

The ASDS maintains an integer heartbeat counter for each registered local agent to

track the amount of time since the arrival of its last Heartbeat message. Periodically, the *Local Agents Persistence Service* increments these counters to verify agent presence. When this service's timer and the *Local EMA Persistence Service* Heartbeat interval are set to the same value, the counter effectively measures how many expected Heartbeat messages the agent is behind. To send a Heartbeat, the *Local EMA Persistence Service*, inside each and every agent, prepares a *KEEPALIVE ACLMessage* containing its *AgentDescription*, utilizing the *KeepAlive* action and the *agent-description* collection concept. Upon receiving this message at the EMA, the *Local Agents Persistence Service* resets the agent's counter to zero. If an agent fails to report and its counter exceeds a predefined threshold, the EMA concludes that the agent has terminated abruptly or experienced an unrecoverable fault. Consequently, the *Local Agents Persistence Service* removes the agent from the ASDS local repository. This workflow is depicted in the UML activity diagram of Figure 4.7. Because the EMA is structurally any other agent, it hosts the *Local EMA Persistence Service* as well; however, instead of sending, receiving, and counting messages, it only registers itself inside the ASDS.

Local Agents Persistence Service Activity Diagram

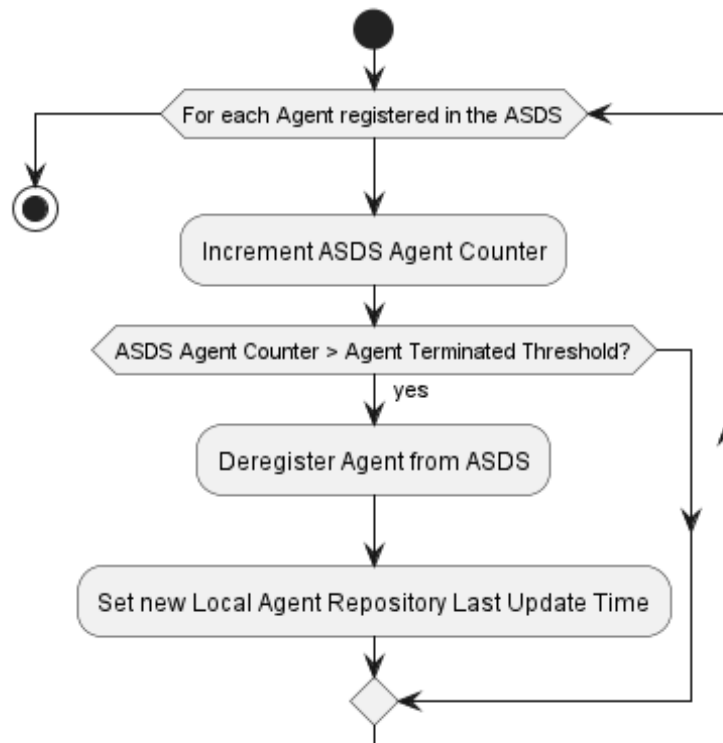


Figure 4.7: EMA Local Agents Persistence Service Activity Diagram.

Because heartbeat payloads include the agents' complete current *AgentDescriptions*, a newly elected replacement EMA can reconstruct the full state of the environment simply by listening to incoming heartbeats. This feature is crucial for the system's robustness, as

it allows for a graceful and seamless recovery of the environment's state without requiring manual intervention or complex state reconstruction processes, maintaining continuity and minimizing downtime. Because the EMA is responsible for keeping the environment alive and updated, and is the only entity capable of doing so, its flexibility and robustness are paramount.

A detailed UML activity diagram of the agent heartbeat processing routine within the EMA is illustrated in Figure 4.8.

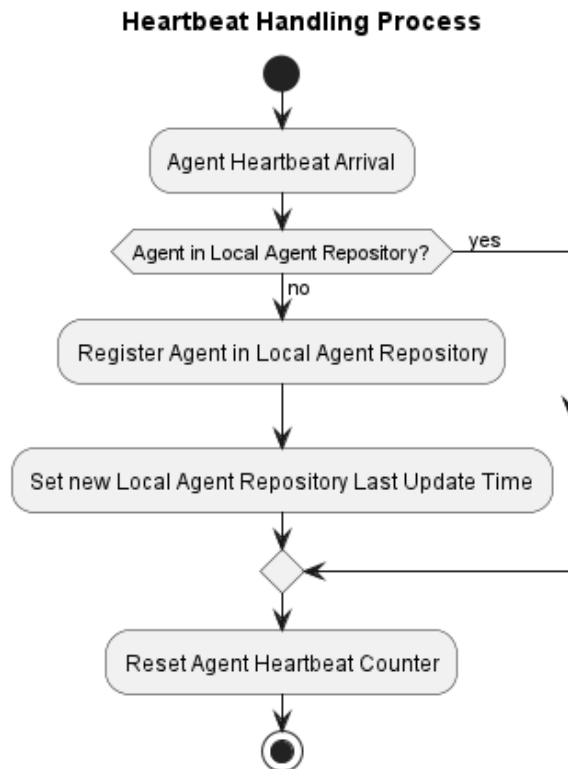


Figure 4.8: Activity diagram of the heartbeat handling process in the Local Agents Persistence Service.

Whenever an environment EMA dies, if many EMAs are launched and/or many agents try to assume the position of the new EMA at the same instant, only one is able to bind to the predefined EMA address. In this case, launched EMAs are terminated and agents trying to take over as the EMA resume normal operation. Already operating agents that become the new EMA must update their name and start providing all the necessary management services. The services previously provided by the agent are maintained under this new name.

If an agent attempts to register with an EMA that already possesses an entry under its AgentID, the EMA evaluates its heartbeat counter against an absence threshold. If the counter has not reached the threshold, the EMA rejects registration with an agent-already-registered status, treating the request as an unauthorized duplicate name and returning a message with the *register-result* action, the *inform* concept, and the

agentalreadyregistered status content. Conversely, if the threshold has elapsed (indicating that the original instance crashed or disconnected before deregistering), the EMA treats the previous record as dormant, allowing the new instance to reclaim its identifier, update its address, and resume operations.

4.3.4 External Management Services

EMAs coordinate with one another using an inter-environment heartbeat mechanism analogous to local agent monitoring. This coordination is orchestrated by two services: the *Environment Discovery Service*, which multicasts local environment announcements, and the *Environments Persistence Service*, which maintains directory records of remote EMAs. Both services are explained in detail below.

4.3.4.1 Environment Discovery Service

Due to the distributed topology of multi-agent networks, EMAs lack static knowledge of peer locations. Consequently, discovery announcements are transmitted over User Datagram Protocol (UDP) to a shared multicast group address. All EMAs in the network subnet join this group, discovering peer environments dynamically.

Because connectionless UDP datagrams are subject to packet fragmentation and loss, multicast messages do not carry full environment descriptions. Instead, to minimize load, each multicast packet contains only the timestamp corresponding to the most recent modification of the local ASDS. The UDP messages carrying the timestamps are structured in accordance with the *EnvironmentManagementOntology*, using the *keep-alive* action and *EnvironmentUpdate* concept. These timestamps are stored inside the ASDS external EMA repository.

Upon receiving an announcement, an EMA compares the incoming timestamp against the timestamp recorded in its ASDS external EMA repository. If the incoming timestamp is newer, the receiving EMA identifies that the remote environment has been modified. Because the multicast listener socket is dedicated strictly to EMA coordination, it is managed independently of the standard MTS TCP server socket, preserving a minimal memory footprint for the core agent structure.

When an updated timestamp is detected, the EMA initiates a point-to-point TCP unicast connection to the remote EMA, issuing a *local-search* query formatted identically to standard registration search requests. The remote EMA returns its updated local *AgentDescription* records, which the querying EMA writes into its ASDS external agent repository.

The heartbeat handling logic within the *Environment Discovery Service* is illustrated in Figure 4.9.

Upon receiving a UDP message from an unregistered EMA, the recipient creates a new entry in its ASDS external EMA repository, initializes a heartbeat counter, and dispatches a unicast discovery query to retrieve its resident agent descriptors. If the EMA is already

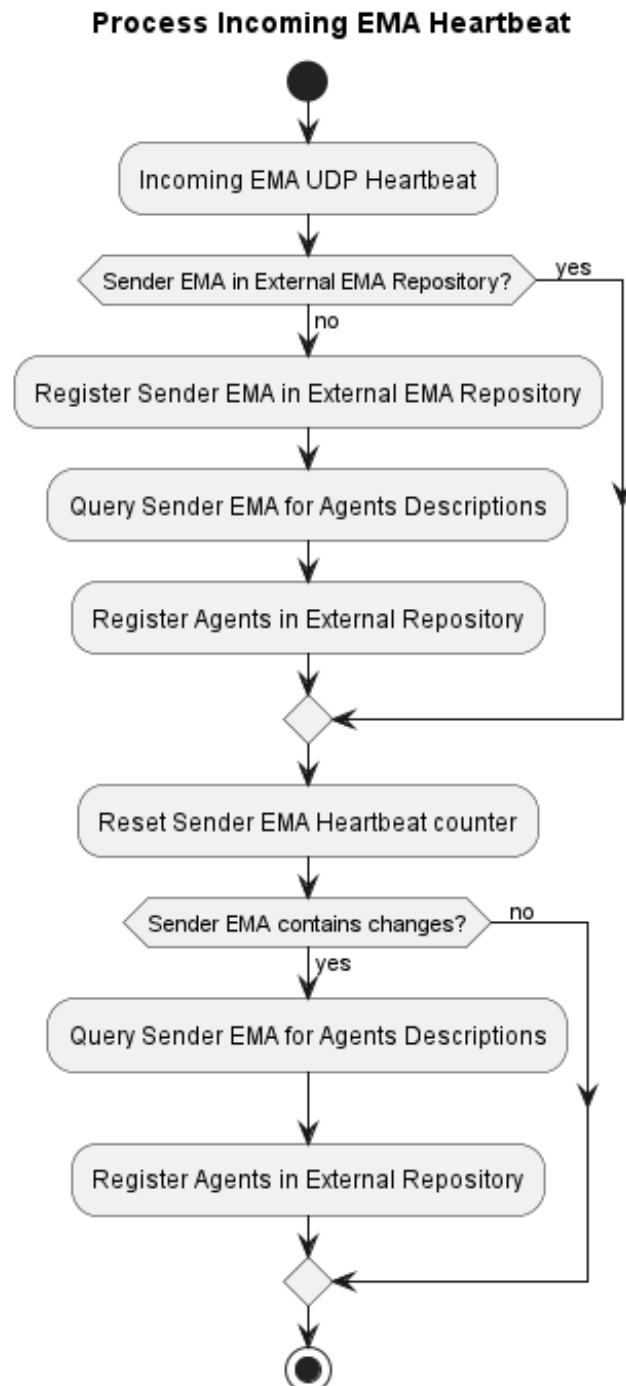


Figure 4.9: Activity diagram of the EMA multicast heartbeat handling process within the Environment Discovery Service.

registered, the timestamps are compared, triggering a discovery query only when the remote timestamp is strictly newer.

4.3.4.2 Environments Persistence Service

Every EMA executes an *Environments Persistence Service* that monitors the availability of peer EMAs using counter-based tracking similar to local agent persistence. Incoming multicast messages from a remote EMA reset its corresponding counter, while the local service increments it periodically. If an EMA's counter exceeds the expiration threshold, it is presumed dead: its records are purged from the ASDS external EMA repository, and its resident agents are deleted from the external agent repository. Because EMAs communicate over pre-configured multicast and unicast channels rather than managed local port ranges, an intermediate absence threshold is unnecessary.

4.4 Behaviors

The primary objective of these specifications is to establish an agile architecture for distributed task execution. Achieving framework robustness hinges directly on how agent task logic is structured, scheduled, and executed.

In these specifications, agent logic is modularized into Behavior abstractions. Behaviors encapsulate operational workflows and are scheduled to run cooperatively and asynchronously. This concurrent model optimizes CPU utilization and enables background management services, such as heartbeat transmission and message polling, to execute seamlessly alongside application-specific domain tasks.

The behavior model is designed for high modularity and reusability, allowing basic behaviors to be composed hierarchically into complex execution trees. Later in this section, a more detailed explanation of these complex behavior abstractions is provided.

4.4.1 Core Behavior Lifecycle

Every behavior follows a structured lifecycle partitioned into three execution phases:

1. **Start:** Preliminary initialization executed prior to the core task (e.g., establishing sockets, acquiring device handles, resource calibration);
2. **Action:** The primary computational routine of the behavior, executed iteratively until the task completion criterion is satisfied;
3. **End:** Teardown and cleanup logic executed once the action phase completes (e.g., releasing resources, persisting state, terminating connections).

This lifecycle is governed by the execution kernel hosted within the agent's AMS. The kernel's cooperative round-robin scheduler ensures that all active behaviors receive

fair execution time, preventing any single task from starving the agent's computational resources.

Under the agent's asynchronous and decoupled execution model, behaviors retrieve incoming messages from the MTS asynchronous queue without blocking concurrent tasks. To prioritize efficiency during the action phase, any behavior requiring a message not yet present in the queue enters a blocked state and is temporarily bypassed by the kernel's scheduler. Users can also programmatically block behaviors to meet specific needs. Upon the arrival of any new network message, all blocked behaviors are momentarily unblocked to re-evaluate the queue and determine if the message belongs to them. Behaviors that remain unblocked, meaning they possess work to perform and can freely run their logic, are classified as active behaviors. This mechanism is crucial for the system's performance, allowing for the concurrent execution of multiple behaviors as they wait for different messages or events to proceed with their tasks.

This model introduces a race condition: if a behavior requires multiple coordinated messages and only one has arrived, a naive non-blocking retrieve would consume the first message and discard it upon blocking for the second. To eliminate this issue, each behavior incorporates a rollback buffer. When the `action` method is invoked, all messages retrieved during that iteration are recorded in the buffer. If the behavior subsequently blocks before completing its action logic, all messages buffered during that round are requeued into the MTS asynchronous queue, in the exact sequence they were received, preserving them for future execution cycles.

Each lifecycle phase corresponds to a dedicated callback method. Completion of the action phase is governed by the `done` method, which returns a boolean value: returning `false` indicates that the task remains incomplete and must be re-executed in the next scheduling pass; returning `true` signals completion, prompting the kernel to invoke `onEnd` and terminate the behavior.

To enforce non-blocking message safety, the kernel wraps the behavior's action method within an internal `execute` wrapper. The wrapper orchestrates message queue management and behavior block handling, isolating execution mechanics from domain-specific task logic.

Consequently, developers provide domain logic exclusively within the `onStart`, `action`, `done`, and `onEnd` methods. The internal `execute` wrapper is managed by the framework and remains unexposed to end-user modification. A tool for blocking and unblocking the behavior is also provided and can be used inside the *Action* logic block.

4.4.2 Complex Behaviors

By prioritizing modularity and reusability, the Behavior architecture allows for diverse combinations to be tailored to specific use cases. Building upon the core Behavior class (also named `SimpleBehavior`), the TAMAF specification defines seven specialized extensions designed to accommodate diverse execution patterns. These extensions enrich the core

infrastructure, providing a robust, higher-level toolset designed to meet more complex developmental requirements. The seven behavior extensions are grouped into three categories: two Specific Execution Behaviors, two Timed Behaviors, and three Composite Behaviors.

Specific Execution Behaviors formalize common single-pass or infinite execution patterns:

- **OneShotBehavior:** Designed to execute exactly once. Its `done` method unconditionally returns `true`, ensuring that `action` executes a single time before transitioning to `onEnd`.
- **CyclicBehavior:** Designed for continuous polling or monitoring. Its `done` method unconditionally returns `false`, executing its `action` phase indefinitely for the duration of the agent's lifecycle.

Timed Behaviors introduce temporal constraints, allowing for delayed or periodic execution:

- **WakerBehavior:** Executes a callback (`onWake`) once after a specified time interval has elapsed. During execution cycles prior to timeout expiration, it evaluates elapsed time and yields control immediately.
- **TickerBehavior:** Executes a periodic callback (`onTick`) at fixed time intervals. It executes indefinitely, with its `done` method returning `false`.

Composite Behaviors orchestrate multiple child sub-behaviors sequentially, concurrently, or as a state machine:

- **SequentialBehavior:** A composite behavior that executes an ordered sequence of child sub-behaviors. It maintains an internal execution pointer, evaluating the `onStart` and `action` phases of the active child until its `done` condition is met, before advancing to the next. The composite behavior terminates only when all of its sub-behaviors have completed. It also provides a `sharedData` blackboard dictionary, allowing sub-behaviors to pass information and exchange state variables with subsequent ones.
- **ParallelBehavior:** Concurrently evaluates multiple child sub-behaviors in a round-robin fashion during each execution cycle. It iterates through all non-completed sub-behaviors, executing their `action` phases if they are not blocked. Its completion policy is configurable: it can terminate either when *any* single child finishes, or when *all* children have completed. Similar to `SequentialBehavior`, it provides a `sharedData` dictionary for inter-behavior communication.

- **FSMBehavior:** Executes child behaviors as discrete states within a Finite State Machine (FSM). Starting from a designated `initialState`, it manages a `currentState` pointer to track the active state, executing its `onStart` and `action` phases until its `done` condition returns true. Upon state completion, the child's `onEnd` method must return a type of transition code, which the FSM maps to dictate the shift to the subsequent target state. Execution concludes when a designated `finalState` completes, or if no valid transition is found. It also provides a `sharedData` dictionary allowing states to pass information between them.

These composite abstractions simplify the implementation of complex interaction protocols, such as FIPA Contract Net and FIPA Request. In the next section, a detailed description of how behavior logic blocks and lifecycles are managed, scheduled, and executed by the kernel is provided.

4.5 AMS

The AMS serves as the heart and brain of the agent, governing agent lifecycle states and orchestrating behavior execution within its internal kernel.

4.5.1 Lifecycle States

The agent lifecycle is formalized as a finite state machine comprising four execution states. Upon instantiation, the agent enters the `INITIATED` state to perform baseline configuration and service discovery. Once initialization succeeds, the agent transitions to the `ACTIVE` state, in which the scheduler actively executes its behavior pool. If commanded to pause task processing, the agent transitions to the `SUSPENDED` state: application behaviors are paused, while core environment management services, specifically the *Local EMA Persistence Service*, remain active to maintain directory registration. When an agent terminates, either gracefully upon task completion or abruptly due to an unrecoverable fault, it enters the `CLOSING` state to release system resources and deregister before shutting down. From `SUSPENDED`, the agent may return to `ACTIVE` upon receiving a resume signal. The lifecycle state machine and valid transitions are depicted in Figure 4.10.

All computational tasks within an agent are executed through behaviors, including the mandatory lifecycle routines executed during the `INITIATED` and `CLOSING` states. The initialization sequence must configure internal services, register with the EMA, and bind to the assigned operational address. Depending on framework implementation, this workflow may be distributed across one or more behaviors, collectively designated as `InitializationBehaviors`. Similarly, shutdown logic releases allocated resources, deregisters the agent from the local EMA, and terminates execution; behaviors executing this routine are termed `ClosingBehaviors`.

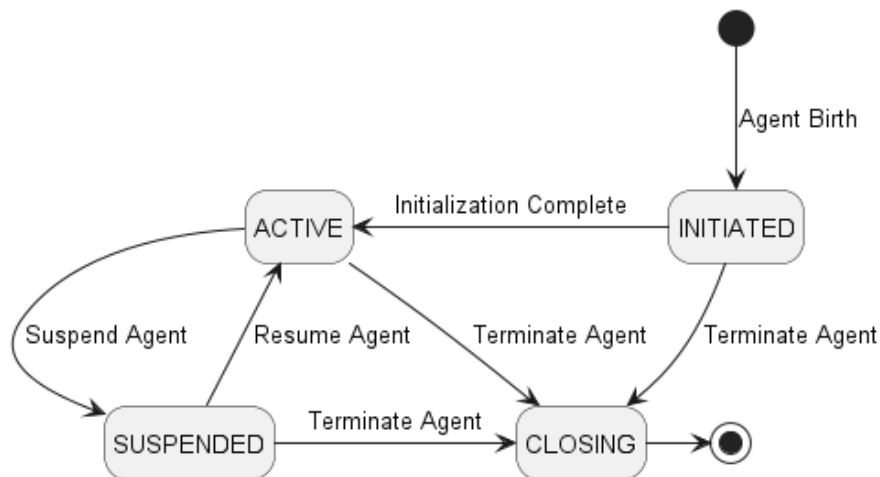


Figure 4.10: State machine diagram of the agent lifecycle.

Both `InitializationBehaviors` and `ClosingBehaviors` are mandatory in every TAMAF agent, guaranteeing controlled startup and deterministic teardown. Furthermore, these behaviors expose developer extension hooks, specifically the `onSetup` and `onTakedown` callback methods, allowing custom domain logic to execute immediately following mandatory initialization and immediately prior to final shutdown.

A dedicated set of behaviors implementing the *Local EMA Persistence Service* is likewise mandatory for every agent. The EMA itself runs a specialized suite of behaviors to sustain its hosting management services. Collectively, these infrastructure routines are designated as `ManagementBehaviors`.

The TAMAF specification mandates behavioral contracts rather than rigid internal implementations, granting developers latitude regarding the quantity and internal decomposition of `InitializationBehaviors`, `ClosingBehaviors`, and `ManagementBehaviors`. These implementations must, however, strictly adhere to the operational specifications defined for Environment Management Services.

In summary, the agent architecture provides developers with `onSetup` and `onTakedown` extension hooks alongside supervisory methods for controlling agent activation, suspension, resumption, and termination.

4.5.2 Execution Kernel

The execution kernel embedded within each agent coordinates operational workflows across the agent lifecycle, orchestrating initialization, behavior scheduling, message dispatching, and final teardown.

The kernel exposes programmatic interfaces allowing behaviors to be dynamically registered or deregistered at runtime, queueing addition and removal requests to prevent race conditions. It additionally provides mechanisms for programmatically asserting and clearing behavior blocking flags.

The internal logic of the kernel is partitioned into two core operations: supervising behavior lifecycle transitions and executing scheduled behavior logic blocks.

The behavior lifecycle management loop comprises four sequential stages. First, the kernel inspects the current lifecycle state of the agent to determine execution permissions: while in the `INITIATED` state, it restricts execution exclusively to `InitializationBehaviors`; in the `ACTIVE` state, it schedules registered active behaviors; and in the `CLOSING` state, it invokes `ClosingBehaviors`. Second, the kernel verifies the MTS `newMessage` notification flag; if asserted, all blocked behaviors are unblocked to evaluate the queue, after which the flag is cleared. Third, the kernel processes pending behavior registrations and deregistrations, updating the active behavior set. Dynamic additions and removals to this set are only permitted while the agent's lifecycle remains in the `ACTIVE` state. Fourth, the kernel dispatches all active behaviors using a round-robin scheduling algorithm. This cyclic process is illustrated in the UML activity diagram of Figure 4.11.

When dispatching an individual behavior, the kernel coordinates its lifecycle execution sequentially. The kernel first verifies whether the behavior has been initialized; if not, it invokes the behavior's `onStart` method. Following initialization, the kernel invokes the internal `execute` wrapper to run the behavior's action logic. As established, the `execute` wrapper provides message buffering and non-blocking retrieval; if the behavior blocks while awaiting an incoming message, all messages buffered during that round are requeued into the MTS asynchronous queue. Once the action phase completes, the kernel calls the behavior's `done` method: if `done` returns `false`, the behavior remains in the active scheduling pool for subsequent cycles; if `done` returns `true`, the kernel invokes its `onEnd` method and deregisters the behavior from the scheduler. This dispatching sequence is illustrated in Figure 4.12.

Chapter 5 presents the concrete framework implementations derived from these specifications, demonstrating how the TAMAF architecture is realized across heterogeneous hardware platforms with disparate computational constraints.

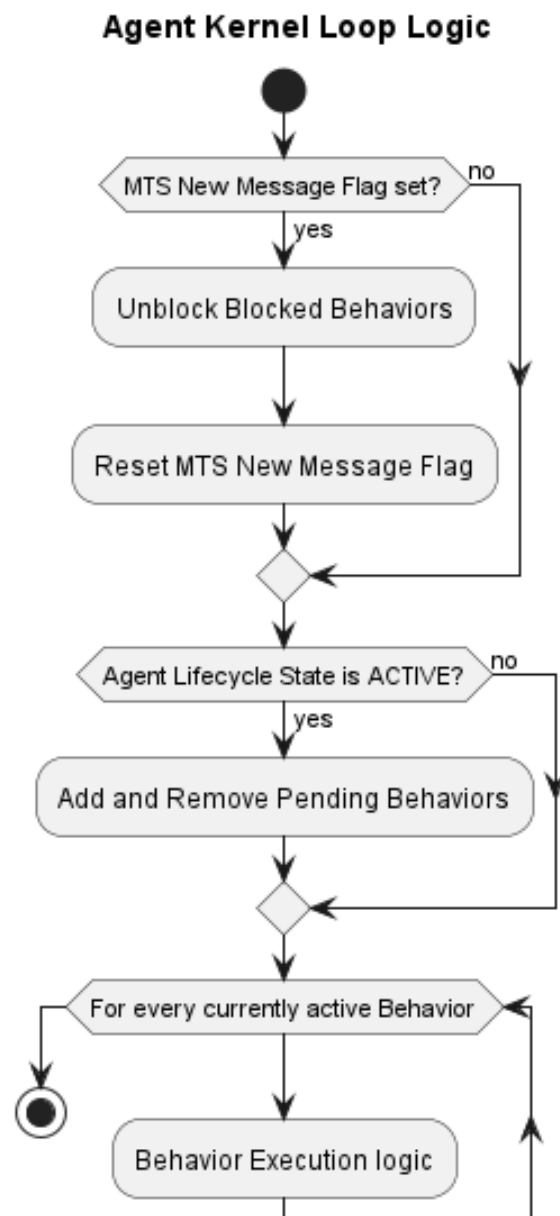


Figure 4.11: Kernel Loop Logic Activity Diagram.

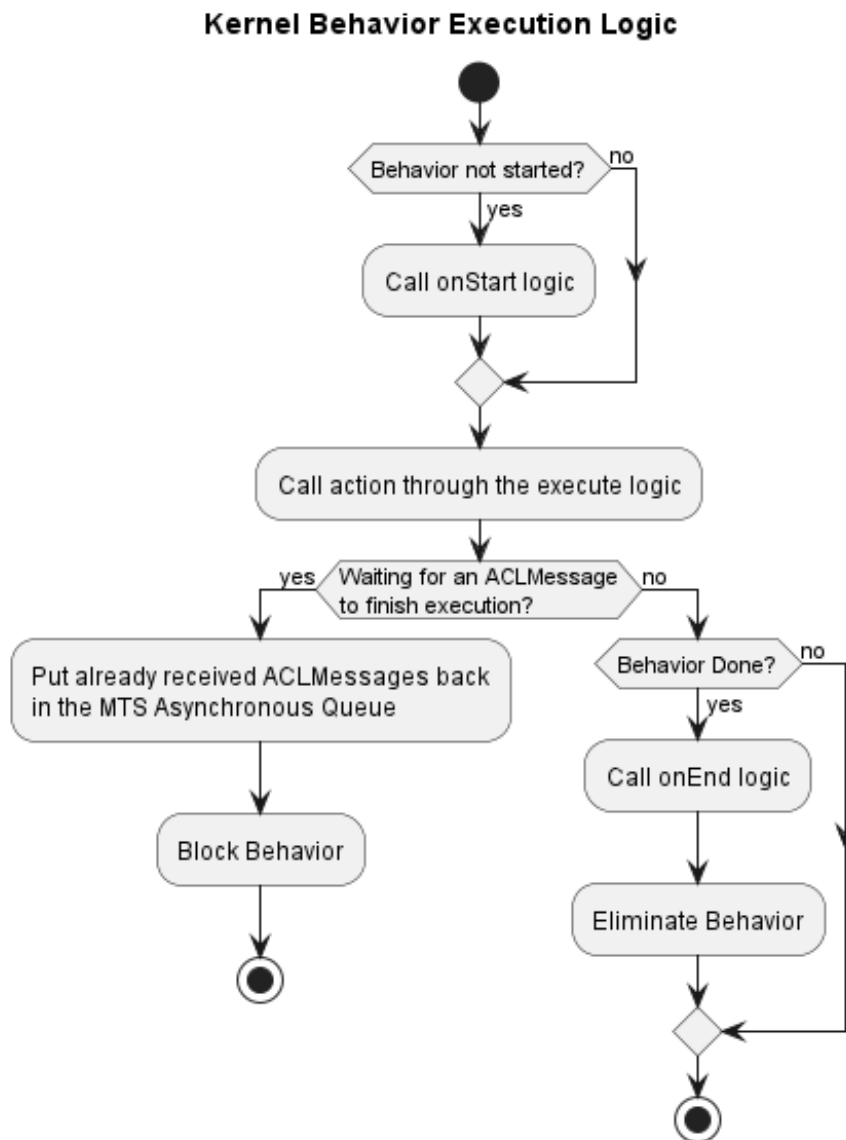


Figure 4.12: Kernel Behavior Execution Logic Activity Diagram.

IMPLEMENTATION

The specifications detailed in Chapter 4 define an abstract, language-independent architecture. This high-level abstraction allows the framework to be implemented across various computing platforms and programming paradigms, ensuring a robust and interoperable distributed system. However, when implementing these specifications across different platforms, unique challenges arise due to the distinct characteristics and limitations of each environment. Thus, a set of implementation specifications must be carefully tailored to each platform to accommodate its specific requirements, while maintaining adherence to the core architectural principles.

This chapter demonstrates how the proposed solution specifications can be effectively implemented across three distinct platforms: Python, Java, and C++. Each implementation serves as a reference model and was chosen to evaluate platform independence, showcasing the adaptability of the architecture to different programming languages and runtime environments. The implementations are designed to be functionally equivalent, ensuring that agents can communicate seamlessly across platforms while preserving the integrity of the multi-agent system. This is achieved by implementing the TAMAF infrastructure, represented in Figure 3.1, with provided tools and objects for each implementation, as depicted by Figure 5.1. In these programming languages, the objects are classes and the tools are accessed through method calls.

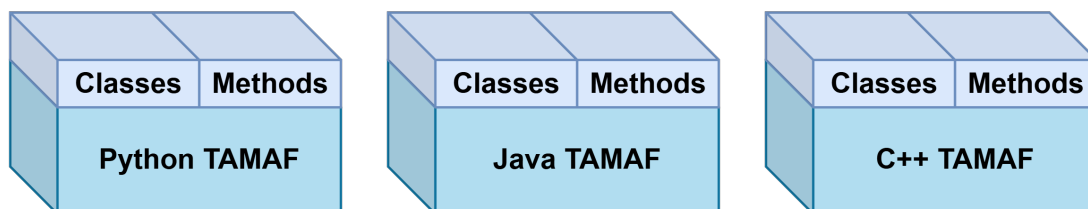


Figure 5.1: TAMAF Python, Java, and C++ deployments.

The TAMAF implementations underwent several iterations throughout the development of this thesis, evolving from an initial, rigid two-threads-per-agent model toward an

event-driven, cooperative execution architecture. While certain implementation specifications are shared across all platforms, others are necessarily tailored to the programming language, runtime environment, and target hardware. The following sections detail both the common cross-platform specifications and the environment-specific adaptations, highlighting the unique considerations and optimizations made to ensure efficient and reliable operation within the constraints of each platform.

5.1 Common Implementation Specifications

The implementation specifications common to all platforms are designed to ensure that the core functionalities of the multi-agent system are preserved regardless of the underlying technology. They were omitted from the architectural specifications in Chapter 4 due to their implementation-specific nature, as alternative implementations may adopt different technologies or mechanisms for these aspects. The common implementation specifications include:

- **Message Serialization:** To achieve cross-platform interoperability, the MTS operationalizes a language-agnostic wire protocol over standard TCP stream sockets. Each transmission is framed in two contiguous segments: a 4-byte big-endian unsigned integer (`uint32`) header indicating the byte length of the upcoming payload, followed immediately by the payload itself. This length-prefixed framing prevents TCP packet fragmentation issues and framing errors across heterogeneous stream parsers. The payload contains JavaScript Object Notation (JSON)-serialized data compliant with the management ontologies established in Chapter 4.
- **Architecture Behaviors:** In this context, architectural symmetry across implementations simplifies development, debugging, and verification. Standardized behavior archetypes are specified across all implementations:
 - **InitializationBehavior:** Specified as a `SimpleBehavior` to coordinate dynamic agent startup and registration.
 - **ClosingBehavior:** Specified as a `OneShotBehavior` to execute orderly teardown and deregistration.
 - **HeartBeatBehavior:** Specified as a `TickerBehavior` to periodically dispatch keep-alive signals to the EMA, implementing the EMA Persistence Service within agents.
- **Behavior Blocking:** All implementations utilize exceptions to suspend the Behavior's Action phase when a required message has not yet arrived, causing the `Execute` method to raise an exception that is subsequently caught by the kernel.
- **FIPA Complex Interaction Protocols:** Both the FIPA Request and FIPA Contract Net protocols follow an identical design schema across all implementations, structured

in accordance with the FIPA specifications using `FSMBehavior` states composed of discrete `SimpleBehavior` and `OneShotBehavior` execution units.

- **EMAInteraction Facilitator:** To streamline interactions with the EMA and simplify agent programming, the `EMAInteraction` class was created to encapsulate and mediate all agent-to-EMA communications and provide high-level services:
 - **KeepAliveAgent:** Dispatches a keep-alive message to the EMA.
 - **RegisterAgent:** Registers an agent with the EMA.
 - **DeRegisterAgent:** Deregisters an agent from the EMA.
 - **ModifyAgent:** Modifies the EMA's registry record for an agent.
 - **LocalSearch:** Conducts a local search request to the EMA, querying only agents registered to the same EMA.
 - **ExternalSearch:** Conducts an external search request to the EMA, querying only agents registered outside of this EMA.
 - **Search:** Conducts a global search request across all federated EMAs.

A distinct `EMAInteraction` instance is created per active behavior, each generating unique `conversationID` tokens for its ACL messages. Consequently, if multiple behaviors within a single agent concurrently await responses from the EMA, incoming messages are correlated strictly to their originating behavior.

- **Debug Infrastructure:** All implementations include a configurable text-based logging and debugging subsystem supporting multiple levels of verbosity. By default, only essential lifecycle and networking events, such as agent initialization and termination, as well as message transmission and arrival, are reported. The developer can configure the debugging system to obtain in-depth diagnostics regarding the attributes of all incoming and outgoing messages, as well as the status of the local EMA Persistence Service and its periodic heartbeat transmissions.

Naturally, each platform possesses distinct characteristics and constraints that necessitate tailored implementation strategies. The following sections describe these platform-specific considerations and architectural choices. Notably, the EMA coordinator is implemented exclusively within the Python reference platform. Because the EMA functions as a centralized or federated directory service, maintaining a single reference implementation was sufficient to validate system functionality while simplifying cross-platform development and interoperability testing.

5.2 Python Implementation

The Python implementation, designed for mid to high level fog computing, leverages several standard packages and mechanisms to realize the specified TAMAF processes. It

utilizes the standard `abc` module to define abstract base classes for `Agent` and `Behavior`, effectively providing an extensible foundation for developing customized multi-agent systems. Consequently, framework classes comprise three distinct categories of methods: abstract methods that must be overridden by user implementations, internal methods that execute core TAMAF infrastructural logic, and accessor/mutator methods that manage object state. The Python implementation was designed to give agents the ability to launch new EMAs if the active one abruptly terminates.

Regarding the MTS and AMS, the Python implementation leverages the `asyncio` package to manage network I/O and concurrent agent execution within a single, shared background event-loop thread. A common event loop is created for the deployment, and all Python agents register their `AMSKernel` routines as asynchronous coroutines. Each kernel cycle cooperatively schedules and executes active behaviors, yielding control in a non-blocking manner via `await asyncio.sleep(0)` to prevent CPU spinning. Concurrently, the MTS handles non-blocking TCP servers and socket transmissions on the same event loop. When an incoming message arrives, the MTS places it into the synchronized message queue and signals an `asyncio.Event` (implementing the `newMessageFlag` as `newMessageEvent`), triggering the AMS kernel to immediately unblock waiting behaviors. This signal-driven, event-based model enables high-density concurrency across multiple agents within a single process.

In this implementation, four core classes encapsulate the infrastructural logic: `Agent`, `MTS`, `Behavior`, and `AMS`. Descriptive entities such as `ACLMessage`, `AgentID`, and `Address` are implemented as standard Python classes providing methods for serialization, data access, and instance comparisons. An `EMAInteraction` class was also developed to mediate message transmission and reception across all previously described EMA interactions.

The `Agent` class, depicted in Figure 5.2, provides a `Setup` method for initialization logic and a `Takedown` method for termination logic, both intended to be implemented by the user. These methods receive an `EMAInteraction` instance to facilitate any required communication with the EMA. The `Start`, `Suspend`, `Resume`, `Stop`, and `isAlive` methods manually manage the agent lifecycle. The `UpdateAgentDescription` method updates the `agentDescription` attribute and, in compliance with the `ModifyAgent` interaction specifications presented in Chapter 4, dispatches a behavior that invokes `ModifyAgent` through its `EMAInteraction` instance.

The `AddBehavior` and `RemoveBehavior` methods add and remove behaviors to and from the agent, respectively. Messaging methods are also provided to send, receive, and restore `ACLMessage` objects within the MTS asynchronous queue (`Send`, `Receive`, and `PutBack`). All behavior and messaging methods expose a higher-level abstraction interface delegating to the corresponding routines within the AMS and MTS, respectively.

The `MTS` class, illustrated in Figure 5.3, encapsulates several key attributes: `emaAddress` and `registerAddress` represent known environment coordinates configured upon agent creation; `messageQueue` is implemented as a standard Python list of `ACLMessage` objects protected by an asynchronous lock for thread-safe access; `newMessageEvent` is an

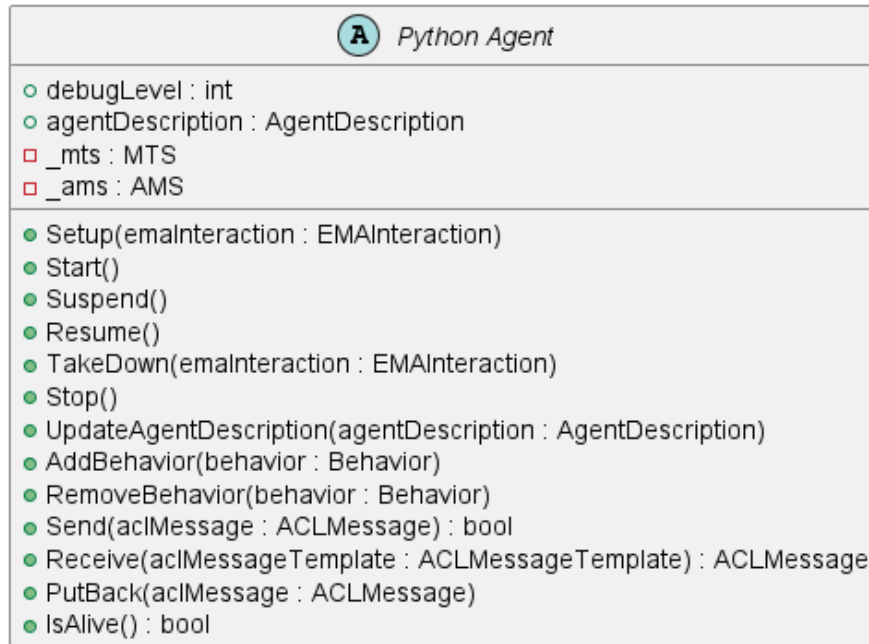


Figure 5.2: Python Agent Class Diagram.

`asyncio.Event` signaling message arrival; and `serverOnline` is a boolean flag indicating socket status. The MTS provides a `StartTCPServer(address : Address)` method and a `StopMessageServer` method used by the `InitializationBehavior` to register the agent at the `registerAddress` and subsequently bind to the assigned operational address provided by the EMA. For behavior messaging, the MTS exposes three core methods: `Send`, which synchronously transmits an `ACLMessage` to all agent recipients identified in the message's receiver attribute; `Receive(aclMessageTemplate : ACLMessageTemplate) : ACLMessage`, which asynchronously retrieves the first `ACLMessage` from `messageQueue` matching the specified template; and `PutBack(aclMessage : ACLMessage)`, which restores unconsumed messages to the front of `messageQueue`.

The `Behavior` class, illustrated in Figure 5.4, defines four lifecycle methods for user implementation: `OnStart` for initialization logic, `Action` for core computational tasks, `Done` to evaluate task completion, and `OnEnd` for cleanup routines. The `Send` and `PutBack` methods delegate directly to the corresponding MTS methods via the enclosing agent.

The `Behavior` `Receive` and `Execute` methods, in conjunction with the AMS kernel, govern behavior blocking via an exception-driven paradigm. The `Receive` method is invoked within `Action` whenever an incoming message is required to proceed. As depicted in Figure 5.5, it calls the agent's `Receive` method; if no matching `ACLMessage` is available, an `EmptyReceiveException` is raised. When a matching message is returned, the behavior does not block, and the message is prepended to a buffer list named `messagesToPutBack`. This buffer serves as a rollback mechanism ensuring no messages are lost if a subsequent receive operation causes the behavior to block before completing its execution cycle.

The `Execute` method, invoked by the kernel to advance behavior execution, completes

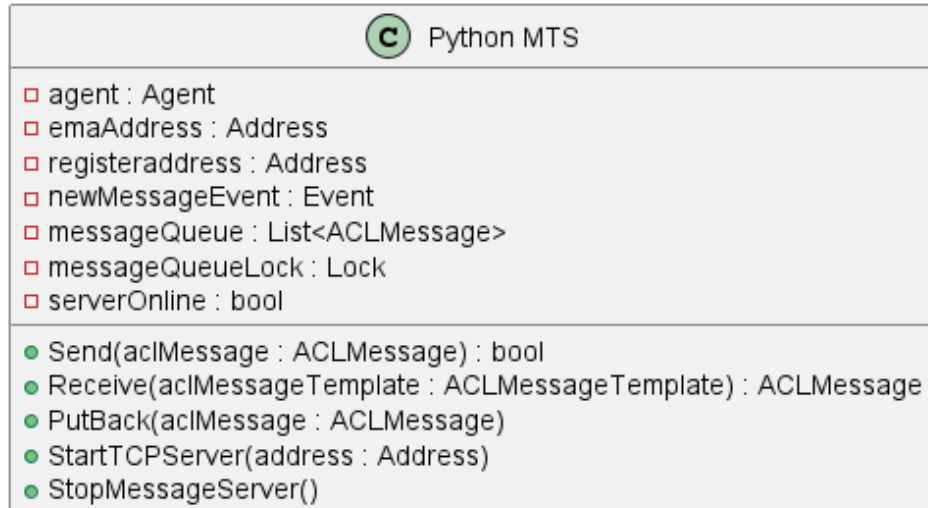


Figure 5.3: Python MTS Class Diagram.

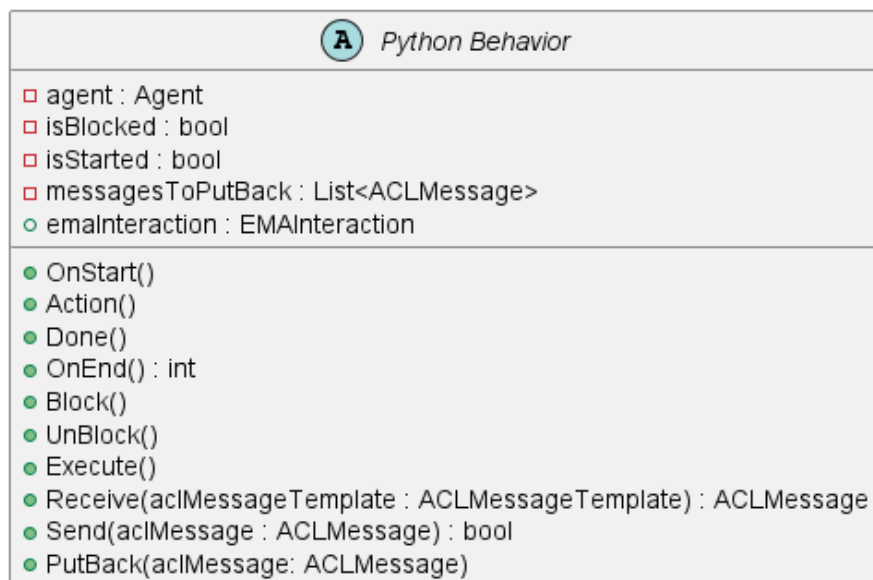


Figure 5.4: Python Behavior Class Diagram.

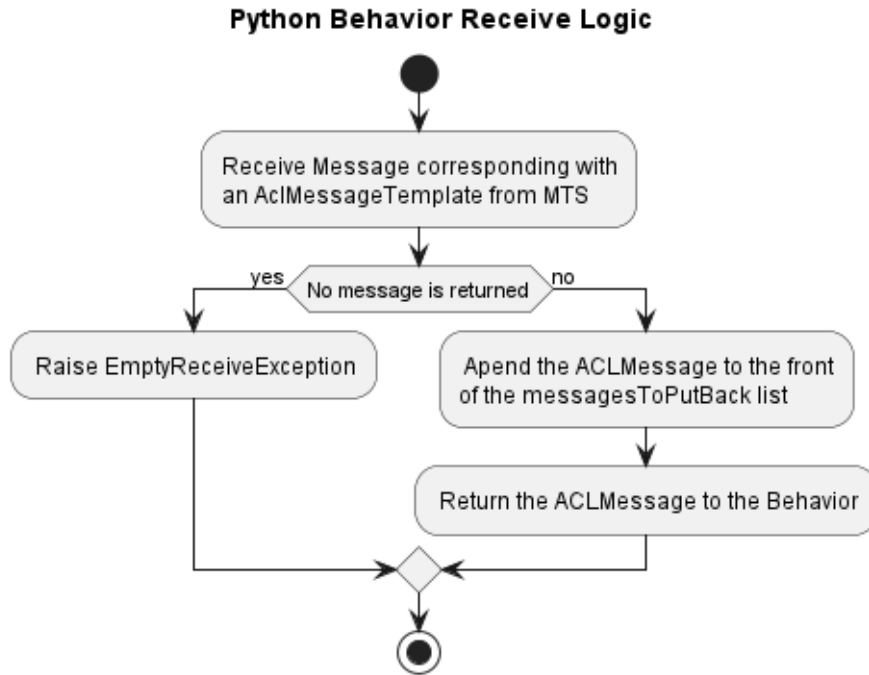


Figure 5.5: Python Behavior Receive Activity Diagram.

the blocking logic. As depicted in Figure 5.6, each execution cycle begins by clearing `messagesToPutBack`. It then clears the behavior's blocked flag and invokes `Action`. If an `EmptyReceiveException` is raised mid-execution, the blocked flag is set to active, and all messages buffered in `messagesToPutBack` are restored in their original reception sequence to the MTS asynchronous queue via `PutBack`. The messaging methods (`Send`, `Receive`, and `PutBack`) delegate to the MTS through the agent interface, preserving encapsulation since behaviors interact solely with their parent agent.

The `AMS` class, illustrated in Figure 5.7, encapsulates attributes for managing agent lifecycle and behavior scheduling. A `lifeCycleState` attribute governs scheduler state transitions. The `Start`, `Suspend`, `Resume`, and `Shutdown` methods manipulate this lifecycle state.

The Python implementation maintains distinct lists for active, blocked, pending addition, and pending removal behaviors, providing corresponding staging methods. Maintaining partitioned active and blocked collections, rather than a single unified list, significantly improves kernel efficiency by restricting execution passes strictly to active behaviors without evaluating blocked candidates. The `AddBehavior(behavior : Behavior)` method appends a behavior to `activeBehaviorsList`, while the `RemoveBehavior(behavior : Behavior)` method removes it. Behaviors can also be programmatically unblocked or blocked via `UnblockBehaviors()` and `BlockBehavior(behavior : Behavior)`, respectively.

The scheduling logic within the Python kernel comprises two sequential phases. The first phase, illustrated in Figure 5.8, processes behavior unblocking, additions, and removals:

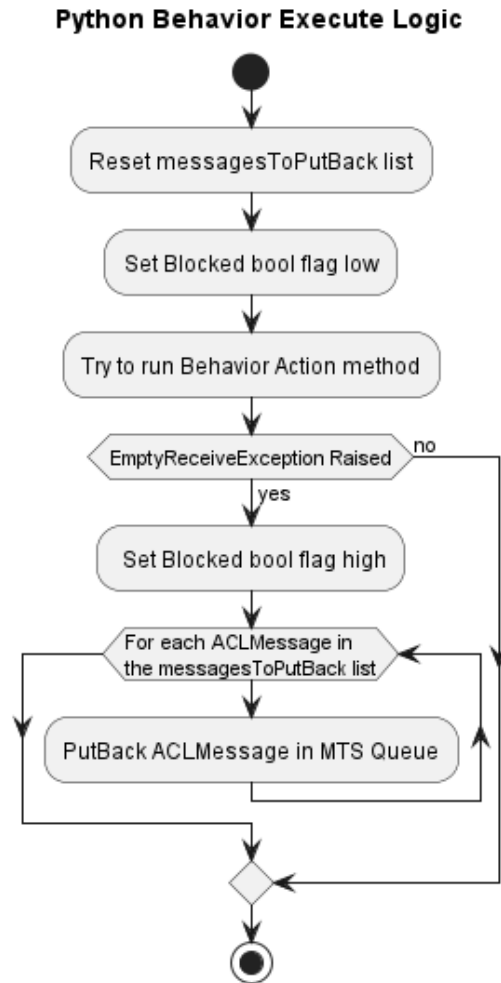


Figure 5.6: Python Behavior Execute Activity Diagram.

if `newMessageEvent` is set, it invokes `UnblockBehaviors()` to awaken blocked behaviors; any behaviors in the pending additions list are appended to `activeBehaviorsList`; and any behaviors in the pending removals list that are currently present in `activeBehaviorsList` are purged.

The second phase executes the scheduled behaviors. Behavior execution depends on `lifeCycleState`: in `INITIATED`, the kernel executes exclusively `InitializationBehavior`; in `ACTIVE`, it executes both `HeartBeatBehavior` and user-defined behaviors; and in `CLOSING`, it executes exclusively `ClosingBehavior`.

As illustrated in Figure 5.9, for each behavior in `activeBehaviorsList`, the kernel confirms that `lifeCycleState` remains `ACTIVE`; if the agent is suspended or stopped mid-execution, scheduling halts immediately. If a behavior has not yet started, its `OnStart` method is invoked and `Started` is called to set the flag. Next, the behavior's `Execute` method is called and `isBlocked` is checked. If set, the behavior has blocked and is transferred to `blockedBehaviorsList` via `BlockBehavior`, allowing the kernel to proceed to the next candidate. If the behavior remains unblocked, its `Done` method is evaluated: if

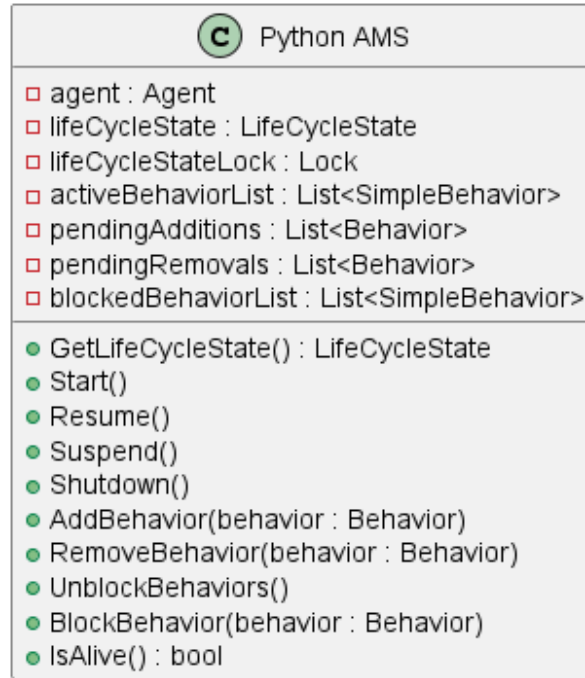


Figure 5.7: Python AMS Class Diagram.

True, `OnEnd` is invoked and the behavior is removed from `activeBehaviorsList`.

At the end of the kernel loop, exception handling catches errors related with registration or behavior execution (`AgentAlreadyExistsException`, `EMANoMorePortsException`, and `EMAFailureException`). Catching these exceptions transitions `lifeCycleState` to `CLOSING`, initiating orderly teardown.

Three standard internal behaviors were implemented as specified by the common specifications:

- **InitializationBehavior:** A `SimpleBehavior` capable of initializing both EMAs and generic agents. A UML activity diagram describing the `InitializationBehavior` logic is illustrated in Figure 5.10.

It begins by checking whether the agent is an EMA based on its identifier. If so, `StartTCPServer` binds the EMA to the predefined `emaAddress`, the agent's `Setup` method is called, and the behavior marks initialization complete. If the agent is not an EMA, `StartTCPServer` binds it to the predefined `registerAddress`, and `RegisterAgent` is invoked via `EMAInteraction` to initiate registration. When the response arrives, if it contains a new `Address`, the MTS server is stopped via `StopMessageServer` and rebound to the newly assigned operational address via `StartTCPServer`. Once bound to its operational address, the agent invokes `Setup`, attaches a `HeartBeatBehavior` via `AddBehavior`, and marks initialization complete. If the EMA response contains an `INFORM`, it signifies one of two scenarios: if the message designates the agent as an external participant, `registerAddress` is

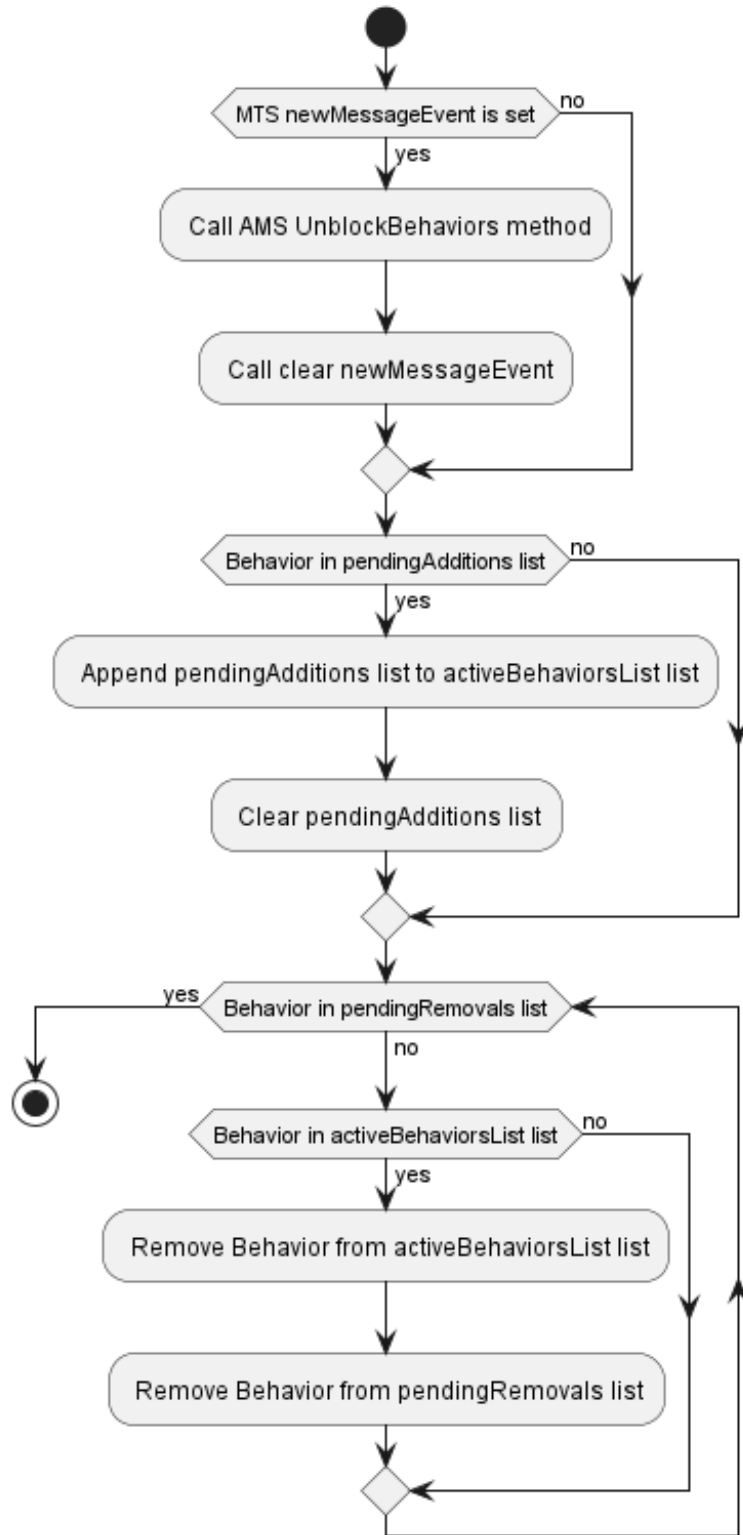
Python Kernel Unblocking, Additions and Removals Logic

Figure 5.8: Python AMS Kernel Behaviors Unblocking, Additions, and Removals Activity Diagram.

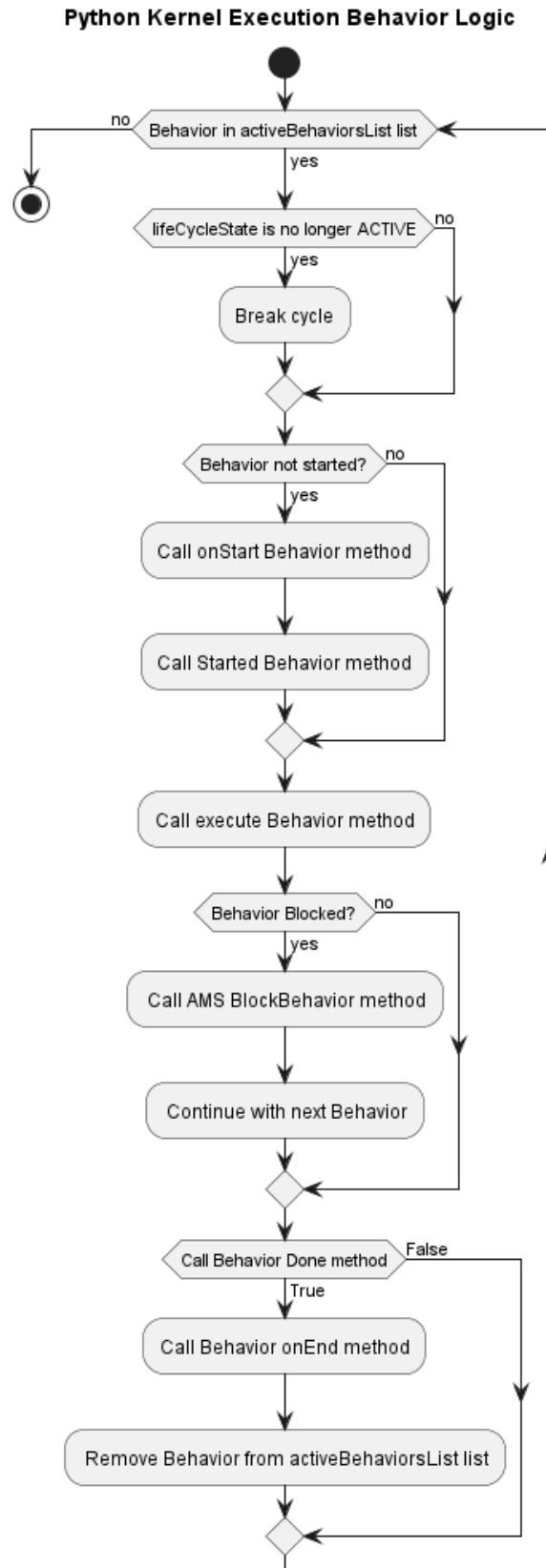


Figure 5.9: Python AMS Kernel Behavior Execution Logic Activity Diagram.

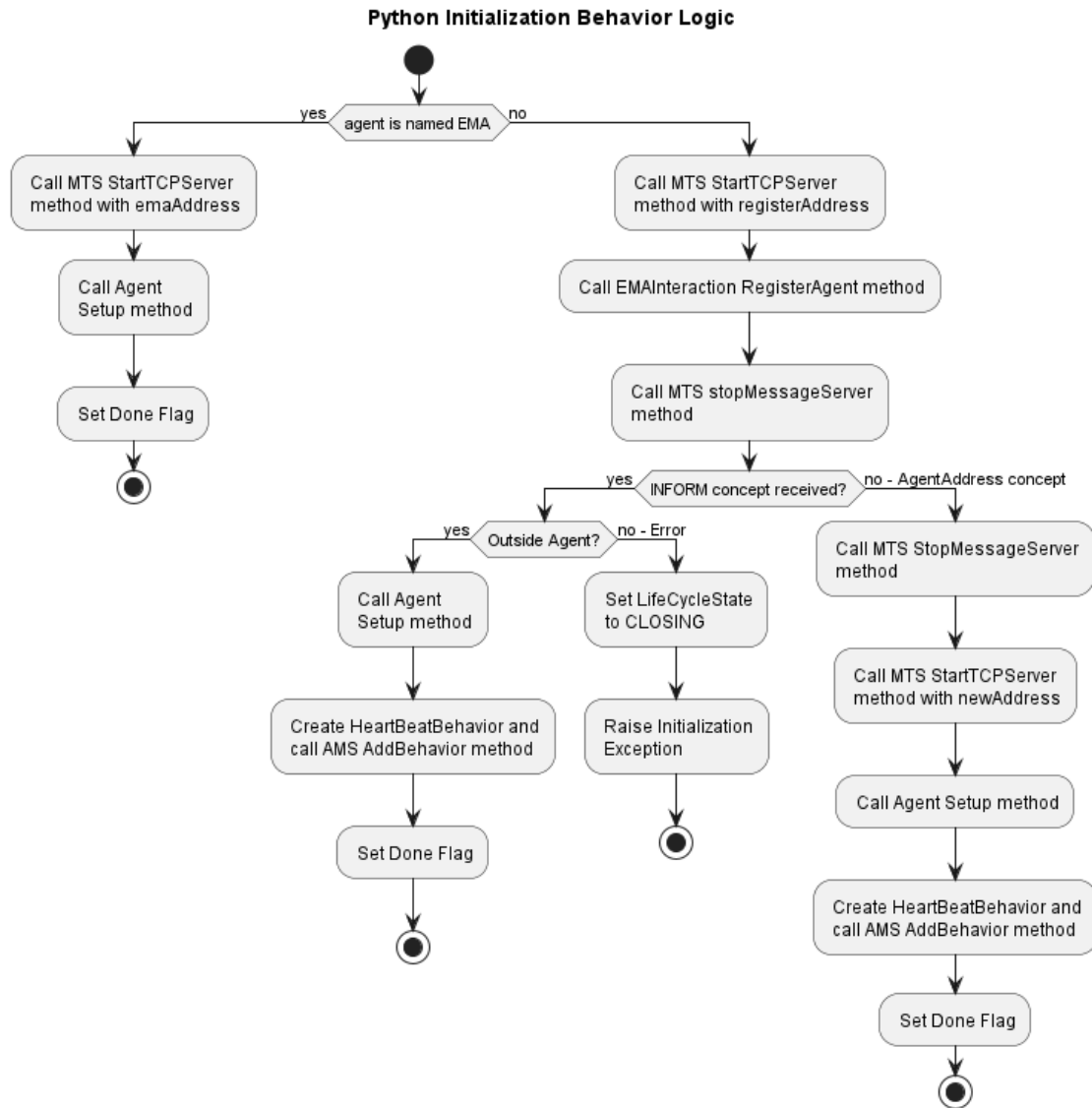


Figure 5.10: Python InitializationBehavior Logic Activity Diagram.

retained as the operational address, Setup is invoked, and initialization completes; if the message conveys an error, lifeCycleState is transitioned to CLOSING and a dedicated initialization exception is raised (`AgentAlreadyExistsException` or `EMANoMorePortsException`).

- **HeartBeatBehavior:** To implement the EMA Persistence Service, a `TickerBehavior` periodically sends keep-alive `ACLMessages` to the EMA by invoking `KeepAliveAgent` through `EMAInteraction` inside `OnTick`. This is the only behavior executed while the agent is in the `SUSPENDED` state.
- **ClosingBehavior:** A `OneShotBehavior` that sequentially calls the agent's `Takedown` method and invokes `DeRegisterAgent` via `EMAInteraction` when registered. If

registration failed during initialization, the deregistration call is bypassed.

5.3 EMA Python Implementation

The EMA, built on top of the Python reference implementation, focuses on the realization of the ASDS and EMA management services. These services employ `CyclicBehavior` instances for continuous request listening and multicast heartbeat reception, alongside `TickerBehavior` instances for periodic heartbeat broadcasts and persistence management. This section describes the ASDS implementation, outlines the services provided by the EMA, and details the underlying behaviors that support them.

5.3.1 ASDS

A crucial element present in the EMA is the ASDS. It was implemented as a class containing attributes that fulfil the requirements presented in Chapter 4:

- The local agent repository, implemented as a list of `AgentDescription` objects.
- The heartbeat counter for local agents, structured as a dictionary mapping each agent's `AgentID` (str) to its current missed-heartbeat count (int).
- A Python set maintaining the pool of available local port addresses.
- A timestamp recording the last update to the local agent repository, managed via the standard `time` module.
- The external agent repository, structured as a dictionary keyed by external EMA `AgentID` strings, where each entry contains a list of associated `AgentDescription` objects.
- The external EMA repository, structured as a dictionary keyed by external EMA `AgentID` strings, storing nested records that contain both the heartbeat counter and the last received timestamp for each remote EMA.

It also provides various methods for interacting with the various lists and dictionaries. Some of these methods already implement the data manipulation part of the management services hosted in the EMA:

- **RegisterAgent:** Registers local agents, implementing part of the Agent Registration Service. It verifies whether the agent already exists in the repository and checks its heartbeat counter. If the agent exists with a counter below the absence threshold, an `agent-already-registered` status is returned. If it has reached the absence threshold, the address recorded in that registry entry is returned. If no record exists, an available address is assigned and a new registry record is created.

- **RegisterEMA**: Registers remote EMAs and their associated `AgentDescription` records within the external EMA repository.
- **Modify**: Replaces an existing `AgentDescription` within the local repository with an updated instance, reflecting modified services or parameters.
- **DeRegister**: Removes an agent from the local repository and, if the agent is locally hosted, reclaims its address into the available port pool.
- **BeatAgent**: Resets the missed-heartbeat counter for a registered local agent upon receiving a keep-alive message.
- **BeatEMA**: Implements heartbeat processing for the Environment Discovery Service. If the transmitting EMA is not yet present in the external repository, a search `REQUEST` is dispatched and a new registry entry with its counter is created. If already registered, the received timestamp is compared against the recorded timestamp; if the received timestamp is newer, a search `REQUEST` is dispatched to synchronize state.
- **HandlePersistence**: Increments the heartbeat counters for all registered local and external agents. If an agent or remote EMA reaches the eviction threshold, it is deregistered via `DeRegister`. The local registry for the EMA itself is not incremented.
- **LocalSearch**, **ExternalSearch**, and **GlobalSearch**: Query the local repository, external repository, or both. They leverage matching methods on `AgentDescription` to evaluate query templates against registered agents.

5.3.2 Behaviors

To deliver the Agent Registration Service, Local Agent Persistence Service, Environment Persistence Service, and Environment Discovery Service, four dedicated behaviors were created:

- **ExternalHeartbeatSenderBehavior**: A `TickerBehavior` implementing the Environment Discovery Service by periodically broadcasting keep-alive messages containing the local repository's last-updated timestamp to the UDP multicast group via Python's standard socket module.
- **ExternalHeartbeatServerBehavior**: A `CyclicBehavior` that listens for multicast UDP heartbeats from peer EMAs and processes them via the `ASDS BeatEMA` method.
- **EnvironmentPersistenceBehavior**: A `CyclicBehavior` that executes both the Local Agent and Environment Persistence Services by periodically invoking the `ASDS HandlePersistence` routine.
- **RequestHandlerBehavior**: Receives and processes inbound `REQUEST` messages for registering, modifying, and deregistering agents, as well as executing local, external,

and global directory searches. It dispatches calls to the corresponding ASDS methods (`RegisterAgent`, `Modify`, `DeRegister`, `LocalSearch`, `ExternalSearch`, and `GlobalSearch`). Furthermore, it processes inbound `INFORM` messages conveying remote environment descriptions and keep-alive signals from local agents via `BeatAgent`.

5.4 Java Implementation

The Java implementation is designed for fog and cloud environments where strong static typing, high throughput, and standard Java Virtual Machine (JVM) multithreading are essential. The implementation itself is a mirror of the Python reference implementation; however, the underlying concurrency and I/O models are adapted to Java's concurrency utilities. Rather than relying on a single-threaded cooperative event loop as in Python's `asyncio`, the Java implementation utilizes a thread-pool architecture based on a shared `ScheduledExecutorService`. To prevent thread monopolization and race conditions while preserving high-density concurrency, the AMS kernel operates on an Actor-style execution model. An atomic guard (`AtomicBoolean isRunning`) guarantees that strictly one worker thread executes an agent's `kernelCycle` at any given time. Each execution pass advances behaviors for a single cycle and cooperatively yields the thread back to the pool, re-queuing only when further work is required. State synchronization across concurrent worker threads and network listeners is guaranteed through thread-safe collections, such as `CopyOnWriteArrayList` for active behaviors, and `synchronized` blocks protecting message queue operations and lifecycle state transitions.

As in Python, the `Agent`, `MTS`, `Behavior`, and `AMS` classes encapsulate the infrastructural logic. All these classes, alongside descriptive data objects (`ACLMessage`, `AgentID`, `Address`, `AgentDescription`, and `ServiceDescription`), are realized as standard Java classes. Payload serialization is handled using Jackson's `ObjectMapper`. An `EMAInteraction` helper class is also provided.

The `Agent` class, depicted in Figure 5.11, provides lifecycle hooks mirroring the Python architecture: `setup(EMAInteraction emaInteraction)` for startup logic and `takeDown(EMAInteraction emaInteraction)` for teardown routines, both designed to be overridden by the developer. These methods receive an `EMAInteraction` instance to streamline communication with the directory services. The `start`, `suspend`, `resume`, `stop`, and `isAlive` methods govern the agent lifecycle by delegating to the internal `AMS`. Behaviors are managed via `addBehavior` and `removeBehavior`, while messaging methods (`send`, `receive`, and `putBack`) dispatch, retrieve, and restore messages within the `MTS` queue. The `updateAgentDescription(AgentDescription agentDescription)` method updates the local registry record and dispatches an asynchronous modification behavior to synchronize state with the `EMA`.

The `MTS` class, illustrated in Figure 5.12, encapsulates network transport mechanisms. It stores the known environment coordinates `emaAddress` and `registerAddress`. The

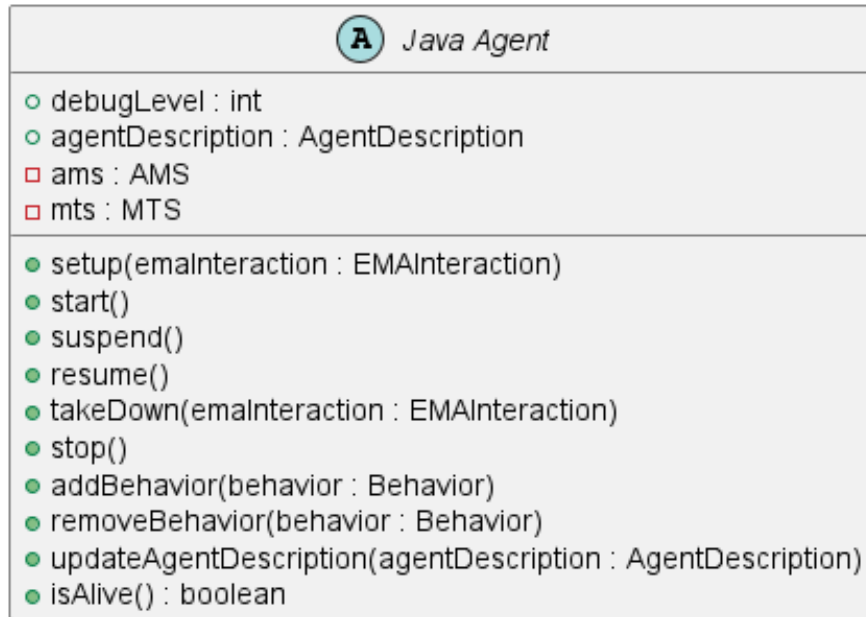


Figure 5.11: Java Agent Class Diagram.

`messageQueue` is implemented using a Java `List<ACLMessage>` synchronized with an explicit monitor object `messageQueueLock`. Network I/O is built entirely on non-blocking Java Non-blocking Input/Output (NIO) asynchronous socket channels: `startServer(int port)` binds an `AsynchronousServerSocketChannel` to the assigned port with an increased listen backlog of 500 to handle high-concurrency connection bursts, accepting inbound connections via non-blocking `CompletionHandler` callbacks. When an incoming transmission arrives, asynchronous loopers guarantee complete stream reception, the deserialized `ACLMessage` is queued, and the MTS immediately invokes `agent.ams.wakeUp()` to awaken the agent kernel without polling latency. A volatile boolean `newMessage` flag is also maintained. The `stopMessageServer` method cleanly shuts down the socket channel. For behavior messaging, the MTS provides: `send(ACLMessage msg)`, which transmits directly over an `AsynchronousSocketChannel` without spawning threads, framing the 4-byte payload length header followed by the JSON payload and reading a response acknowledgment before closing the channel to prevent socket reset errors; `receive(ACLMessageTemplate template)`, which searches the queue under `messageQueueLock`, removing and returning the first matching message (or null if none matches); and `putBack(ACLMessage msg)`, which prepends unconsumed messages back to the front of `messageQueue`.

The `Behavior` class, illustrated in Figure 5.13, is an abstract base class defining standard lifecycle hooks: `onStart`, `action`, `done`, and `onEnd`. It maintains references to the parent agent, an `emaInteraction` instance, execution flags `isStarted` and `isBlocked`, and the `messagesToPutBack` rollback buffer. The `block` and `unblock` methods update execution status, while `send` and `putBack` delegate to the corresponding MTS routines via the agent interface.

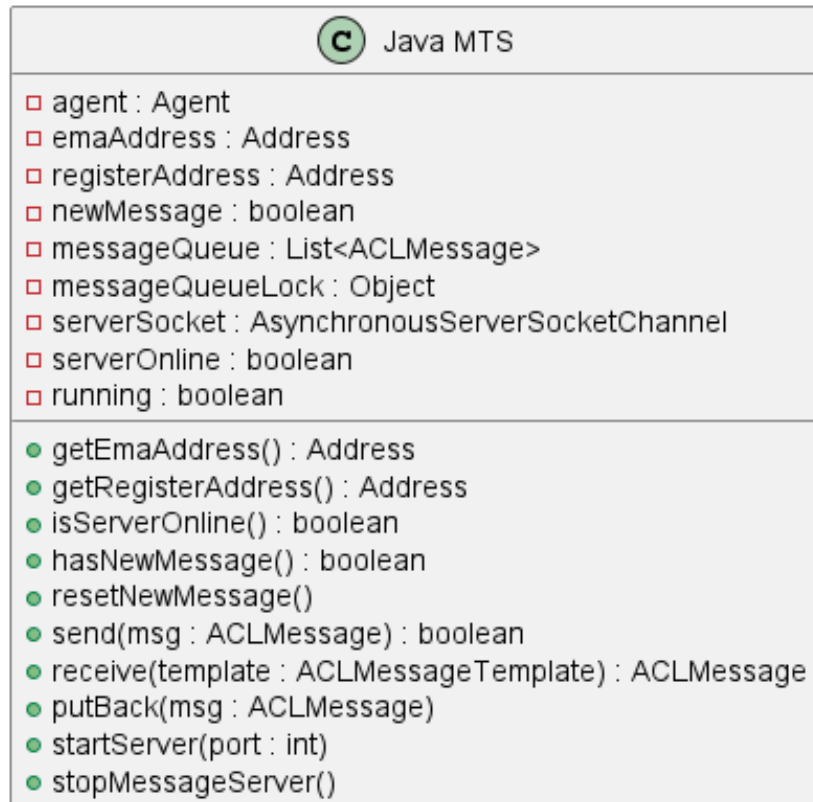


Figure 5.12: Java MTS Class Diagram.

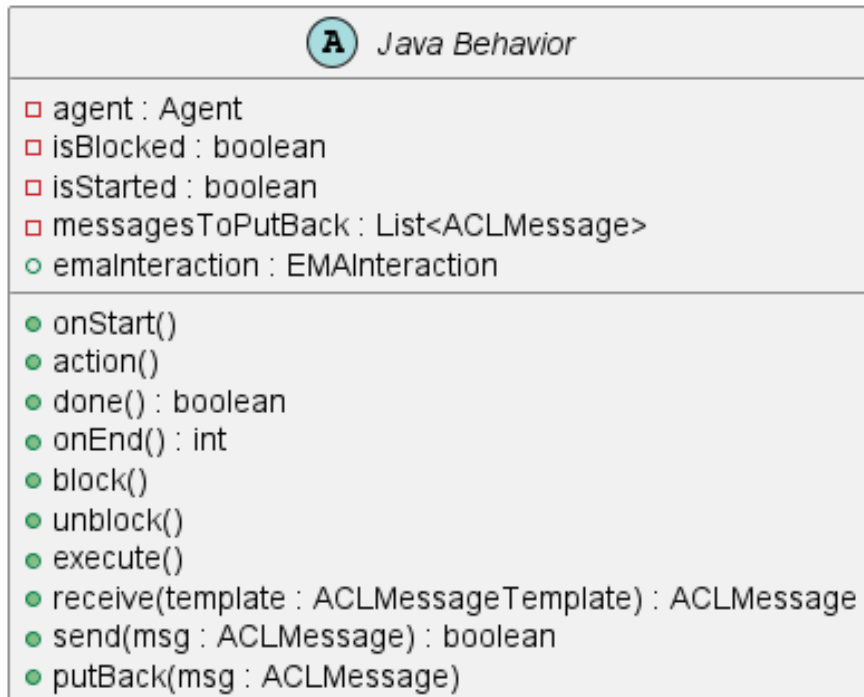


Figure 5.13: Java Behavior Class Diagram.

The cooperative multitasking and behavior blocking mechanism in Java directly replicates the exception-driven paradigm of the Python implementation, utilizing Java's native try/catch construct and a custom `EmptyReceiveException` (inheriting from `RuntimeException`). As depicted in Figure 5.14, when a behavior calls the `receive` method inside its action method, it queries the MTS via the Agent. If no matching message is returned (null), an `EmptyReceiveException` is thrown to halt further execution of action. If a message is found, it is added to the front of the `messagesToPutBack` buffer list to prevent message loss in case a subsequent receive fails before the cycle completes, and the message is returned.

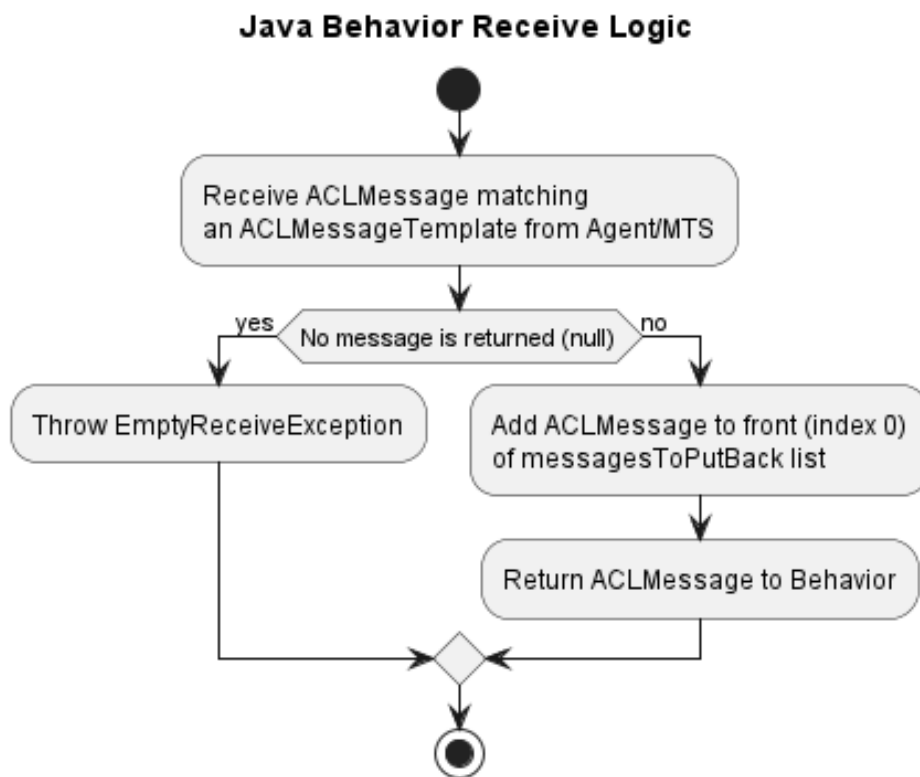


Figure 5.14: Java Behavior Receive Activity Diagram.

The enclosing `execute` method, shown in Figure 5.15, handles exception interception. At each invocation, it clears the `messagesToPutBack` list, unblocks the behavior (`isBlocked = false`), and invokes the action method inside a try/catch block. If `EmptyReceiveException` is caught, the behavior is marked blocked (`isBlocked = true`), and all messages accumulated in `messagesToPutBack` are restored to the front of the MTS `messageQueue` in their original order via the `putBack` method.

The AMS class, illustrated in Figure 5.16, governs lifecycle states and behavior scheduling. It maintains the current `lifeCycleState` guarded by `stateLock`, an atomic execution guard `isRunning`, a thread-safe `CopyOnWriteArrayList<Behavior>` for `activeBehaviors`, and `ArrayList<Behavior>` collections for `pendingAdditions`, `pendingRemovals`, and

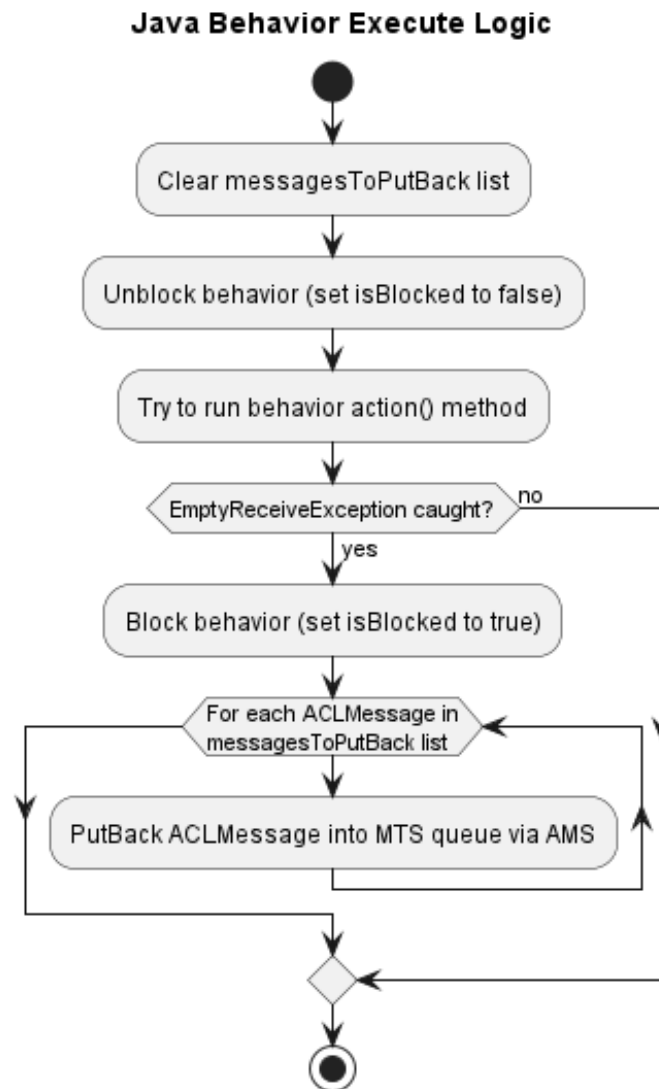


Figure 5.15: Java Behavior Execute Activity Diagram.

blockedBehaviors. The AMS exposes lifecycle management methods (start, suspend, resume, and shutdown), event-driven dispatch routines (wakeUp and scheduleWakeUp), and behavior staging routines (addBehavior, removeBehavior, blockBehavior, and unblockBehaviors).

The scheduling logic executed by the kernel inside the `kernelCycle` method is partitioned into three distinct segments. The first segment, shown in Figure 5.17, checks whether new messages arrived by evaluating the MTS new message flag using the `MTS hasNewMessage()` method. If true, it unblocks all waiting behaviors with the `unblockBehaviors()` method and clears the flag using the `resetNewMessage` method. Subsequently, any behaviors in `pendingAdditions` and `pendingRemovals` are respectively appended and removed from the `activeBehaviors` list and cleared. The `addBehavior()`, `removeBehavior()`, and `unblockBehaviors()` behavior management methods also wake

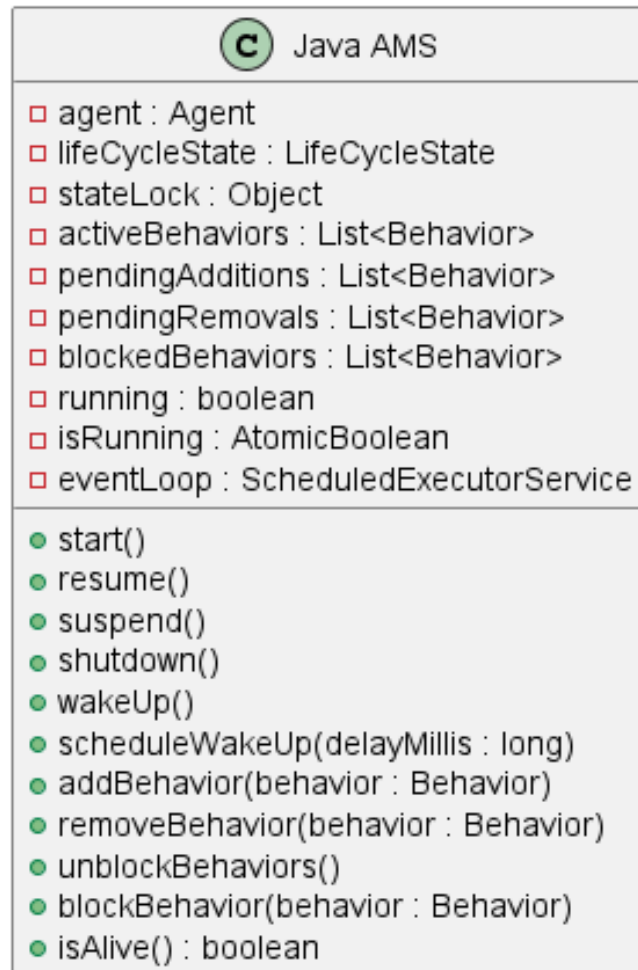


Figure 5.16: Java AMS Class Diagram.

up the agent kernel using the `wakeup()` method.

The second segment, illustrated in Figure 5.18, executes the scheduled behaviors. For each behavior in `activeBehaviors`, the kernel verifies that `lifeCycleState` is still `ACTIVE`, breaking the loop if the state changes. If the behavior has not yet started, its `onStart` method is invoked and the behavior is marked as started via `setStarted(true)`. Then, the `execute` method is called. If the behavior becomes blocked, it is moved from `activeBehaviors` to `blockedBehaviors` by calling the `blockBehavior` method. If the `done` method returns `true`, the kernel executes the `onEnd` method and removes the behavior from `activeBehaviors`.

In the third segment, following behavior execution, the kernel determines whether rescheduling is required rather than resubmitting the behavior unconditionally. As illustrated in the UML activity diagram of Figure 5.19, if immediate work remains, such as unconsumed messages, pending additions or removals, or active unblocked behaviors, the kernel invokes `wakeup()` to re-queue itself. If an active `TickerBehavior` or `WakerBehavior` is awaiting its next period or trigger time, the kernel queries its remaining

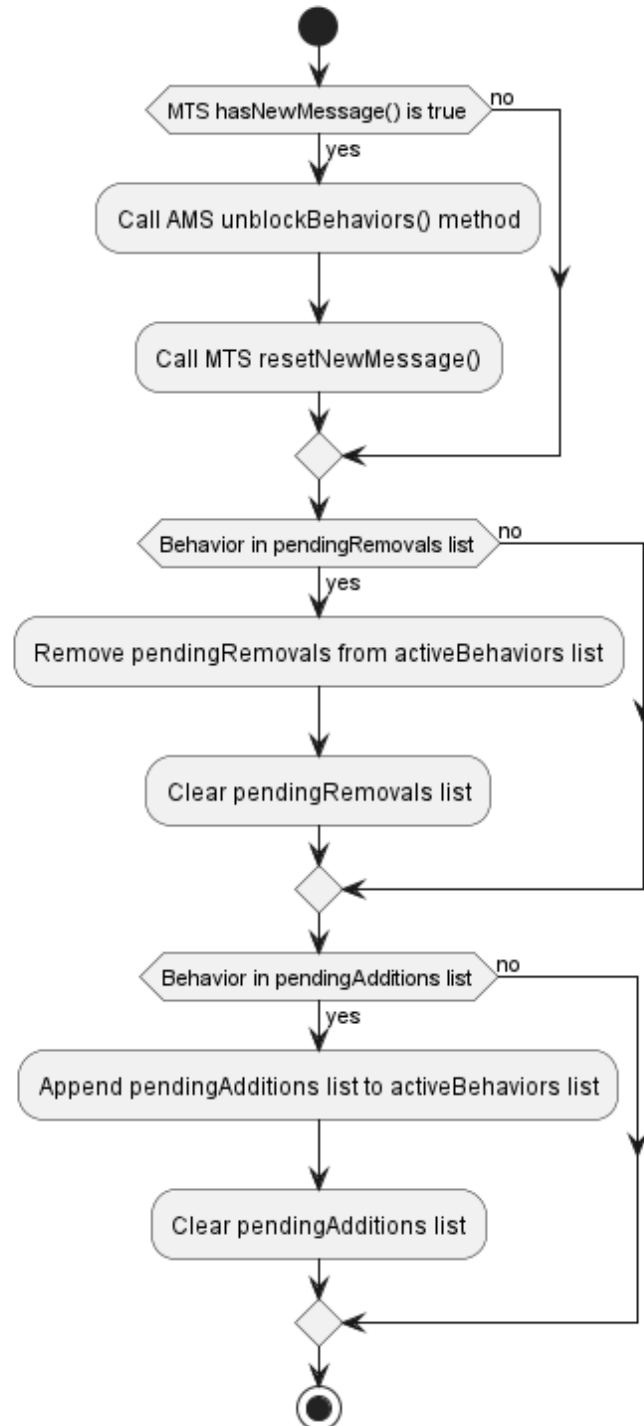
Java Kernel Unblocking, Additions and Removals Logic

Figure 5.17: Java AMS Kernel Behaviors Unblocking, Additions, and Removals Activity Diagram.

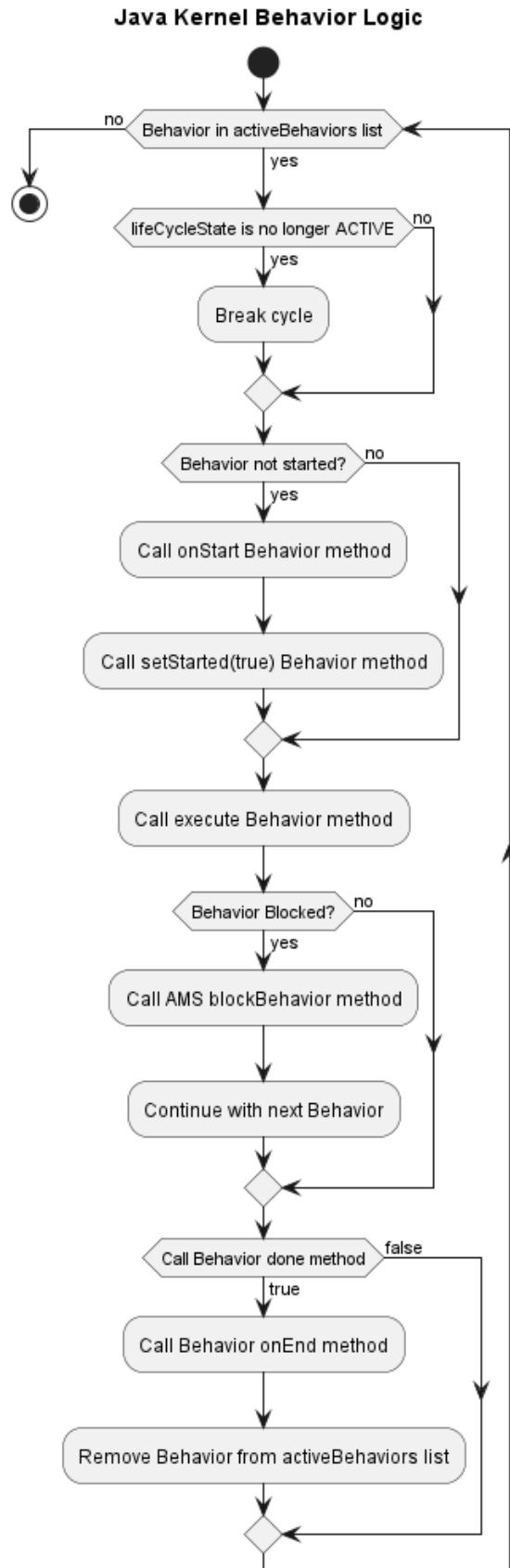


Figure 5.18: Java AMS Kernel Behavior Execution Logic Activity Diagram.

duration via `getRemainingTime()` and schedules a non-blocking timed wakeup through `scheduleWakeUp`. Otherwise, if all behaviors are blocked or idle, the kernel yields its worker thread back to the pool and remains dormant until awakened by an incoming message or lifecycle event.

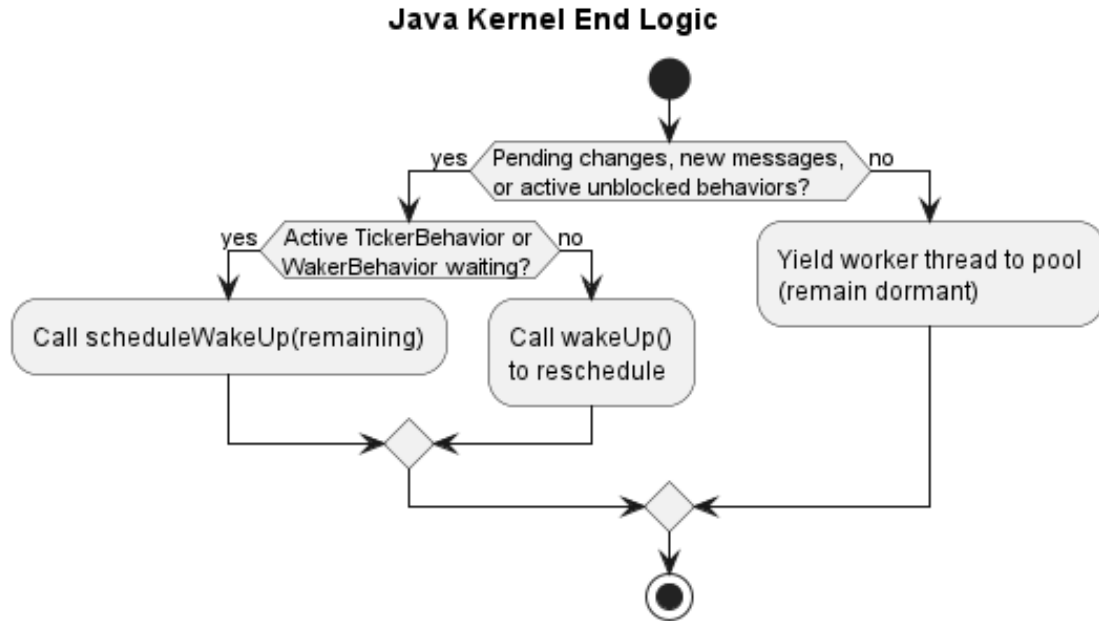


Figure 5.19: Java AMS Kernel End Execution Logic Activity Diagram.

The kernel cycle contains an exception handling wrap to catch any registration and operational faults (`AgentAlreadyExistsException`, `EMANoMorePortsException`, and `EMAFailureException`). When an unhandled exception occurs, `lifeCycleState` transitions to `CLOSING`, initiating agent teardown.

The three standard internal behaviors are implemented as inner classes within AMS:

- InitializationBehavior:** A `SimpleBehavior` whose logic is illustrated in Figure 5.20. It follows the identical protocol flow as the Python implementation. If the Agent is named EMA, it binds the server socket to `emaPort`, calls the Agent `setup` method, and marks initialization complete. Otherwise, it starts the server socket on `regPort` and invokes `registerAgent` via the `EMAInteraction` instance, subsequently closing the temporary server socket. If the EMA responds with an `INFORM` indicating an outside agent, it sets `isRegistered = true`, attaches a `HeartBeatBehavior`, invokes `setup`, and completes; if the `INFORM` conveys an error (`AgentAlreadyExists` or `EMANoMorePorts`), it sets the lifecycle state to `CLOSING` and throws an initialization exception. If the response contains an `AGENTADDRESS`, it restarts the MTS server bound to the newly assigned operational port, sets `isRegistered = true`, adds the `HeartBeatBehavior`, invokes the Agent `setup` method, and marks initialization done.

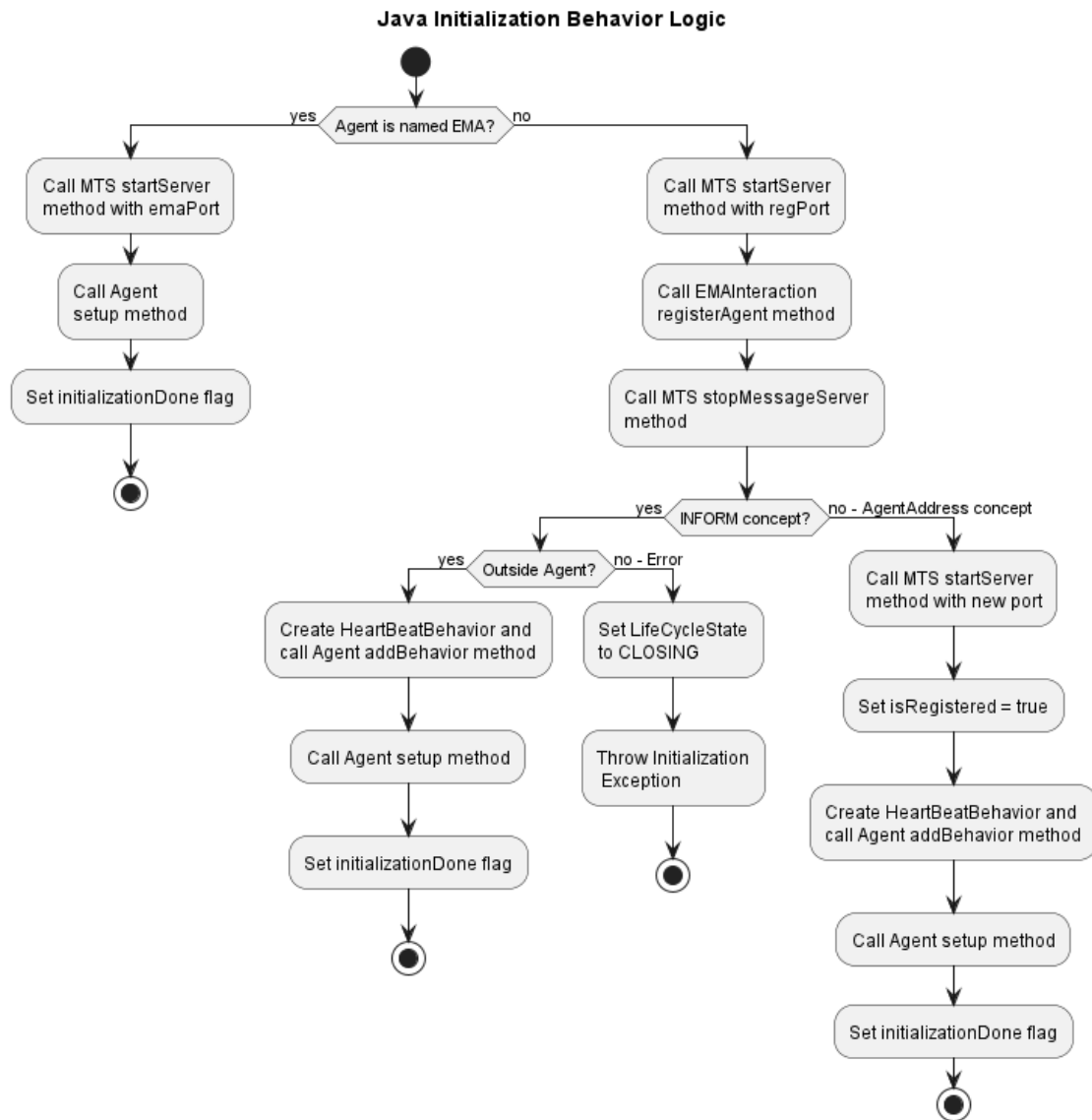


Figure 5.20: Java InitializationBehavior Logic Activity Diagram.

- **HeartBeatBehavior:** A `TickerBehavior` that periodically sends keep-alive messages to the EMA via the `emaInteraction.keepAliveAgent()` method. The kernel explicitly runs this behavior even while the agent is `SUSPENDED`.
- **ClosingBehavior:** A `OneShotBehavior` that invokes the agent's `takeDown` method, calls `emaInteraction.deRegisterAgent()` if registered, and closes the MTS server socket.

5.5 C++ Implementation

The C++ reference implementation targets edge resource-constrained embedded systems, developed specifically for dual-core ESP32 microcontrollers operating under Free Real-Time Operating System (FreeRTOS) and the Arduino framework. In this domain, hardware constraints such as limited SRAM, lack of virtual memory, strict stack depth limits, and FreeRTOS Watchdog Timer triggers preclude high-overhead runtime paradigms like garbage collection, necessitating deterministic manual heap management. However, C++ exceptions are explicitly enabled on the ESP32 platform to correctly implement the TAMAF infrastructure.

To prevent stack overflow and watchdog panics while maximizing throughput, hardware task pinning is employed across the ESP32's dual cores, cleanly partitioning network I/O from behavior execution. For every agent in the system, two 8 KB FreeRTOS background tasks are created and distributed across the cores. Core 0 groups together every `MTSLoopTask` to handle the MTS message servers, network I/O, packet stream framing, and JSON deserialization. Meanwhile, Core 1 executes every `AMSLoopTask` to manage the AMS behavior scheduling kernels and lifecycle state machines. Synchronization between the Core 0 network task and Core 1 behavior execution is accomplished using a thread-safe double-ended queue guarded by a FreeRTOS mutex semaphore. To notify Core 1 of incoming messages without latency, Core 0 atomically sets the `hasNewMessages` flag and issues a direct FreeRTOS task notification to immediately awaken `AMSLoopTask`, if other tasks are running on Core 1.

By pairing hardware-aware task pinning with exception-driven cooperative multi-tasking, the C++ implementation preserves functional equivalence with the TAMAF specifications.

The core classes encapsulating infrastructural logic are the `Agent`, `AMS`, `MTS`, and `Behavior` classes. Descriptive objects such as `ACLMessage`, `ACLMessageTemplate`, `AgentID`, `Address`, `AgentDescription`, and `ServiceDescription` utilize the `ArduinoJson` library for efficient serialization and zero-copy JSON parsing. An `EMAInteraction` helper class is also provided.

The `Agent` class, depicted in Figure 5.21, serves as the unified user-facing abstraction aggregating identity and environmental attributes within `agentDescription`. It composes the `MTS` and `AMS` instances, exposing lifecycle management and messaging routines. It

defines the pure virtual `Setup(EMAInteraction* emaInteraction)` lifecycle hook for initialization logic and `TakeDown(EMAInteraction* emaInteraction)` for teardown routines. Lifecycle operations (`Start`, `Stop`, `Suspend`, and `Resume`) and behavior management operations (`AddBehavior`, `RemoveBehavior`, `BlockBehavior`, and `UnblockBehaviors`) are delegated to the underlying AMS, while messaging calls (`Send`, `Receive`, and `PutBack`) are routed to the MTS. The `Loop` method provides an optional synchronization point with the Arduino main loop, coordinating safely with the autonomous `AMSLoopTask`.

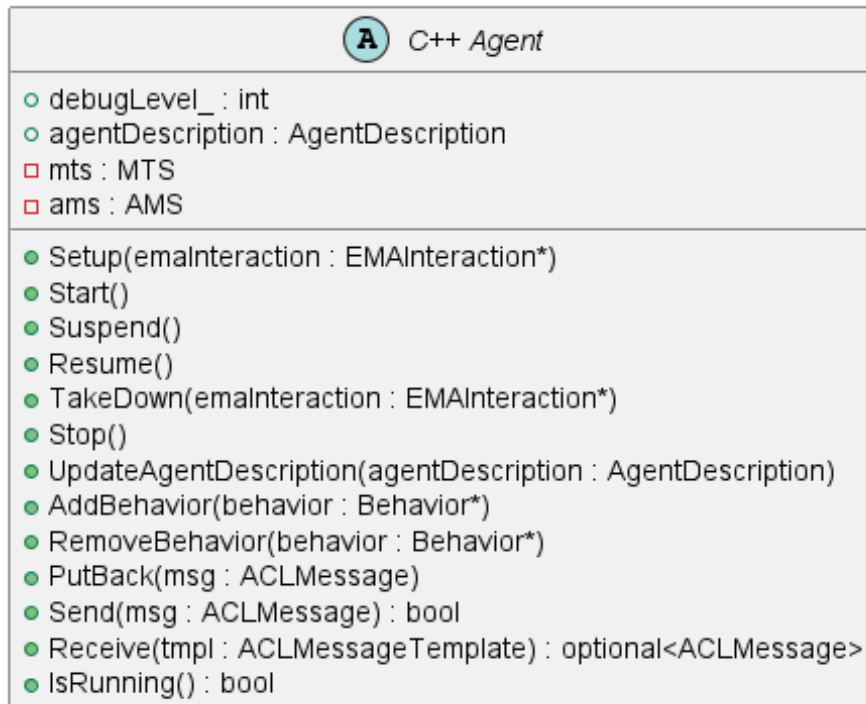


Figure 5.21: C++ Agent Class Diagram.

A critical deviation from the garbage-collected Python and Java environments is memory management. In the C++ implementation, dynamic memory on microcontrollers must be carefully managed to avoid fragmentation and leaks. In this case, the `Agent` class assumes full heap ownership of all dynamically allocated `Behavior` instances passed to `AddBehavior`. When a behavior's `Done` method returns true, the kernel calls `OnEnd`, deallocates its heap memory via `delete`, and removes it from `activeBehaviors`. Furthermore, upon agent destruction, all remaining behaviors across `activeBehaviors`, `blockedBehaviors`, and `pendingAdditions` are systematically deleted to ensure zero memory leakage.

The `MTS` class, illustrated in Figure 5.22, manages physical TCP communications on the ESP32 using the native `WiFiServer` and `WiFiClient` libraries. It holds a `WiFiServer` instance, a FreeRTOS task handle `mtsTaskHandle`, a double-ended message queue to easily push messages to the front or back (`std::deque<ACLMessage> messageQueue`), and a FreeRTOS mutex semaphore `queueMutex`. To protect against microcontroller RAM exhaustion, the MTS enforces a strict payload size limit of 64 KB. Calling `StartServer(address :`

Address) creates the `MTSLoopTask` where incoming client connections have their 4-byte big-endian length header read, followed by the JSON payload, which is deserialized into an `ACLMessage` using `ArduinoJson`. The message is placed into the asynchronous `messageQueue` under `queueMutex`, and the AMS is notified via FreeRTOS direct task notification (`NotifyNewMessage()`).

Crucially, adopting `std::deque<ACLMessage>` rather than a standard first-in, first-out queue enables `PutBack(const ACLMessage& msg)` to atomically restore messages to the front of the queue under mutex protection. The `Receive(const ACLMessageTemplate& tmpl = ACLMessageTemplate())` method retrieves, from `messageQueue`, the first message matching the specified template. For outbound communication, `Send(const ACLMessage& msg)` transmits the message via `WiFiClient` directly to each designated recipient.

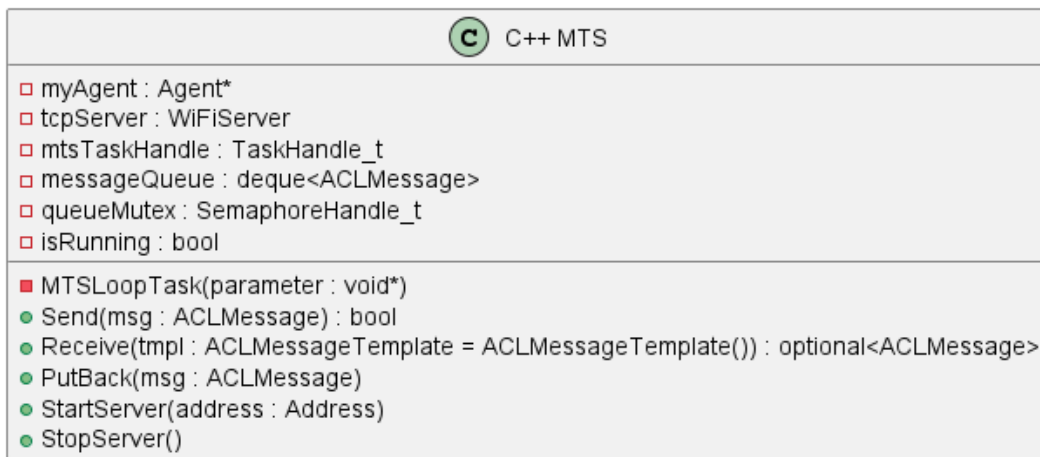


Figure 5.22: C++ MTS Class Diagram.

The `Behavior` class, illustrated in Figure 5.23, defines the standard lifecycle methods: `OnStart`, pure virtual `Action`, pure virtual `Done`, and `OnEnd`. Internal state is tracked using boolean flags (`startedFlag`, `blockedFlag`, and `doneFlag`) alongside a rollback buffer (`std::vector<ACLMessage> messagesToPutBack`). The class also exposes `Block`, `Unblock`, and `Restart` methods for managing the state of the behavior. Messaging methods (`Send` and `PutBack`) delegate to the enclosing `Agent`, which forwards them to the MTS. An `EMAInteraction` instance is also present for every behavior.

The cooperative multitasking and behavior blocking mechanism in C++ directly mirrors the exception-driven paradigm of Python and Java through the enabled `-fexceptions` build flag. As depicted in Figure 5.24, the primary `Receive(tmpl)` method queries the MTS message deque. If no matching message is found in the queue, an `EmptyReceiveException` is immediately thrown. If a matching message is found, it is inserted at the beginning of the local `messagesToPutBack` rollback buffer list to guard against message loss if a subsequent operation interrupts the behavior cycle, and the message is returned directly to the caller.

The enclosing `Execute` method, shown in Figure 5.25, encapsulates exception handling

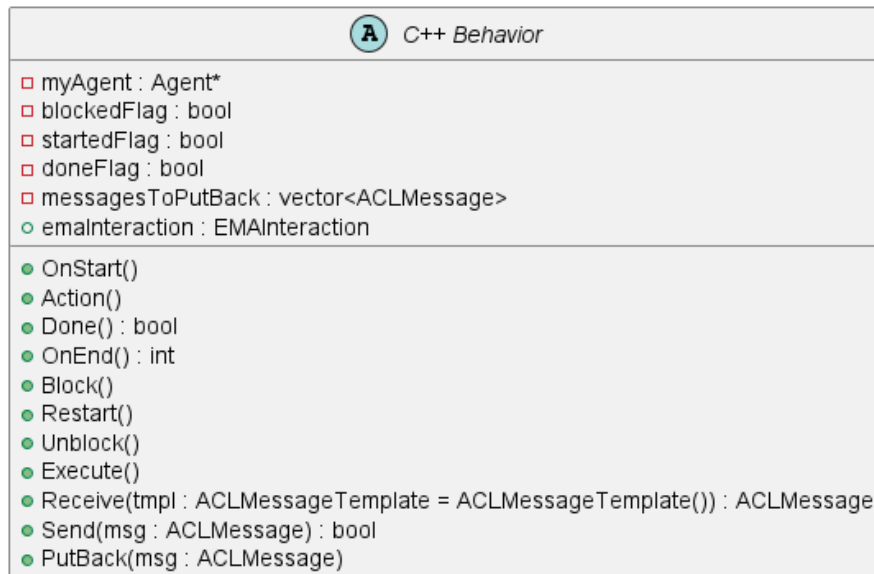


Figure 5.23: C++ Behavior Class Diagram.

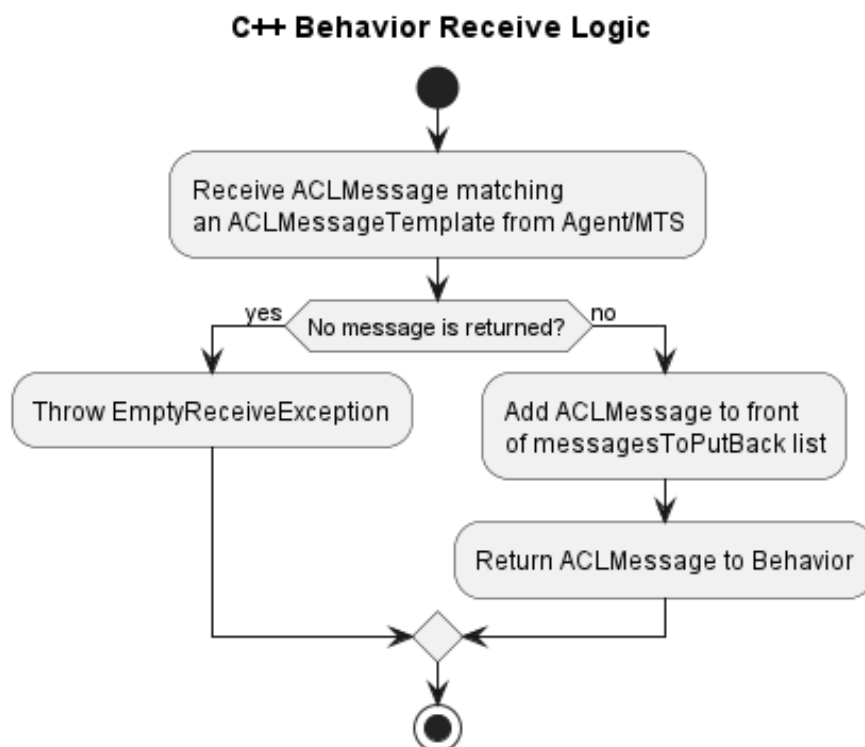


Figure 5.24: C++ Behavior Receive Activity Diagram.

and atomic state rollback. At each invocation, it clears the local `messagesToPutBack` list, resets the blocked state via `Restart`, and invokes the `Action` method inside a try/catch block. If an `EmptyReceiveException` is caught mid-execution, the method immediately calls `Block` (`blockedFlag = true`) and performs an atomic rollback by iterating through all messages accumulated in `messagesToPutBack` and restoring them to the front of the MTS `messageQueue` in their original sequence via the `PutBack` method. If `Action` completes without raising an exception, all consumed messages are retained.

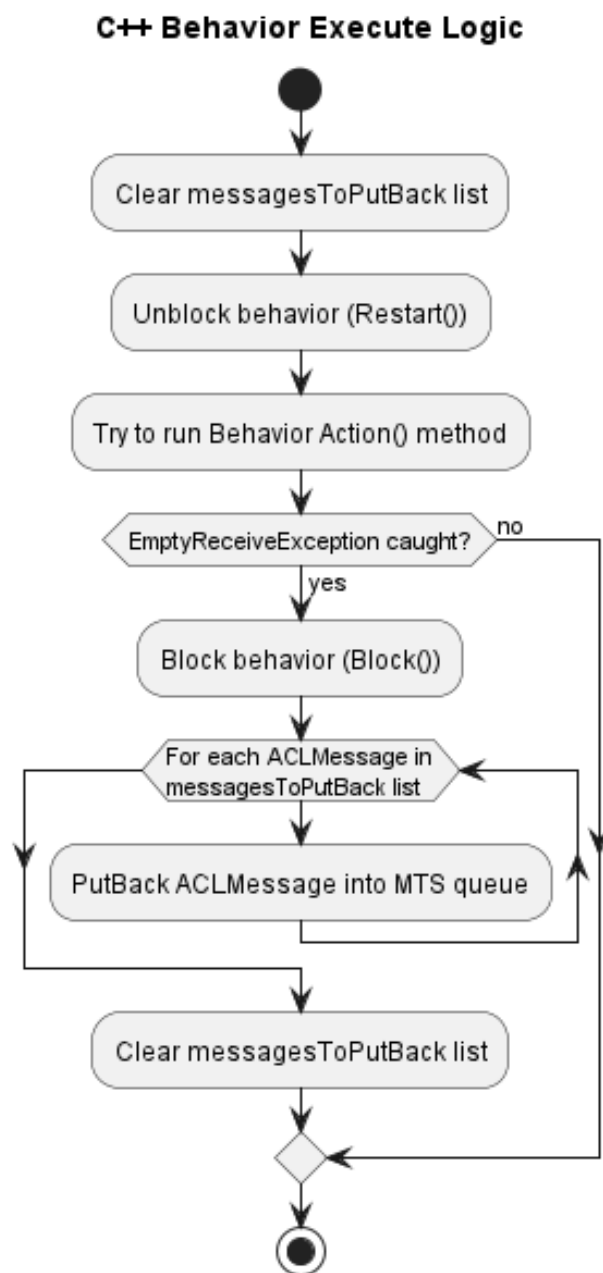


Figure 5.25: C++ Behavior Execute Activity Diagram.

The AMS class, illustrated in Figure 5.26, governs lifecycle state transitions and behavior scheduling. Symmetrically to the Python and Java implementations, it maintains `lifeCycleState`, the `activeBehaviors` and `blockedBehaviors` collections, staging collections for thread-safe behavior additions and removals (`pendingAdditions` and `pendingRemovals`), a FreeRTOS task handle `amsTaskHandle` for the kernel task, and the atomic notification flag `hasNewMessages` for the MTS task to awaken. The class provides lifecycle methods (Start, Stop, Suspend, and Resume) and behavior queueing routines (AddBehavior, RemoveBehavior, BlockBehavior, and UnblockBehaviors).

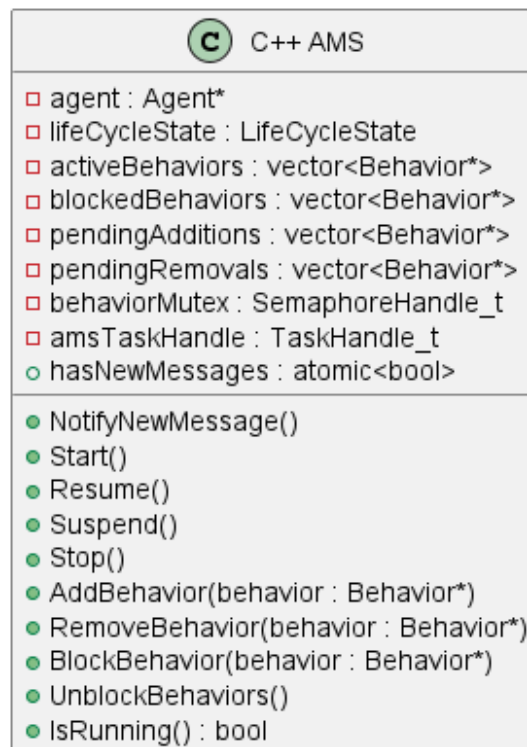


Figure 5.26: C++ AMS Class Diagram.

The behavior execution logic inside the AMS kernel is partitioned into two distinct segments. The first phase, illustrated in Figure 5.27, processes behavior unblocking, additions, and removals. It evaluates and resets the atomic flag `hasNewMessages`. If a new message has arrived, it unblocks behaviors according to the current `lifeCycleState`: in `INITIATED`, it unblocks `InitializationBehavior`; in `SUSPENDED`, it unblocks `HeartBeatBehavior`; and in `ACTIVE`, it calls `UnblockBehaviors()`, which invokes `Unblock()` on every behavior in `blockedBehaviors`, transferring them back into `activeBehaviors`. Following unblocking, any behaviors in `pendingRemovals` that are present in `activeBehaviors` are erased from both collections. Subsequently, all behaviors in `pendingAdditions` are bound to the agent via `setAgent(agent)`, appended to `activeBehaviors`, and purged from `pendingAdditions`.

The second segment contains the logic that effectively executes behaviors. The behaviors run by the kernel are determined by `lifeCycleState`: in `INITIATED`, it executes

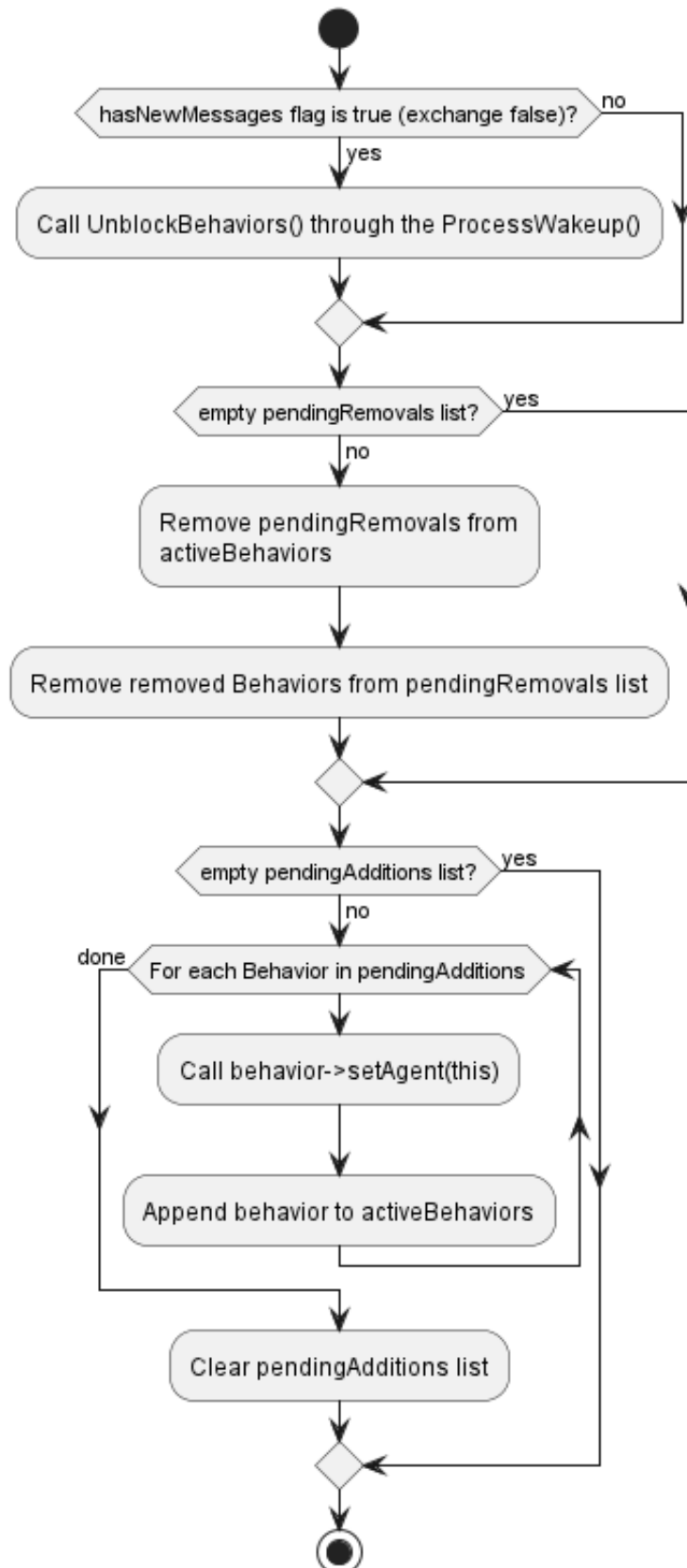
C++ Kernel Wakeup, Additions and Removals Logic

Figure 5.27: C++ Kernel Wakeup and Behavior Transfer Activity Diagram.

only `InitializationBehavior`; in `ACTIVE`, it executes both `HeartBeatBehavior` and user-defined behaviors; in `SUSPENDED`, it executes only `HeartBeatBehavior`; and in `CLOSING`, it executes only `ClosingBehavior`.

Figure 5.28 illustrates the execution of behaviors, specifically for the `ACTIVE` state. It cycles through `activeBehaviors`, completely bypassing blocked behaviors, eliminating wasteful polling cycles. For each behavior in `activeBehaviors`, the kernel confirms that `lifeCycleState` is still `ACTIVE`, breaking if the state has changed. If the behavior has not yet started (`IsStarted` returns false), the `OnStart` method is invoked and `Started` is called to set the flag. The kernel then calls the `Execute` method, which runs the behavior `Action` method within its exception wrapper. If the behavior becomes blocked during execution (verified by calling the `IsBlocked` method), it is immediately extracted from `activeBehaviors` and appended to `blockedBehaviors`. Only if the behavior remains unblocked does the scheduler evaluate `Done`: if true, the `OnEnd` method is called, the heap memory is deallocated with `delete`, and the pointer is erased from `activeBehaviors`. Otherwise, iteration advances to the next active behavior.

Furthermore, robust exception handling wraps the entire kernel cycle to catch registration and operational faults (`AgentAlreadyExistsException`, `EMANoMorePortsException`, `EMAFailureException`, and generic exceptions). Whenever an unhandled exception occurs, the AMS immediately transitions the agent into the `CLOSING` state, initiating teardown.

The three standard internal behaviors handle lifecycle and EMA coordination:

- **InitializationBehavior:** A `SimpleBehavior` whose logic is illustrated in Figure 5.29.

If the Agent is the EMA, the server binds to `emaAddress` with the MTS `StartServer` method, invokes the `Agent Setup` method, and completes immediately. Otherwise, it starts the server socket on `registerAddress` using the same MTS `StartServer` method and calls `RegisterAgent` from the `EMAInteraction` instance. When a response arrives: if it is an `INFORM` indicating an outside agent, it marks registration successful; if it conveys an error, it throws a dedicated initialization exception rather than resetting the board: `AgentAlreadyExistsException` or `EMANoMorePortsException`. If the response contains an `AGENTADDRESS` concept, it rebinds the MTS server to the newly assigned operational address with the MTS `StartServer` method and marks registration successful. Once registered, it creates the `HeartBeatBehavior`, registers it with the Agent, invokes the `Agent Setup` method, and sets the `initializationDone` flag to true.

- **HeartBeatBehavior:** A `TickerBehavior` that periodically dispatches keep-alive messages to the EMA via the `KeepAliveAgent` method of the `EMAInteraction` instance. It is scheduled to run even while the agent is in the `SUSPENDED` state to prevent eviction by the EMA.

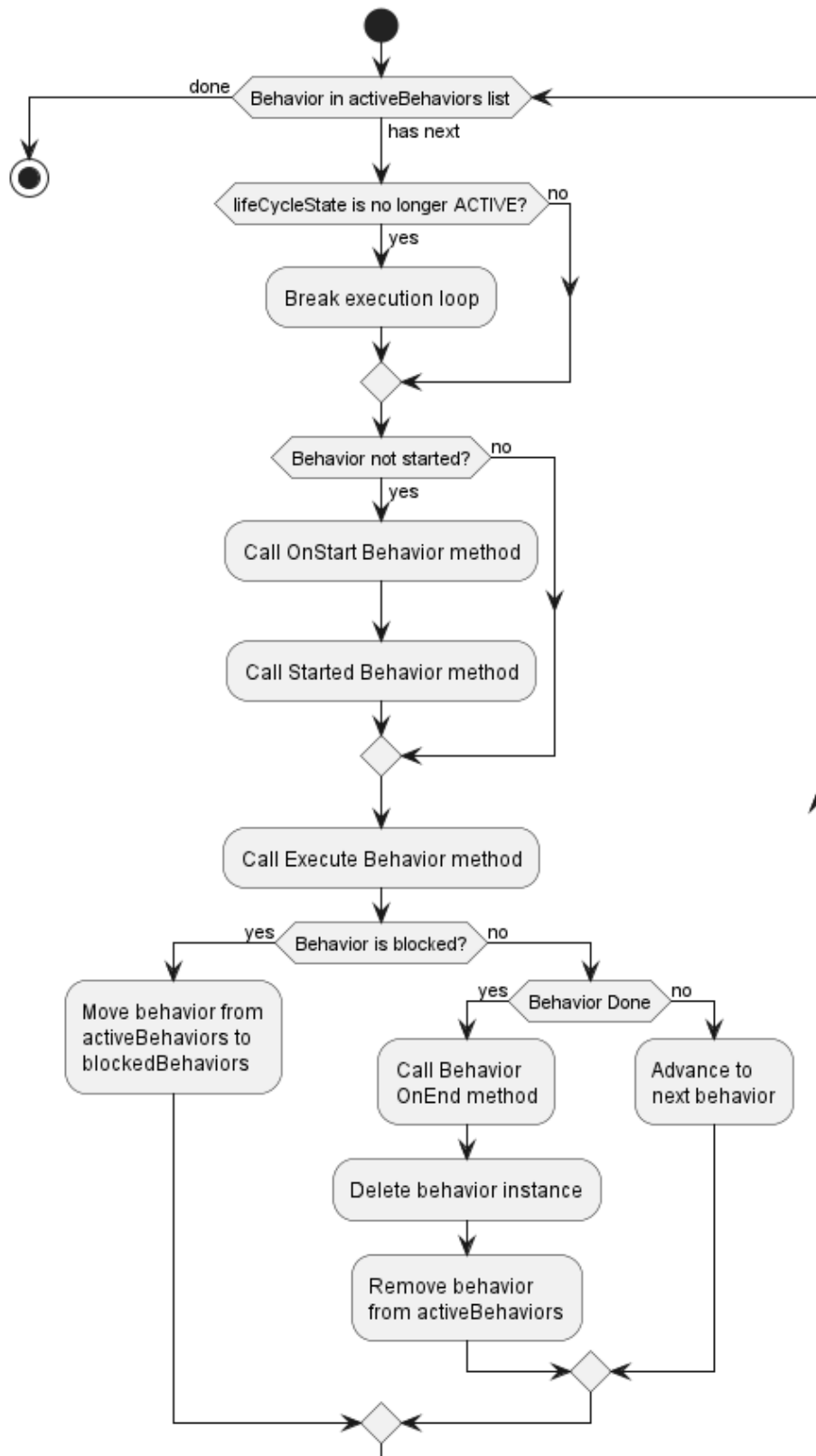
C++ Kernel Behavior Execution Logic

Figure 5.28: C++ Kernel Behavior Execution Logic Activity Diagram.

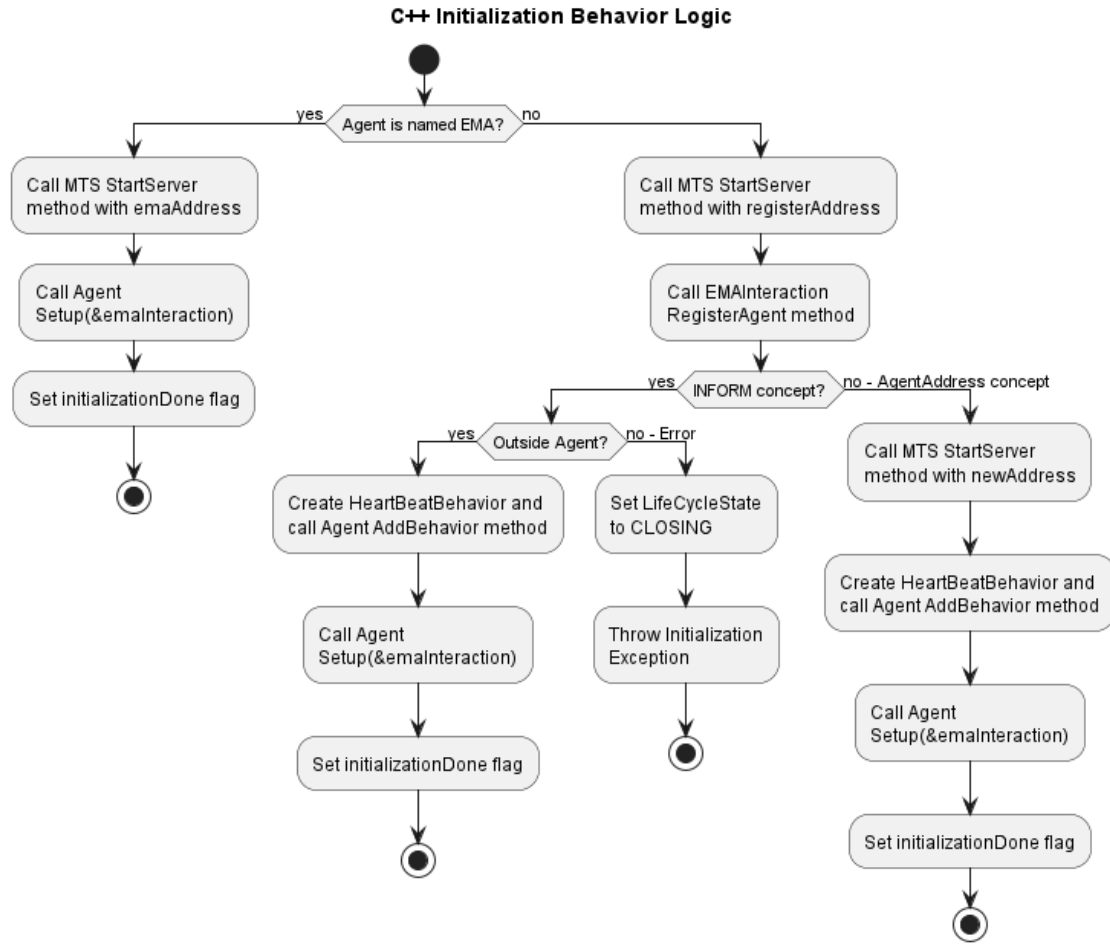


Figure 5.29: C++ InitializationBehavior Logic Activity Diagram.

- **ClosingBehavior:** A OneShotBehavior that invokes the agent’s TakeDown routine, attempts to deregister the agent via the EMAInteraction DeRegisterAgent method inside a try/catch block to prevent blocking during network disconnects, and cleanly closes the MTS server socket using StopServer.

The complete source code for all three platform implementations of the proposed framework is provided online in the project repository at <https://github.com/NOVA-RICS-Open-Lab/technology-agnostic-multi-agent-framework>.

A comprehensive empirical evaluation covering functionality, latency, and throughput, accompanied by detailed comparative analysis, is presented in Chapter 6.

VALIDATION AND TESTING

To verify the feasibility and efficacy of the proposed framework, a comprehensive evaluation of the three implementations was divided into two distinct phases: quantitative stress testing and a qualitative functionality use case. The stress testing phase benchmarks the framework’s core message transport system and agent registration mechanisms against a recognized industry standard to evaluate its operational scalability and latency bounds under heavy concurrent loads. Following this, the functionality use case demonstrates the framework’s practical applicability in a heterogeneous laboratory industrial scenario, thereby validating its primary objective of true language and platform independence across diverse hardware architectures and software paradigms.

All tests were conducted using a Windows 11 Home Laptop Computer (LC) equipped with an Intel Core i7-10750H CPU (6 cores, 12 threads) and 16 GB of RAM, a 4 GB Raspberry Pi 4 Model B, an ESP32 DevKit V1, and a TP-Link TL-WR940N router.

6.1 Stress Testing

To comprehensively evaluate the robustness and scalability of TAMAF compared to established standards, a comparative stress test was conducted against JADE, the predominant, benchmark Java-based MAS framework. The performance evaluation focused specifically on the message transport layer and the directory facilitation mechanisms under heavy concurrent loads. To isolate the effects of the programming language runtime from the protocol design, TAMAF was evaluated using both its Python and Java implementations.

To ensure an unbiased evaluation of peak concurrent load, free from network latency variability, an analogous testing infrastructure was developed for all platforms. The primary objective was to quantify the duration required for a given number of agents to register with the EMA, as well as the latency incurred by a receiving agent when enqueueing incoming messages into the MTS asynchronous queue. This benchmarks not only message transport latency, but also the total time required for the infrastructure to register agents, given that TAMAF, as previously detailed, employs a decentralized, sequential (one-by-one) registration approach.

Each test scenario instantiated a single Receiver Agent alongside N Sender Agents, evaluating network scales of $N \in \{1, 5, 10, 15, 20, 30, 40, 50, 75, 100, 200\}$. All Sender Agents were synchronized using a scheduled future timestamp calculated based on the network scale N , compelling them to wait for one another to register with the EMA, block execution, and subsequently release their message payloads to the designated Receiver agent simultaneously. The total elapsed duration required to register all agents was recorded to determine the overall registration overhead. Upon receiving each message, the Receiver added it to the internal message queue and promptly responded with an acknowledgment. Each sender recorded the transmission timestamp and logged the arrival of the acknowledgment, allowing the exact Round-Trip Time (RTT) to be calculated.

Due to hardware availability constraints regarding physical ESP32 microcontrollers, only the Python and Java implementations underwent stress testing. For every scenario, the test was replicated ten times with the results averaged to dissipate any type of statistical anomaly that may originate from the TCP connections. To systematically analyze the results, three widely adopted candidate regression models (linear, quadratic, and power-law) were fitted to the registration time and RTT data. Given the analysis objective and context, the coefficient of determination (R^2) and the Root Mean Square Error (RMSE) were selected as the primary evaluation metrics, as they adequately capture both the variance explained by the model and the absolute magnitude of its errors.

6.1.1 Registration Time

The benchmarking results yielded distinct operational profiles for the frameworks. As illustrated in Figure 6.1, the average infrastructure initialization times reveal the underlying differences in how each runtime handles concurrent TCP socket and port binding.

In Tables 6.1, 6.2, and 6.3, the regression analysis of the agent registration data is presented for JADE, Python, and Java, respectively.

Table 6.1: Regression Models and Evaluation Metrics for JADE.

Model	Equation	R^2	RMSE
Linear	$y = 0.0005x + 0.0050$	0.9947	0.0018
Quadratic	$y = 0.0000x^2 + 0.0004x + 0.0059$	0.9955	0.0017
Power Law	$y = 0.0010 \cdot x^{0.8609}$	0.9862	0.0030

Table 6.2: Regression Models and Evaluation Metrics for Python TAMAF.

Model	Equation	R^2	RMSE
Linear	$y = 0.2865x - 4.8557$	0.9462	3.8137
Quadratic	$y = 0.0011x^2 + 0.0763x - 0.5089$	0.9969	0.9182
Power Law	$y = 0.0090 \cdot x^{1.6564}$	0.9976	0.7983

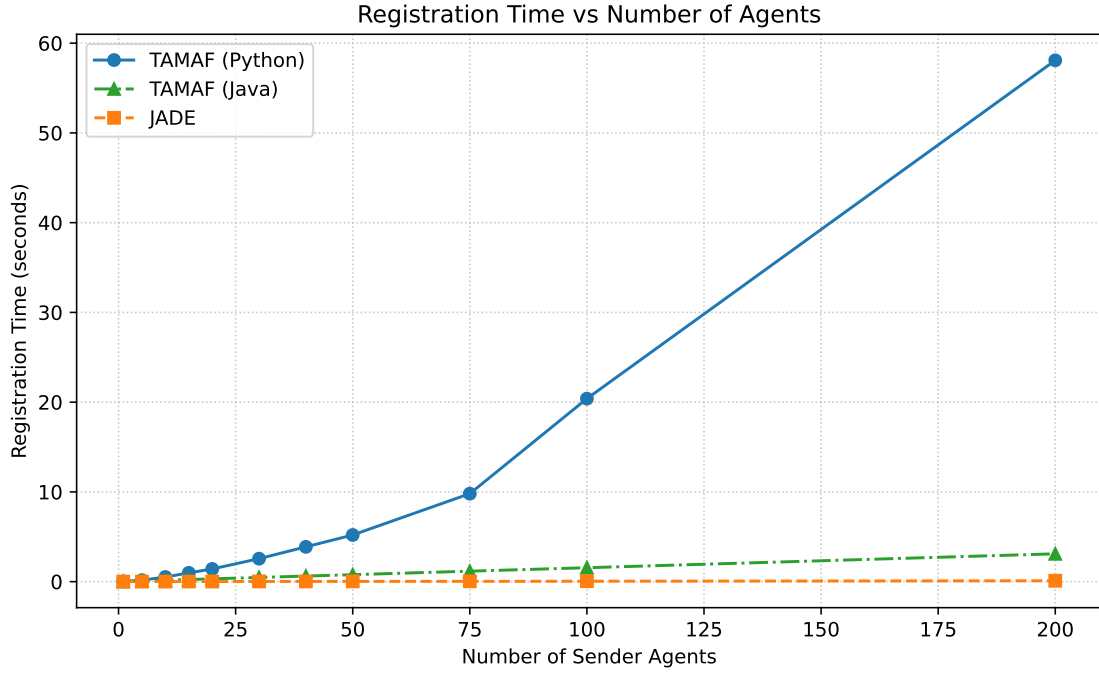


Figure 6.1: Agent Registration Time vs. Network Scale.

Table 6.3: Regression Models and Evaluation Metrics for Java TAMAF.

Model	Equation	R^2	RMSE
Linear	$y = 0.0155x + 0.0030$	1.0000	0.0014
Quadratic	$y = -0.0000x^2 + 0.0155x + 0.0028$	1.0000	0.0014
Power Law	$y = 0.0157 \cdot x^{0.9971}$	1.0000	0.0019

This regression analysis quantitatively characterizes the infrastructure initialization profiles of each platform. For JADE, the quadratic ($R^2 = 0.9955$, $\text{RMSE} = 0.0017$) and power-law ($R^2 = 0.9862$, $\text{RMSE} = 0.0030$) models describe an initialization that remains nearly instantaneous across all tested scales (0.105 s at $N = 200$), as all agents reside within a single shared JVM container without binding individual external network sockets.

In contrast, for Python TAMAF, the power-law model yields the highest coefficient of determination ($R^2 = 0.9976$, $\text{RMSE} = 0.7983$), with a superlinear exponent of $b = 1.6564$. This confirms that Python’s registration overhead grows as $\approx O(N^{1.66})$ due to asynchronous socket binding serialization and TCP scheduler contention, causing the simple linear model to perform poorly ($R^2 = 0.9462$, $\text{RMSE} = 3.8137$).

Conversely, Java TAMAF achieves its highest correlation with the power-law model ($R^2 = 1.0000$, $\text{RMSE} = 0.0019$), where its scaling exponent $b = 0.9971 \approx 1.0$ confirms effectively linear scalability. Furthermore, the linear model for Java attains an outstanding fit ($R^2 = 1.0000$, $\text{RMSE} = 0.0014$), demonstrating that Java TAMAF binds sockets with a predictable and near-constant marginal cost of ≈ 15.5 ms per agent on top of a baseline

initialization of 3.0 ms.

While JADE registers 200 agents internally in a fraction of a second (0.105 s), TAMAF exhibits near-linear growth in registration time relative to the number of agents due to its fully distributed architecture, which requires every agent to register with the EMA endpoint and subsequently initialize its own independent TCP/IP server on a unique port. Although JADE's containerized approach remains the fastest in absolute initialization, Java TAMAF proves that a fully distributed, multi-port socket architecture can still initialize rapidly when backed by a highly optimized multithreaded runtime.

6.1.2 Round-Trip Time

The average latency data, depicted in Figure 6.2, reveals the impact of heavy synchronization on message throughput. A detailed comparative analysis of the frameworks demonstrates stark differences in how their respective runtimes handle massive concurrent networking tasks.

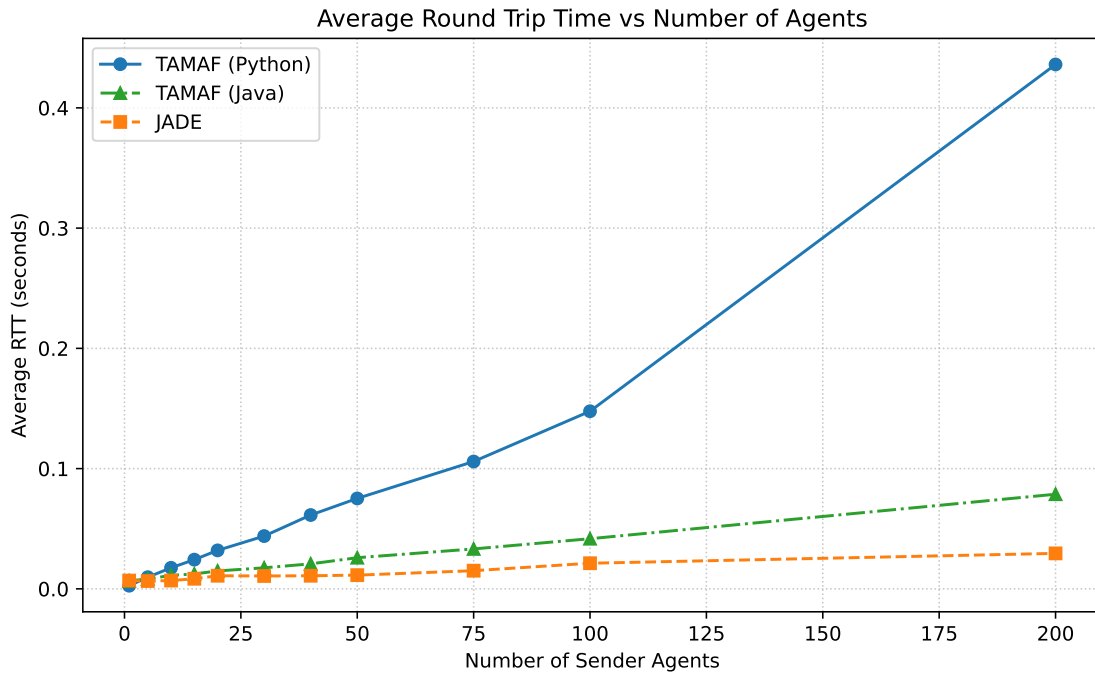


Figure 6.2: Average Message Round-Trip Time vs. Network Scale.

In Tables 6.4, 6.5, and 6.6, the regression analysis of the RTT data is presented for JADE, Python, and Java, respectively.

Table 6.4: Regression Models and Evaluation Metrics for JADE.

Model	Equation	R^2	RMSE
Linear	$y = 0.0001x + 0.0067$	0.9688	0.0012
Quadratic	$y = -0.0000x^2 + 0.0001x + 0.0062$	0.9727	0.0011
Power Law	$y = 0.0022 \cdot x^{0.4762}$	0.8983	0.0021

Table 6.5: Regression Models and Evaluation Metrics for Python TAMAF.

Model	Equation	R^2	RMSE
Linear	$y = 0.0021x - 0.0162$	0.9647	0.0222
Quadratic	$y = 0.0000x^2 + 0.0009x + 0.0090$	0.9978	0.0055
Power Law	$y = 0.0004 \cdot x^{1.3418}$	0.9913	0.0110

Table 6.6: Regression Models and Evaluation Metrics for Java TAMAF.

Model	Equation	R^2	RMSE
Linear	$y = 0.0004x + 0.0069$	0.9990	0.0006
Quadratic	$y = 0.0000x^2 + 0.0003x + 0.0071$	0.9990	0.0006
Power Law	$y = 0.0013 \cdot x^{0.7623}$	0.9799	0.0028

JADE exhibits highly predictable near-linear behavior ($R^2 = 0.9727$, RMSE = 0.0011 for quadratic and $R^2 = 0.9688$, RMSE = 0.0012 for linear), showing a baseline internal delivery latency of 6.7 ms and an incremental scaling overhead of only 0.1 ms per agent. As a framework native to Java, JADE effectively leverages thread pooling and object-level serialization.

For Python TAMAF, the quadratic model achieves the highest explanatory power ($R^2 = 0.9978$, RMSE = 0.0055), closely followed by the power-law model with a superlinear exponent of 1.3418. This quadratic elevation in latency is a classic symptom of the Python Global Interpreter Lock (GIL) and its single-threaded event loop architecture, which becomes a bottleneck under heavy parallel bursts, forcing the average waiting time to scale with N .

Conversely, Java TAMAF’s latency is best explained by the linear model ($R^2 = 0.9990$, RMSE = 0.0006), as the second-order term in the quadratic model is effectively zero ($0.0000x^2$). Java’s thread pool parallelizes JSON deserialization and routing across multiple CPU cores, yielding a baseline latency of 6.9 ms and an incremental queuing cost of 0.4 ms per additional concurrent agent. It exhibits a base latency closely matching JADE’s 6.7 ms baseline (6.9 ms vs. 6.7 ms). When tested using 200 agents, it scales robustly and linearly, processing the message burst with an average RTT of 0.0869 s (86.9 ms).

This 86.9 ms RTT is substantially faster than Python TAMAF’s 403.8 ms, representing a greater than fourfold reduction in latency under peak concurrency. While JADE’s

in-memory containerized messaging incurs lower queuing overhead at peak scale (26.7 ms at $N = 200$) due to purely intra-JVM thread communication, the TAMAF implementation, also built on Java, demonstrates that a fully decentralized, multi-port TCP architecture can still maintain sub-100 ms round-trip communication. By operating directly peer-to-peer, TAMAF avoids central broker bottlenecks, confirming that decentralized multi-agent communication remains highly responsive under extreme concurrency.

6.2 Functionality Use Case

To validate the framework's interoperability and hardware- and software-agnostic capabilities, a representative industrial use case based on a modular CPPS was developed.

It consists of a physical production system divided into multiple segments, each controlled by an agent. The goal is to have different implementations of TAMAF controlling these agents and collaborating to accomplish a common goal, in this case, assembling a product that requires drilling, fastening (screwing), or both. The workflow encompasses not only the execution of these manufacturing skills but also the transport of the workpiece across the production line.

The physical setup comprises a Fischertechnik automation kit controlled by an Industrial Shields 57R+ PLC. It features a circular track made of transport belts and a machining tower equipped with a rotary tool head. The workpiece itself is a box fitted with metal sides designed to trigger the transport system's magnetic sensors. The PLC that controls the sensors and actuators exposes TCP/IP services related to movement steps (from one belt to another) and provides two specialized skill services for the tower: `screw` and `drill`.

On the TAMAF side, three distinct types of agents were used to model the system:

- **Product Agent:** Represents individual workpieces, managing their production recipes and execution states. It is highly proactive in completing its tasks, frequently initiating communication.
- **Skilled Agent:** Provides manufacturing services and task propositions upon request. Each Skilled Agent is associated with a physical workstation or position in the production system where its services are executed.
- **Transport Agent:** Manages workpiece transport logistics, providing conveyance services and routing proposals upon request.

Specifically for this use case, a Tower Agent was implemented as a specialization of the Skilled Agent to provide the two tower machining services. Multiple Skilled Agents may be involved in the production process other than the Tower Agent, such as human operator stations or robotic stations coupled with the kit at different locations, offering other types of services (e.g., `paint` and `inspect`). However, only the Tower Agent was considered, as it was sufficient to accomplish the objectives defined for this use case. Figure 6.3 depicts the

physical experimental kit, indicating the controlled electromechanical modules alongside their corresponding controlling agents.

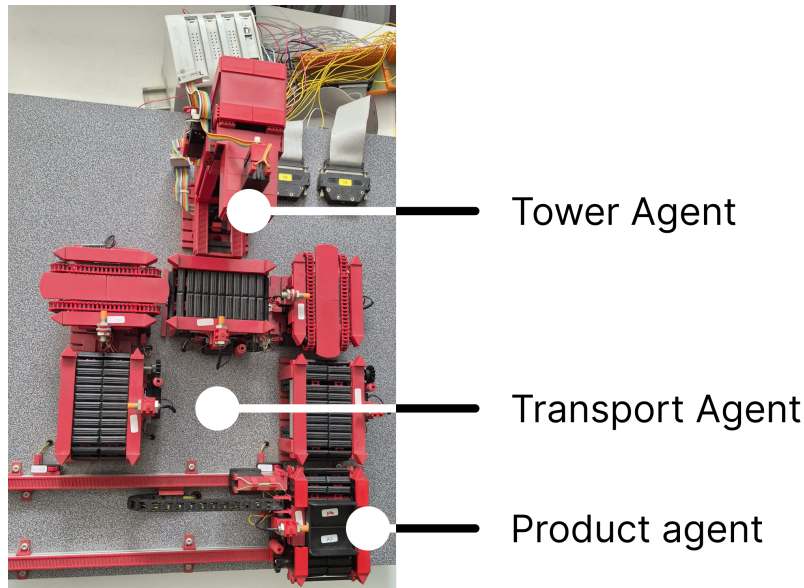


Figure 6.3: Physical Fischertechnik Automation Kit and Agent Modules.

The manufacturing cycle initiates with the introduction of a modular box workpiece with a unique production recipe. To fulfill this recipe, the screw and/or drill skills specified within it must be applied. To accomplish this, the part moves between the stations that provide the necessary skills, and once processing is complete, it is dropped off at a designated exit location. Figure 6.4 provides a detailed technical UML sequence diagram detailing the messages and actions performed by the agents throughout the production lifecycle.

The technical sequence begins with the instantiation of the Product Agent. It checks the next step in its production process and queries the EMA for Skilled Agents that offer the corresponding service. Upon identifying candidate agents, it issues a cfp. A Skilled Agent is selected based on proposal criteria; in this scenario, the closest workstation is preferred. If the workpiece is already positioned at the Skilled Agent's workstation, a service request is issued, the skill is executed, and the cycle advances to the next step. If the workpiece is not located where the Skilled Agent operates, the Product Agent queries the EMA for a Transport Agent and requests conveyance to the target location. Upon arrival, the sequence resumes with the service request to the Skilled Agent.

All agents were built entirely upon the developed TAMAF libraries and interacted exclusively using standard FIPA Contract Net Protocol and FIPA Request Protocol specifications, with the exception of internal EMA search requests and periodic keep-alive messages. To rigorously confirm platform independence, the agents were deployed across a highly heterogeneous network environment. The deployment consisted of:

- A LC hosting Java and Python agents concurrently, alongside a Python-implemented EMA.

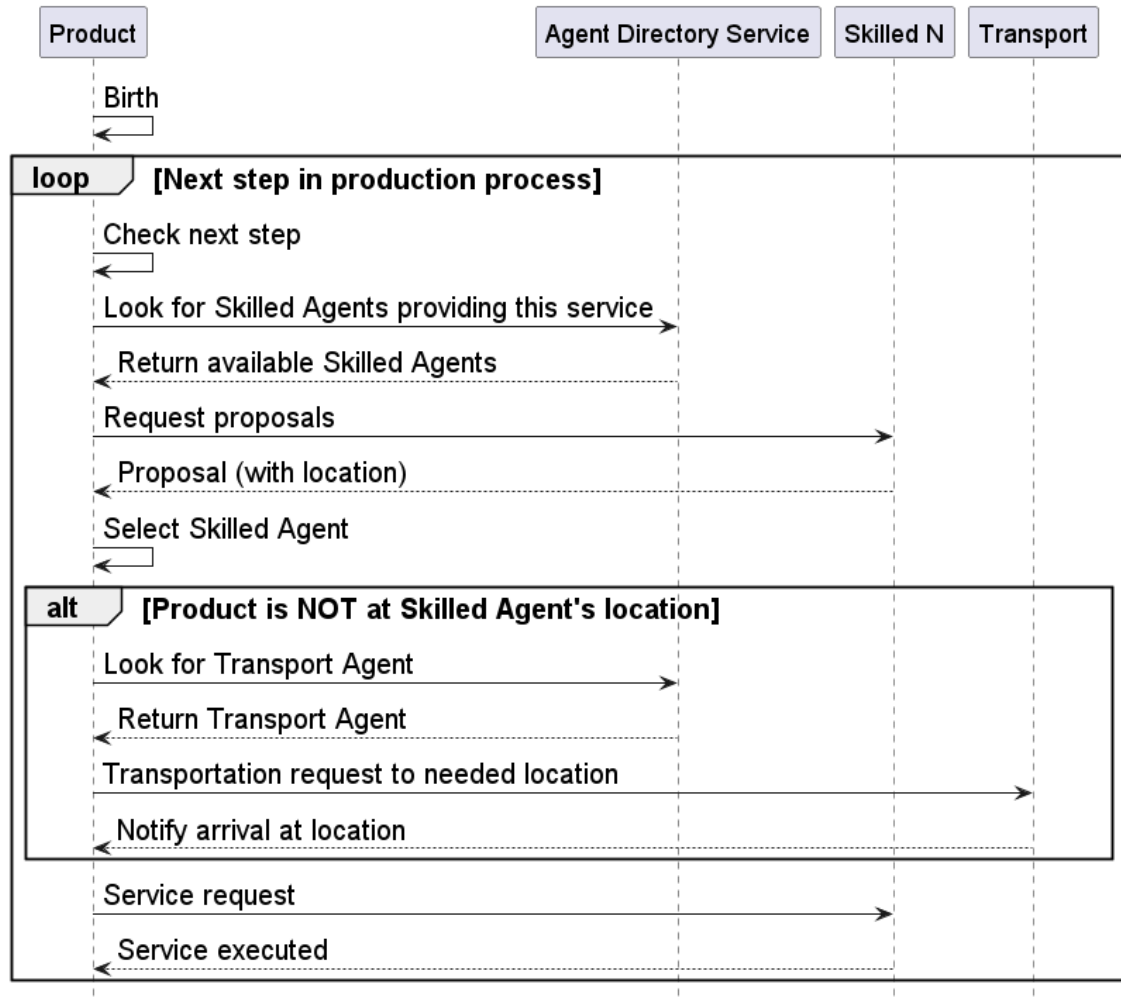


Figure 6.4: Functionality Use Case Sequence Diagram.

- A Raspberry Pi running Python agents, alongside a Python-implemented EMA.
- An ESP32 microcontroller executing C++ agents.

During testing, multiple deployment configurations were orchestrated by launching agents dynamically across different platforms and runtimes. At least two environments must exist (each with an EMA): one hosted on the LC and one on the Raspberry Pi. The ESP32 agent must be associated with one of these environments. One representative configuration, depicted in Figure 6.5, involved the Tower Agent running in Java on the LC, the Product Agent deployed in Python on the Raspberry Pi, and the Transport Agent in C++ on the ESP32 registered with the LC environment.

The complete execution log for this configuration is presented in Listing 6.1. It separately details the agents' instantiation and registration with the EMA, the execution of the first skill, the second skill, the final transport to the exit location, and the Product Agent's deregistration and teardown. The log demonstrates that all agents, regardless of their implementation language or platform, successfully communicated and collaborated

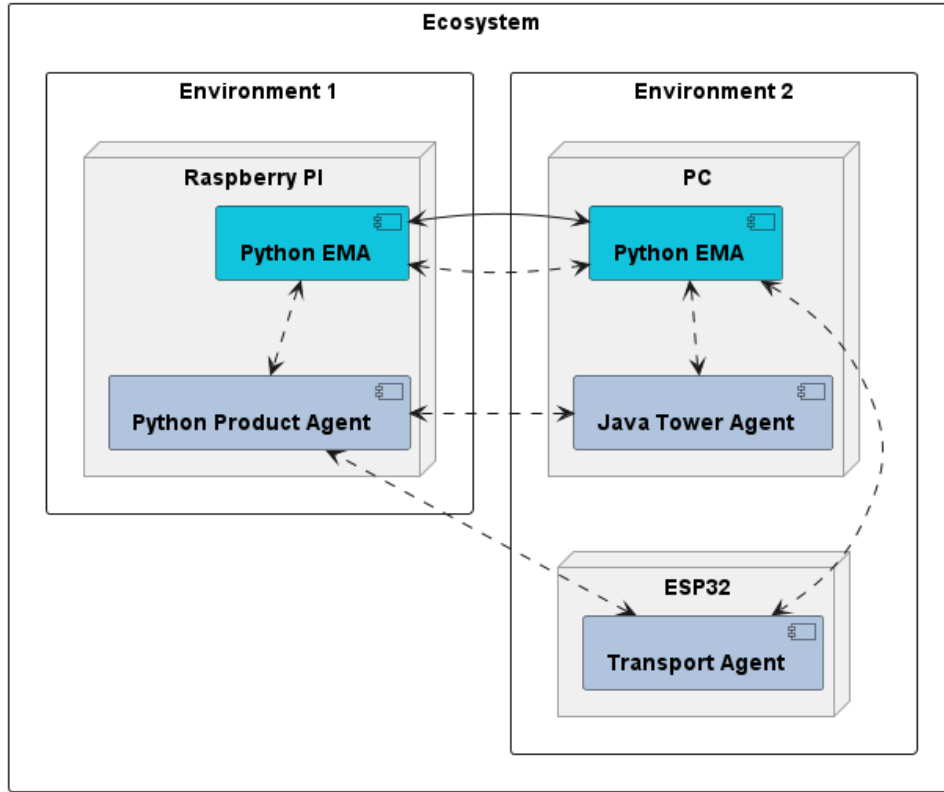


Figure 6.5: Functionality Use Case Components Diagram.

to complete the production process, validating the framework's core objective of true language and platform independence. The logs were formatted and aggregated across all participating devices, with periodic keep-alive messages omitted to enhance readability.

```

1 ////////////// Agents Birth and Registration //////////////
2 [Tower_LC] Agent Born: Tower_LC@192.168.2.90
3 [Tower_LC] Agent active at: 192.168.2.90:4001
4 [Tower_LC] Sent message to EMA (REGISTRATION REQUEST)
5 [Tower_LC] Received message from EMA (INFORM)
6 [Tower_LC] Agent active at: 192.168.2.90:4096
7 [Transport_ESP32] Connecting to WiFi..
8 [Transport_ESP32] WiFi Connected! ESP32 IP: 192.168.250.105
9 [Transport_ESP32] Starting Transport Agent...
10 [Transport_ESP32] Agent active at: 0.0.0.0:4001
11 [Transport_ESP32] Agent Born: Transport_ESP32
12 [Transport_ESP32] Sent message to EMA@192.168.2.90:4000 (REGISTRATION REQUEST)
13 [Transport_ESP32] Received message from EMA (INFORM)
14 [Transport_ESP32] Agent setup completed, started successfully.
15 [Product_RPi] Agent Born: Product_RPi@192.168.4.4:None
16 [Product_RPi] Agent active at: 192.168.4.4:4001
17 [Product_RPi] Sent message to EMA@192.168.4.4:4000 (REGISTRATION REQUEST)
18 [Product_RPi] Received message from EMA@192.168.4.4:4000 (INFORM)
19 [Product_RPi] Agent active at: 192.168.4.4:4013
20
21 ////////////// First Skill //////////////
22 [Product_RPi] Sent message to EMA@192.168.4.4:4000 (SEARCH REQUEST)
23 [Product_RPi] Received message from EMA@192.168.4.4:4000 (INFORM)
24 [Product_RPi] Sent message to Tower_LC@192.168.2.90:4096 (CFP)

```

```
25 [Tower_LC] Received message from Product_RPi (CFP)
26 [Tower_LC] Creating proposal
27 [Tower_LC] Sent message to Product_RPi (PROPOSE)
28 [Product_RPi] Received message from Tower_LC@192.168.2.90:4096 (PROPOSE)
29 [Product_RPi] Sent message to Tower_LC@192.168.2.90:4096 (ACCEPT_PROPOSAL)
30 [Tower_LC] Received message from Product_RPi (ACCEPT_PROPOSAL)
31 [Tower_LC] Proposal Accepted
32 [Tower_LC] Sent message to Product_RPi (INFORM)
33 [Product_RPi] Received message from Tower_LC@192.168.2.90:4096 (INFORM)
34 [Product_RPi] Sent message to EMA@192.168.4.4:4000 (SEARCH REQUEST)
35 [Product_RPi] Received message from EMA@192.168.4.4:4000 (INFORM)
36 [Product_RPi] Sent message to Transport_ESP32@192.168.250.105:4001 (REQUEST)
37 [Transport_ESP32] Received message from Product_RPi (REQUEST)
38 [Transport_ESP32] Received transport request to target location: D (Current location: A)
39 [Transport_ESP32] Sent message to Product_RPi@192.168.4.4:4013 (AGREE)
40 [Product_RPi] Received message from Transport_ESP32@192.168.250.105:4001 (AGREE)
41 [Transport_ESP32] Moving product from A to B
42 [Transport_ESP32] Arrived at B
43 [Transport_ESP32] Moving product from B to C
44 [Transport_ESP32] Arrived at C
45 [Transport_ESP32] Moving product from C to D
46 [Transport_ESP32] Arrived at D
47 [Transport_ESP32] Sent message to Product_RPi@192.168.4.4:4013 (INFORM)
48 [Product_RPi] Received message from Transport_ESP32@192.168.250.105:4001 (INFORM)
49 [Product_RPi] Sent message to Tower_LC@192.168.2.90:4096 (REQUEST)
50 [Tower_LC] Received message from Product_RPi (REQUEST)
51 [Tower_LC] Received REQUEST execute skill
52 [Tower_LC] Sent message to Product_RPi (AGREE)
53 [Product_RPi] Received message from Tower_LC@192.168.2.90:4096 (AGREE)
54 [Tower_LC] Skill executed: Drill
55 [Tower_LC] Sent message to Product_RPi (INFORM)
56 [Product_RPi] Received message from Tower_LC@192.168.2.90:4096 (INFORM)
57
58 ////////// Second Skill //////////
59 [Product_RPi] Sent message to EMA@192.168.4.4:4000 (SEARCH REQUEST)
60 [Product_RPi] Received message from EMA@192.168.4.4:4000 (INFORM)
61 [Product_RPi] Sent message to Tower_LC@192.168.2.90:4096 (CFP)
62 [Tower_LC] Received message from Product_RPi (CFP)
63 [Tower_LC] Creating proposal
64 [Tower_LC] Sent message to Product_RPi (PROPOSE)
65 [Product_RPi] Received message from Tower_LC@192.168.2.90:4096 (PROPOSE)
66 [Product_RPi] Sent message to Tower_LC@192.168.2.90:4096 (ACCEPT_PROPOSAL)
67 [Tower_LC] Received message from Product_RPi (ACCEPT_PROPOSAL)
68 [Tower_LC] Proposal Accepted
69 [Tower_LC] Sent message to Product_RPi (INFORM)
70 [Product_RPi] Received message from Tower_LC@192.168.2.90:4096 (INFORM)
71 [Product_RPi] Sent message to EMA@192.168.4.4:4000 (SEARCH REQUEST)
72 [Product_RPi] Received message from EMA@192.168.4.4:4000 (INFORM)
73 [Product_RPi] Sent message to Transport_ESP32@192.168.250.105:4001 (REQUEST)
74 [Transport_ESP32] Received message from Product_RPi (REQUEST)
75 [Transport_ESP32] Received transport request to target location: D (Current location: D)
76 [Transport_ESP32] Sent message to Product_RPi@192.168.4.4:4013 (AGREE)
77 [Product_RPi] Received message from Transport_ESP32@192.168.250.105:4001 (AGREE)
78 [Transport_ESP32] Product is already at target location: D
79 [Transport_ESP32] Sent message to Product_RPi@192.168.4.4:4013 (INFORM)
80 [Product_RPi] Received message from Transport_ESP32@192.168.250.105:4001 (INFORM)
81 [Product_RPi] Sent message to Tower_LC@192.168.2.90:4096 (REQUEST)
82 [Tower_LC] Received message from Product_RPi (REQUEST)
```

```

83 [Tower_LC] Received REQUEST execute skill
84 [Tower_LC] Sent message to Product_RPi (AGREE)
85 [Product_RPi] Received message from Tower_LC@192.168.2.90:4096 (AGREE)
86 [Tower_LC] Skill executed: Screw
87 [Tower_LC] Sent message to Product_RPi (INFORM)
88 [Product_RPi] Received message from Tower_LC@192.168.2.90:4096 (INFORM)
89
90 ////////////// End Location Transport //////////////
91 [Product_RPi] Sent message to EMA@192.168.4.4:4000 (SEARCH REQUEST)
92 [Product_RPi] Received message from EMA@192.168.4.4:4000 (INFORM)
93 [Product_RPi] Sent message to Transport_ESP32@192.168.250.105:4001 (REQUEST)
94 [Transport_ESP32] Received message from Product_RPi (REQUEST)
95 [Transport_ESP32] Received transport request to target location: A (Current location: D)
96 [Transport_ESP32] Sent message to Product_RPi@192.168.4.4:4013 (AGREE)
97 [Product_RPi] Received message from Transport_ESP32@192.168.250.105:4001 (AGREE)
98 [Transport_ESP32] Moving product from D to E
99 [Transport_ESP32] Arrived at E
100 [Transport_ESP32] Moving product from E to F
101 [Transport_ESP32] Arrived at F
102 [Transport_ESP32] Moving product from F to A
103 [Transport_ESP32] Arrived at A
104 [Transport_ESP32] Sent message to Product_RPi@192.168.4.4:4013 (INFORM)
105 [Product_RPi] Received message from Transport_ESP32@192.168.250.105:4001 (INFORM)
106
107 ////////////// Product Deregistration and Cleanup //////////////
108 [Product_RPi] Sent message to EMA@192.168.4.4:4000 (DEREGISTRATION REQUEST)
109 [Product_RPi] Received message from EMA@192.168.4.4:4000 (INFORM)
110 [Product_RPi] Agent Taken Down

```

Listing 6.1: Functionality Use Case Execution Log.

During various test combinations, the EMA on either the LC or the Raspberry Pi was terminated to evaluate the framework's fault tolerance. For instance, one test evaluated a Python-based configuration with the Transport Agent running on the LC and the Tower Agent on the Raspberry Pi with no Product Agent launched yet, as depicted in Figure 6.6.

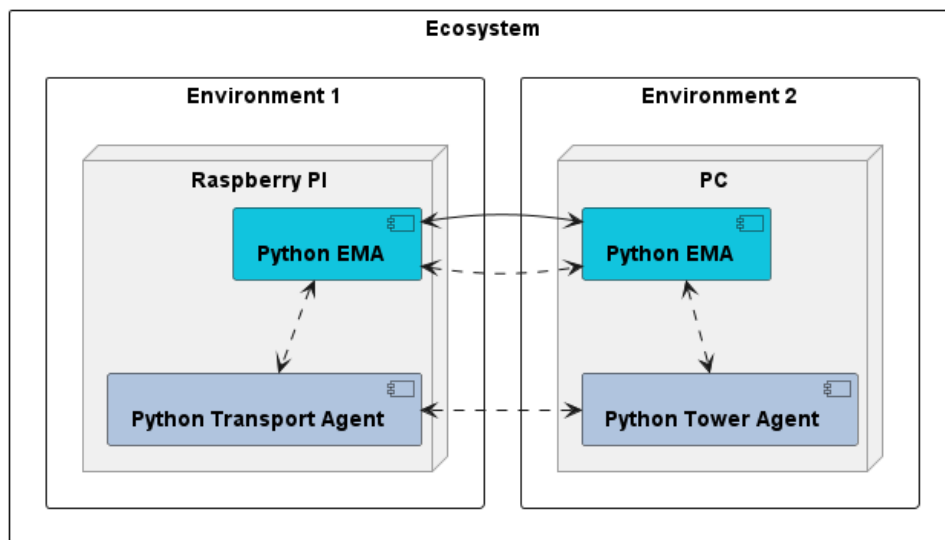


Figure 6.6: EMA Persistence Use Case Components Diagram.

The execution log for a test in which the LC EMA was terminated is presented in Listing 6.2. Following the abrupt termination of the EMA, the Transport Agent in the same environment detected its absence and launched a replacement EMA. This new instance efficiently reconstructed the environment state using incoming agent heartbeats. The log also illustrates the interactions between EMAs, showing that requests are sent whenever one EMA detects a change in another EMA's environment. This log was formatted to enhance readability.

```

1  ////////// 192.168.2.90 EMA Birth //////////
2  [EMA] Agent Born: EMA@192.168.2.90:None
3  [EMA] Agent active at: 192.168.2.90:4000
4
5  [EMA] INTERIOR AGENTS: {'EMA@192.168.2.90:4000'}
6  [EMA] EXTERIOR EMAS: {}
7  [EMA] EXTERIOR AGENTS: {}
8
9  [EMA] Received message from EMA@192.168.4.4:4000 (REQUEST)
10 [EMA] Sent message to EMA@192.168.4.4:4000 (INFORM)
11
12 [EMA] INTERIOR AGENTS: {'EMA@192.168.2.90:4000'}
13 [EMA] EXTERIOR EMAS: {'EMA@192.168.4.4:4000'}
14 [EMA] EXTERIOR AGENTS: {}
15
16 [EMA] Sent message to EMA@192.168.4.4:4000 (REQUEST)
17 [EMA] Received message from EMA@192.168.4.4:4000 (INFORM)
18
19 [EMA] INTERIOR AGENTS: {'EMA@192.168.2.90:4000'}
20 [EMA] EXTERIOR EMAS: {'EMA@192.168.4.4:4000'}
21 [EMA] EXTERIOR AGENTS: {'EMA@192.168.4.4:4000', 'Tower_RPi@192.168.4.4:4096'}
22
23 ////////// Transport Agent Birth //////////
24 [EMA] Received message from Transport_LC@192.168.2.90:4001 (REQUEST)
25 [EMA] Sent message to Transport_LC@192.168.2.90:4001 (INFORM)
26
27 [EMA] INTERIOR AGENTS: {'EMA@192.168.2.90:4000', 'Transport_LC@192.168.2.90:4096'}
28 [EMA] EXTERIOR EMAS: {'EMA@192.168.4.4:4000'}
29 [EMA] EXTERIOR AGENTS: {'EMA@192.168.4.4:4000', 'Tower_RPi@192.168.4.4:4096'}
30
31 [EMA] Received message from Transport_LC@192.168.2.90:4096 (KEEPALIVE)
32 [EMA] Received message from EMA@192.168.4.4:4000 (REQUEST)
33 [EMA] Sent message to EMA@192.168.4.4:4000 (INFORM)
34
35 ////////// 192.168.2.90 EMA Termination and New EMA Launch by Transport Agent //////////
36 [EMA] Agent Born: EMA@192.168.2.90:None
37 [EMA] Agent active at: 192.168.2.90:4000
38
39 [EMA] INTERIOR AGENTS: {'EMA@192.168.2.90:4000'}
40 [EMA] EXTERIOR EMAS: {}
41 [EMA] EXTERIOR AGENTS: {}
42
43 [EMA] Received message from Transport_LC@192.168.2.90:4096 (KEEPALIVE)
44
45 [EMA] INTERIOR AGENTS: {'EMA@192.168.2.90:4000', 'Transport_LC@192.168.2.90:4096'}
46 [EMA] EXTERIOR EMAS: {'EMA@192.168.4.4:4000'}
47 [EMA] EXTERIOR AGENTS: {}
48

```

```

49 [EMA] Sent message to EMA@192.168.4.4:4000 (REQUEST)
50 [EMA] Received message from EMA@192.168.4.4:4000 (INFORM)
51 [EMA] Received message from EMA@192.168.4.4:4000 (REQUEST)
52 [EMA] Sent message to EMA@192.168.4.4:4000 (INFORM)
53
54 [EMA] INTERIOR AGENTS: {'EMA@192.168.2.90:4000', 'Transport_LC@192.168.2.90:4096'}
55 [EMA] EXTERIOR EMAS: {'EMA@192.168.4.4:4000'}
56 [EMA] EXTERIOR AGENTS: {'EMA@192.168.4.4:4000', 'Tower_RPi@192.168.4.4:4096'}
57
58 [EMA] Received message from Transport_LC@192.168.2.90:4096 (KEEPALIVE)

```

Listing 6.2: Functionality Use Case EMA Persistence Log.

Despite the radical differences in hardware architectures and programming paradigms, ranging from dynamic exception handling in Python to manual memory management under FreeRTOS in C++, all agents successfully registered with the EMA, discovered each other, negotiated, and executed tasks. They produced a finished workpiece compliant with specifications while seamlessly recovering from an abrupt EMA termination. These results validate that the TAMAF ecosystem is entirely agnostic to the underlying technology stack, confirming its viability for complex, multi-platform industrial deployments.

6.3 Combined Results Analysis

Synthesizing the findings from both the quantitative stress testing and the qualitative functionality use case demonstrates that TAMAF achieves true technology and platform agnosticism without sacrificing operational communication speed. In the stress testing benchmarks, Java TAMAF maintained a responsive round-trip time under heavy concurrent load, while exhibiting a baseline latency comparable to JADE (6.9 ms vs. 6.7 ms). These results show how TAMAF's decentralized architecture, which enables agents to communicate peer-to-peer directly over raw TCP sockets without being bottlenecked by an intermediary central broker, is not inherently slow and quite competitive. In the physical CPPS use case, this direct communication enabled fluid, responsive FIPA Contract Net and FIPA Request negotiations between the Product, Tower, and Transport Agents without burdening a central coordinator.

The benchmarking results also clearly frame the primary architectural trade-off of TAMAF's decentralized model: agent registration latency. Because every TAMAF agent operates as an independent network entity binding its own TCP port and registering individually with the EMA directory, initialization scales linearly in Java (≈ 15.5 ms marginal cost per agent) and superlinearly in Python ($\approx O(N^{1.66})$), contrasting with JADE's near-instantaneous in-memory container registration (0.105 s). Crucially, this trade-off is well-aligned with industrial automation realities: agent registration is an infrequent lifecycle event performed during commissioning or reconfiguration, whereas operational message exchange occurs continuously throughout production. Accepting a small, predictable startup cost during initial deployment in exchange for halved ongoing messaging latency and zero broker bottlenecks represents an optimal engineering decision

for industrial shop floors. It is also important to note that it is very unlikely that a production system will have more than 200 agents registered to the same EMA, as this would represent an exceptionally dense industrial deployment. At a reduced network scale of $N = 50$ (one quarter of the tested maximum), the registration time drops to 4.89 s and 0.69 s in Python and Java, respectively, representing a substantial reduction compared to the 58.10 s and 2.35 s observed at peak scale ($N = 200$).

Finally, orchestrating agents across a Windows laptop computer, a Raspberry Pi 4, and an ESP32 microcontroller validates the framework against the core research question posed in Chapter 1. The results illustrate the distinct role of each runtime across the automation hierarchy: Java delivers enterprise-grade throughput, Python offers agile development on edge single-board computers, and C++ brings first-class autonomous decision-making directly to bare-metal microcontrollers without the overhead of heavyweight containerization engines. High fault tolerance was also verified with agents launching new EMAs that regain environment state and control, when the active ones abruptly terminate. By validating that heterogeneous runtimes can seamlessly coordinate physical manufacturing processes, TAMAF proves to be a viable, robust, and truly technology-agnostic foundation for decentralized industrial automation.

CONCLUSIONS AND FUTURE WORK

Driven by the paradigms of I4.0, modern manufacturing increasingly demands decentralized, modular, and adaptable architectures capable of dynamically responding to high-mix, low-volume production requirements. While MASs have long served as the primary theoretical framework to address these needs, their practical realization has historically been impeded by technological lock-in, reliance on centralized system management or message brokers, and an inability to operate natively on resource-constrained edge hardware.

This research demonstrates significant progress in overcoming these fundamental challenges. The primary contributions of this work are the formal TAMAF architectural and communication specifications (Chapters 3 and 4) and the empirical validation of three distinct reference implementations targeting Java, Python, and C++ on ESP32 microcontrollers (Chapter 5). The experimental benchmarks and physical testbed results presented in Chapter 6 not only validate that TAMAF enables the development of distributed MASs across heterogeneous software and hardware environments, but also highlight its competitive performance alongside its operational limitations relative to a traditional MAS approach.

7.1 Future Work

Continued research on TAMAF can pave the way for the further development of such architectures. Achieving an architecture that minimizes complexity while maintaining high fault tolerance, industrial robustness, and cross-platform interoperability remains a significant engineering challenge. Building upon the solid foundation established by TAMAF and addressing the limitations identified during development and testing, several key research directions emerge for future work.

While TCP sockets and JSON serialization successfully demonstrated the feasibility of technology-agnostic communication, industrial environments encompass diverse communication requirements. Future iterations of the MTS should support a modular transport layer offering alternative protocols. Lightweight publish/subscribe mechanisms such as MQTT can be integrated for efficient sensor data ingestion; WebSocket channels can

provide persistent, full-duplex communication for web-based operator interfaces; and OPC-UA can be incorporated to establish standardized, vendor-neutral connectivity with industrial PLCs.

The current reference libraries serve as a robust proof of concept, but deploying TAMAF in production environments requires addressing strict industrial dependability and safety standards. Future work should focus on developing hardened, production-grade libraries based on formalized implementation profiles that incorporate comprehensive safety assertions. These should also contain the infrastructure for any operational agent to be able to deploy management services and take control over a terminated EMA.

To achieve comprehensive interoperability, agent and service descriptions must transition toward formal semantic standards. Future work should align TAMAF's asset and skill modeling with reference architectures such as RAMI 4.0 through the AAS concept. Implementing standardized AAS submodel templates for TAMAF agents, operational skills, and device capabilities will ensure semantic interoperability between shop-floor agent behaviors and high-level enterprise systems, such as Manufacturing Execution Systems.

Managing and troubleshooting distributed MASs becomes increasingly complex as the number of interacting agents grows. To enhance developer usability and operational transparency, an integrated inspection infrastructure should be developed. This includes incorporating distributed tracing mechanisms within the MTS to monitor conversation flows. Coupled with an interactive graphical monitoring dashboard, such an infrastructure would provide real-time visibility into agent lifecycle states, active behaviors, communication traffic, and dynamic reconfigurations.

Expanding industrial validation is paramount, as only two of the three implementations underwent rigorous stress testing, and the functional evaluation was limited to a single scenario. Future research should evaluate TAMAF within a full-scale, multi-stage industrial production facility for robustness, versatility, and usability, while extensively benchmarking the ESP32 implementation and Raspberry Pi deployments. This may yield additional insights into the deployment of TAMAF on resource-constrained platforms.

The framework also presents a compelling opportunity to explore emerging coordination paradigms within the field of AI agents, acting as a modern foundation for implementing MASs. Specifically, it can offer a more robust mechanism for orchestrating interactions among agents, with an emphasis on decentralized service discovery.

To maximize the practical impact of this thesis, the TAMAF reference libraries, and validation test suites will be released as open-source software here. Accompanied by comprehensive documentation, this release is intended to foster a collaborative ecosystem. By lowering the barrier to entry for both academic researchers and industrial practitioners, TAMAF can evolve into an open, community-driven standard that accelerates the adoption of decentralized intelligence in smart manufacturing.

In conclusion, this dissertation demonstrates that the long-standing divide between high-level multi-agent deliberation and low-level industrial control can be bridged without

sacrificing autonomy, performance, or simplicity. By stripping away platform dependencies and monolithic broker architectures, TAMAF provides a scalable, resilient, and versatile foundation for the next generation of CPPSs. As manufacturing continues its inevitable progression toward distributed, collaborative, and self-optimizing ecosystems, TAMAF stands as a proven architectural enabler, bringing intelligent agent coordination directly to the hardware where physical production takes place.

BIBLIOGRAPHY

- [1] The L^AT_EX Project team. *L^AT_EX – A document preparation system*. 1985. URL: <https://www.latex-project.org> (cit. on p. xv).
- [2] J. M. Lourenço. *The aidisclose package: Generative AI disclosure checklist and statements*. Version 1.12.0. 2025. URL: <https://ctan.org/pkg/aidisclose/aidisclose-doc.pdf> (cit. on p. xv).
- [3] Y. Suchikova, N. Tsybuliak, J. A. T. da Silva, and S. Nazarovets. “GAIDeT (Generative AI Delegation Taxonomy): A Taxonomy for Humans to Delegate Tasks to Generative Artificial Intelligence in Scientific Research and Publishing”. In: *Accountability in Research* 33.3 (2026). DOI: 10.1080/08989621.2025.2544331 (cit. on p. xv).
- [4] K. Henning. “Recommendations for Implementing the Strategic Initiative INDUSTRIE 4.0”. In: 2013-04-01. URL: <https://www.semanticscholar.org/paper/Recommendations-for-implementing-the-strategic-4.0-Henning/80d4b3eccd6951cf6dba61f1b33889a0edbf0407> (visited on 2026-01-10) (cit. on pp. 1, 5).
- [5] D. Sinha and R. Roy. “Reviewing Cyber-Physical System as a Part of Smart Factory in Industry 4.0”. In: *IEEE Engineering Management Review* 48.2 (2020), pp. 103–117. ISSN: 1937-4178. DOI: 10.1109/EMR.2020.2992606 (cit. on pp. 1, 9, 10).
- [6] R. F. Babiceanu and F. F. Chen. “Development and Applications of Holonic Manufacturing Systems: A Survey”. In: *Journal of Intelligent Manufacturing* 17.1 (2006-02-01), pp. 111–131. ISSN: 1572-8145. DOI: 10.1007/s10845-005-5516-y (cit. on pp. 1, 7, 14).
- [7] O. Antons and J. C. Arlinghaus. “Distributed Control for Industry 4.0 -A Comparative Simulation Study”. In: *IFAC-PapersOnLine*. 17th IFAC Symposium on Information Control Problems in Manufacturing INCOM 2021 54.1 (2021-01-01), pp. 516–521. ISSN: 2405-8963. DOI: 10.1016/j.ifacol.2021.08.059 (cit. on pp. 1, 12, 13).
- [8] T. Pulikottil, L. A. Estrada-Jimenez, H. Ur Rehman, F. Mo, S. Nikghadam-Hojjati, and J. Barata. “Agent-Based Manufacturing — Review and Expert Evaluation”. In: *The International Journal of Advanced Manufacturing Technology* 127.5 (2023-07-01), pp. 2151–2180. ISSN: 1433-3015. DOI: 10.1007/s00170-023-11517-8 (cit. on pp. 1, 14, 19).

- [9] K. Zhang, Y. Shi, S. Karnouskos, T. Sauter, H. Fang, and A. W. Colombo. “Advancements in Industrial Cyber-Physical Systems: An Overview and Perspectives”. In: *IEEE Transactions on Industrial Informatics* 19.1 (2023-01), pp. 716–729. ISSN: 1941-0050. DOI: 10.1109/TII.2022.3199481 (cit. on pp. 1, 9, 10).
- [10] Foundation for Intelligent Physical Agents Foundation. *FIPA Specifications and Standards*. URL: <http://www.fipa.org> (visited on 2025-10-07) (cit. on pp. 1, 15, 16).
- [11] K. Gan, C. On, P. Anthony, and A. Hamdan. “A FIPA-ACL Ontology in Enhancing Interoperability Multi-agent Communication”. In: 2018-02-24, pp. 151–160. ISBN: 978-981-10-8275-7. DOI: 10.1007/978-981-10-8276-4_15 (cit. on pp. 1, 16, 20).
- [12] Z. Wrona, W. Buchwald, M. Ganzha, M. Paprzycki, F. Leon, N. Noor, and C.-V. Pal. “Overview of Software Agent Platforms Available in 2023”. In: *Information* 14.6 (2023-06), p. 348. ISSN: 2078-2489. DOI: 10.3390/info14060348 (cit. on pp. 1, 18).
- [13] F. Bellifemine, G. Caire, A. Poggi, and G. Rimassa. “JADE: A Software Framework for Developing Multi-Agent Applications. Lessons Learned”. In: *Information and Software Technology*. Special Issue with Two Special Sections. Section 1: Most-cited Software Engineering Articles in 2001. Section 2: Requirement Engineering: Foundation for Software Quality 50.1 (2008-01-01), pp. 10–21. ISSN: 0950-5849. DOI: 10.1016/j.infsof.2007.10.008 (cit. on pp. 1, 19).
- [14] L. S. Melo, R. F. Sampaio, R. P. S. Leão, G. C. Barroso, and J. R. Bezerra. “Python-Based Multi-Agent Platform for Application on Power Grids”. In: *International Transactions on Electrical Energy Systems* 29.6 (2019), e12012. ISSN: 2050-7038. DOI: 10.1002/2050-7038.12012 (cit. on pp. 2, 19).
- [15] J. Alves, P. Sousa, T. Cruz, and J. Mendes. “A Review of Architecture Features for Distributed and Resilient Industrial Cyber-Physical Systems”. In: *Journal of Manufacturing Systems* 82 (2025-10-01), pp. 1069–1090. ISSN: 0278-6125. DOI: 10.1016/j.jmsy.2025.07.012 (cit. on pp. 2, 11–14).
- [16] B. Acar, T. Küster, O. F. Kupke, R. K. Strehlow, M. G. Augusto, F. Sivrikaya, and S. Albayrak. “OPACA: Toward an Open, Language- and Platform-Independent API for Containerized Agents”. In: *IEEE Access* 12 (2024), pp. 10012–10022. ISSN: 2169-3536. DOI: 10.1109/ACCESS.2024.3353613 (cit. on pp. 2, 20, 21).
- [17] G. Herrera-Vidal, J. R. Coronado-Hernández, and J. Maheut. “Complexity Management Challenges in the Industry 4.0 Era: A Systematic Review in Production Systems”. In: *Results in Engineering* 26 (2025-06-01), p. 105329. ISSN: 2590-1230. DOI: 10.1016/j.rineng.2025.105329 (cit. on pp. 2, 8, 11).
- [18] M. Seitz, F. Gehlhoff, L. A. Cruz Salazar, A. Fay, and B. Vogel-Heuser. “Automation Platform Independent Multi-Agent System for Robust Networks of Production Resources in Industry 4.0”. In: *Journal of Intelligent Manufacturing* 32.7 (2021-10-01), pp. 2023–2041. ISSN: 1572-8145. DOI: 10.1007/s10845-021-01759-2 (cit. on pp. 2, 20).

- [19] S. Vaidya, P. Ambad, and S. Bhosle. "Industry 4.0 – A Glimpse". In: *Procedia Manufacturing*. 2nd International Conference on Materials, Manufacturing and Design Engineering (iCMMD2017), 11-12 December 2017, MIT Aurangabad, Maharashtra, INDIA 20 (2018-01-01), pp. 233–238. ISSN: 2351-9789. DOI: 10.1016/j.promfg.2018.02.034 (cit. on p. 5).
- [20] N. Anumbe, C. Saidy, and R. Harik. "A Primer on the Factories of the Future". In: *Sensors* 22.15 (2022-01), p. 5834. ISSN: 1424-8220. DOI: 10.3390/s22155834 (cit. on pp. 6, 8, 10).
- [21] M. Ghobakhloo, M. Fathi, M. Iranmanesh, P. Maroufkhani, and A. Et. "Industry 4.0 Ten Years on: A Bibliometric and Systematic Review of Concepts, Sustainability Value Drivers, and Success Determinants". In: *Journal of Cleaner Production* (2021). DOI: 10.1016/j.jclepro.2021.127052 (cit. on pp. 6, 7).
- [22] M. Soori, B. Arezoo, and R. Dastres. "Internet of Things for Smart Factories in Industry 4.0, a Review". In: *Internet of Things and Cyber-Physical Systems* 3 (2023-01-01), pp. 192–204. ISSN: 2667-3452. DOI: 10.1016/j.iotcps.2023.04.006 (cit. on p. 6).
- [23] Š. Mozol, P. Grznár, and M. Oravec. "Manufacturing Paradigms and Their Change in Time". In: 2020-01-01, pp. 163–168. DOI: 10.24132/PI.2020.09693.163–168 (cit. on p. 6).
- [24] DeepShah. "A Comparative Study on Cloud, Fog and Edge Computing". In: *2021 5th International Conference on Electrical, Electronics, Communication, Computer Technologies and Optimization Techniques (ICEECCOT)*. 2021 5th International Conference on Electrical, Electronics, Communication, Computer Technologies and Optimization Techniques (ICEECCOT). 2021-12, pp. 501–507. DOI: 10.1109/ICEECCOT52851.2021.9707931 (cit. on p. 7).
- [25] B. Tank and V. Gandhi. "A Comparative Study on Cloud Computing, Edge Computing and Fog Computing". In: *Recent Developments in Electronics and Communication Systems*. IOS Press, 2023, pp. 665–670. DOI: 10.3233/ATDE221329 (cit. on p. 7).
- [26] H. Kagermann and W. Wahlster. "Ten Years of Industrie 4.0". In: *Sci* 4.3 (2022-09), p. 26. ISSN: 2413-4155. DOI: 10.3390/sci4030026 (cit. on p. 9).
- [27] I. Onaji, D. Tiwari, P. Soulatiantork, B. Song, and A. Tiwari. "Digital Twin in Manufacturing: Conceptual Framework and Case Studies". In: *International Journal of Computer Integrated Manufacturing* 35.8 (2022-08-03), pp. 831–858. ISSN: 0951-192X. DOI: 10.1080/0951192X.2022.2027014 (cit. on p. 9).
- [28] A. Alexopoulos, G. Kalogeras, K. Koutras, and A. Kalogeras. "Why Asset Administration Shells: A Survey on Uses and Challenges". In: *IEEE Access* 13 (2025), pp. 126582–126609. ISSN: 2169-3536. DOI: 10.1109/ACCESS.2025.3589931 (cit. on p. 9).

- [29] T. Müller, N. Jazdi, J.-P. Schmidt, and M. Weyrich. “Cyber-Physical Production Systems: Enhancement with a Self-Organized Reconfiguration Management”. In: *Procedia CIRP*. 14th CIRP Conference on Intelligent Computation in Manufacturing Engineering, 15-17 July 2020 99 (2021-01-01), pp. 549–554. ISSN: 2212-8271. DOI: 10.1016/j.procir.2021.03.075 (cit. on p. 10).
- [30] A. Ma, A. Nassehi, and C. Snider. “Anarchic Manufacturing: Implementing Fully Distributed Control and Planning in Assembly”. In: *Production & Manufacturing Research* 9.1 (2021-01-01), pp. 56–80. ISSN: null. DOI: 10.1080/21693277.2021.1963346 (cit. on p. 13).
- [31] C. Vetorazzi and G. Telles. “The Holonic System Paradigm [for Manufacturing]”. In: *PICMET '01. Portland International Conference on Management of Engineering and Technology. Proceedings Vol.1: Book of Summaries*. PICMET '01. Portland International Conference on Management of Engineering and Technology Vol.1: Book of Summaries. Vol. 1. 2001-07, 88 vol.1-. DOI: 10.1109/PICMET.2001.951791 (cit. on p. 14).
- [32] M. Ahmed, O. Kazar, and S. Harous. “Cyber-Physical System Model Based on Multi-Agent System”. In: *IET Cyber-Physical Systems: Theory & Applications* 9.4 (2024), pp. 424–434. ISSN: 2398-3396. DOI: 10.1049/cps2.12096 (cit. on p. 14).
- [33] J. Fan, L. Yue, and S. Zheng. “A Hierarchical Structure of Cyber-Physical Production Systems Based on Heterogeneous Multi-Agent Systems”. In: *2022 China Automation Congress (CAC)*. 2022 China Automation Congress (CAC). 2022-11, pp. 4716–4720. DOI: 10.1109/CAC57257.2022.10055603 (cit. on p. 15).
- [34] A. D. Rocha, J. Tripa, D. Alemão, R. S. Peres, and J. Barata. “Agent-Based Plug and Produce Cyber-Physical Production System – Test Case”. In: *2019 IEEE 17th International Conference on Industrial Informatics (INDIN)*. 2019 IEEE 17th International Conference on Industrial Informatics (INDIN). Vol. 1. 2019-07, pp. 1545–1551. DOI: 10.1109/INDIN41052.2019.8972169 (cit. on p. 15).
- [35] L. D. Silva, F. Meneguzzi, and B. Logan. “BDI Agent Architectures: A Survey”. In: *Proceedings of the Twenty-Ninth International Joint Conference on Artificial Intelligence*. Twenty-Ninth International Joint Conference on Artificial Intelligence and Seventeenth Pacific Rim International Conference on Artificial Intelligence {IJCAI-PRICAI-20}. Yokohama, Japan: International Joint Conferences on Artificial Intelligence Organization, 2020-07, pp. 4914–4921. ISBN: 978-0-9992411-6-5. DOI: 10.24963/ijcai.2020/684 (cit. on p. 15).
- [36] Foundation for Intelligent Physical Agents. *FIPA Abstract Architecture Specification*. Specification SC00001L. IEEE / FIPA, 2002 (cit. on pp. 15, 31).
- [37] Foundation for Intelligent Physical Agents. *FIPA Agent Management Specification*. Specification SC00023K. IEEE / FIPA, 2002 (cit. on p. 15).

- [38] T. Finin, R. Fritzson, D. McKay, and R. McEntire. "KQML as an Agent Communication Language". In: *Proceedings of the Third International Conference on Information and Knowledge Management - CIKM '94*. The Third International Conference. Gaithersburg, Maryland, United States: ACM Press, 1994, pp. 456–463. ISBN: 978-0-89791-674-5. DOI: 10.1145/191246.191322 (cit. on p. 16).
- [39] Foundation for Intelligent Physical Agents. *FIPA ACL Message Structure Specification*. Specification SC00061G. IEEE / FIPA, 2002 (cit. on p. 17).
- [40] Foundation for Intelligent Physical Agents. *FIPA Request Interaction Protocol Specification*. Specification SC00026H. IEEE / FIPA, 2002 (cit. on p. 17).
- [41] Foundation for Intelligent Physical Agents. *FIPA Contract Net Interaction Protocol Specification*. Specification SC00029H. IEEE / FIPA, 2002 (cit. on p. 18).
- [42] B. K. de Freitas, L. Fritzen Venturini, M. A. Domingues, M. Augusto da Rosa, and D. Issicaba. "Exploiting PADE to the Simulation of Multiagent Restoration Actions". In: *2019 11th International Symposium on Advanced Topics in Electrical Engineering (ATEE)*. 2019 11th International Symposium on Advanced Topics in Electrical Engineering (ATEE). 2019-03, pp. 1–5. DOI: 10.1109/ATEE.2019.8724852 (cit. on p. 19).
- [43] J. Palanca, J. Rincon, V. Julian, C. Carrascosa, and A. Terrasa. "Developing IoT Artifacts in a MAS Platform". In: *Electronics* 11.4 (2022-01), p. 655. ISSN: 2079-9292. DOI: 10.3390/electronics11040655 (cit. on pp. 19, 20).
- [44] S. Al-Agtash, X. Xian, K. Petroschchuk, H. Syeddah, and H. B. Mallampati. "Developing Microgrid Agents U Sing Smart Python Agent Development Environment (SPADE)". In: *2023 Congress in Computer Science, Computer Engineering, & Applied Computing (CSCE)*. 2023 Congress in Computer Science, Computer Engineering, & Applied Computing (CSCE). 2023-07, pp. 2683–2686. DOI: 10.1109/CSCE60160.2023.00428 (cit. on p. 19).
- [45] F. Bergenti, G. Caire, S. Monica, and A. Poggi. "The First Twenty Years of Agent-Based Software Development with JADE". In: *Autonomous Agents and Multi-Agent Systems* 34.2 (2020-05-11), p. 36. ISSN: 1573-7454. DOI: 10.1007/s10458-020-09460-z (cit. on p. 19).
- [46] D. Briola, G. Vizzari, and M. Montinaro. "Enhancing Testing of MASs with a Simulator of JADE". In: *ECAI 2025*. IOS Press, 2025, pp. 3687–3694. DOI: 10.3233/FAIA251247 (cit. on p. 19).
- [47] "IEEE Recommended Practice for Industrial Agents: Integration of Software Agents and Low-Level Automation Functions". In: *IEEE Std 2660.1-2020* (2021-01), pp. 1–43. DOI: 10.1109/IEEESTD.2021.9340089 (cit. on p. 20).

- [48] J. Jirsa, F. Zezulka, P. Marcoň, T. Pečinka, L. Nováček, V. Kaczmarczyk, and J. Arm. “Use of Multi-Agent System for Industrial Production Control”. In: *IFAC-PapersOnLine*. 18th IFAC Conference on Programmable Devices and Embedded Systems PDES 2024 58.9 (2024-01-01), pp. 205–210. ISSN: 2405-8963. DOI: 10.1016/j.ifacol.2024.07.397 (cit. on p. 20).
- [49] S. Komesker, W. Motsch, J. Popper, A. Sidorenko, A. Wagner, and M. Ruskowski. “Enabling a Multi-Agent System for Resilient Production Flow in Modular Production Systems”. In: *Procedia CIRP*. Leading Manufacturing Systems Transformation – Proceedings of the 55th CIRP Conference on Manufacturing Systems 2022 107 (2022-01-01), pp. 991–998. ISSN: 2212-8271. DOI: 10.1016/j.procir.2022.05.097 (cit. on p. 20).
- [50] F. M. Oliveira, A. Rocha, D. Alemão, N. Freitas, and J. Barata. “Asynchronous Communication Between Modular Cyber-Physical Production Systems and Arduino Based Industrial Controllers”. In: *Technological Innovation for Digitalization and Virtualization*. Ed. by L. M. Camarinha-Matos. Vol. 649. Cham: Springer International Publishing, 2022, pp. 53–61. ISBN: 978-3-031-07519-3 978-3-031-07520-9. DOI: 10.1007/978-3-031-07520-9_5 (cit. on p. 20).
- [51] J. M. Lourenço. *novathesis: A LaTeX Template for Academic Theses and Dissertations*. Version 8.3.1. NOVA University Lisbon. 2026. DOI: 10.5281/zenodo.21938603.

